ASSIGNMENT 3-QUESTIONS

CO3: Examine cryptographic hash algorithms and digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication and non-repudiation.

Q1 – What Makes a Hash Function Trustworthy? (6 Marks)
Part A – Avalanche and Fixed-Length Digests
Shared Group Work Together:

- Understand the required properties of a cryptographic hash (pre-image, second pre-image and collision resistance, avalanche effect)
- Implement a deliberately weak toy hash (for example, a byte-sum or XOR-fold hash) alongside a standard hash (SHA-256 from a library, used for comparison only)
- Decide how "avalanche" will be measured
- Run and test the experiment

Student A – Design & Mathematical Analysis
Student A must individually:

- Explain what each property means as an attack scenario
- Derive the expected number of output bits that change when one input bit is flipped in an ideal hash
- Explain mathematically why the toy hash fails the avalanche and second pre-image properties
- Explain why a fixed-length digest must have collisions

Student B – Experimental Validation
Student B must individually:

- Design the experimental inputs (single-bit flips, similar messages, long and short messages)
- Instrument the code to count the number of changed output bits
- Test at least three message sizes and a minimum number of trials per size
- Compare predicted and observed bit-change averages
- Explain any discrepancy

Part B – Collision Effort and the Birthday Bound
Shared Group Work Together, build a collision finder for a truncated hash (take only the first k bits of SHA-256, for k = 16, 20, 24, 28, 32).
Student A – Experimental Investigation
Student A must:

- Design the collision-search runs and repeat each several times
- Record the number of hashes computed before the first collision
- Investigate increasing values of k
- Present the experimental evidence in a table and graph

Student B – Birthday-Bound Reasoning

Student B must:

- Derive why the expected effort is roughly 2^(k/2), not 2^k
- Explain the difference between finding any collision and finding a collision with a specific message
- Explain how this determines the minimum safe digest length
- Relate the result to the retirement of MD5 and SHA-1

Individual Verification

Faculty may ask Student A to explain the birthday bound and Student B to explain the avalanche experiment. Primary responsibility does not mean exclusive knowledge.

Q2 – Integrity Is Not the Same as Authentication: Hash, MAC and HMAC (7 Marks)

Problem Context

A software distributor publishes a file together with its SHA-256 digest on the same web page. Another company protects internal messages with the construction H(key || message).

Shared Group Work

The pair must:

- Implement a simple Merkle–Damgård style toy hash (fixed block size, chaining value, padding with length)
- Implement the naive MAC H(key || message) and the HMAC construction
- Design tamper scenarios (modified message, modified message and recomputed digest, appended data)
- Conduct the experiments

Student A – Attack and Construction Investigation

Student A is individually responsible for:

- Showing that an attacker who can change both the file and the published digest defeats a plain hash
- Demonstrating a length-extension attack on the naive H(key || message) using your toy Merkle–Damgård hash
- Recording what the attacker needs to know and what the attacker does not need to know
- Explaining which property of the chaining construction makes the attack possible

Student B – Comparative Security Analysis

Student B is individually responsible for:

Explaining the structure of HMAC (inner and outer hash, key padding) and why it resists length extension

- Comparing hash-only, naive MAC and HMAC in a table: which provides integrity, which provides authentication, and which provides neither
- Explaining why a MAC cannot provide non-repudiation
- Verifying through tests that HMAC detects every tamper scenario you designed Joint Design Question

Together answer:

If you were designing the download-verification and message-protection components for an organisation, which mechanism would you choose for each, what must be kept secret, and what residual risk remains?
Both students must be able to defend the answer.

Q3 – Digital Signatures: Hash-then-Sign and What Can Break Them (7 Marks)
Shared Group Work
Together:

- Implement RSA signature generation and verification using hash-then-sign (small keys for measurement)
- Implement a DSA-style (or ECDSA-style) signature using toy parameters
- Test valid signatures, modified messages, modified signatures and wrong public keys
- Conduct the nonce experiment described below

Student A – Algorithmic Derivation
Student A must individually:

- Explain why the message is hashed before signing and what would go wrong with signing the raw message
- Prove why verification succeeds for a correctly generated RSA signature (s = H(m)^d mod n and s^e ≡ H(m) mod n)
- Derive the DSA/ECDSA verification equation and show why it recovers the signer's value
- Explain what security goal each step contributes (integrity, authentication, non-repudiation)

Student B – Attack Experiment and Evidence
Student B must individually:

- Design the test messages and tamper cases and record accept/reject results
- Demonstrate nonce reuse in DSA/ECDSA: sign two messages with the same per-signature secret k, then recover k and the private key from the two signatures
- Demonstrate a textbook (no hash) RSA signature forgery (for example, multiplicative property: signature of $m_1 \cdot m_2$ from signatures of m_1 and m_2)
- Explain what these results show about "correct maths but unsafe implementation"

Joint Conclusion
Together answer:
A valid signature proves that someone holding the private key signed the message. Under what conditions does this actually amount to non-repudiation, and what could the signer claim to deny it? Consider key compromise, timestamping, and the role of a trusted third party.

Q4 – Trust Through Certificates: X.509 Chain Validation (7 Marks)
Here the responsibilities are deliberately reversed so that the same student does not always perform the theoretical component.
Shared Group Work
Together:

- Study the structure of an X.509 v3 certificate (subject, issuer, serial number, validity, public key, extensions such as key usage and basic constraints, signature)
- Build a small three-level hierarchy (root CA → intermediate CA → end-entity) using a library or a toolkit such as OpenSSL
- Implement a chain-validation procedure that follows the steps a client must perform
- Verify the procedure on valid and invalid chains

Student B – Trust Model & Validation Logic

Student B must individually:

- Write the complete validation algorithm in pseudocode (signature check on each link, issuer–subject matching, validity period, basic-constraints CA flag, key usage, name matching, revocation check, anchoring at a trusted root)
- Explain what each step protects against
- Explain why trust in the root cannot itself be verified cryptographically
- Explain how X.509 binds an identity to a public key and why that binding is what makes a signature meaningful

Student A – Failure-Case Experiments

Student A must individually:

- Design at least six failure cases: expired certificate, not-yet-valid certificate, wrong issuer, tampered certificate field, hostname mismatch, end-entity certificate used as a CA, revoked certificate
- Record which step of validation rejects each case
- Compare CRL and OCSP (freshness, size, privacy, availability) and show a case in which revocation is detected only if the check is actually performed
- Discuss what a "fail-open" revocation policy means for security

Joint Engineering Decision

Together determine:

If a CA is compromised or a certificate is mis-issued, what limits the damage, how long can the damage persist, and what would you change in a deployment to shorten that window?

Q5 – Kerberos: Authentication Without Sending Passwords (6 Marks)

Shared Group Work

Together:

- Study the Kerberos message flow (Authentication Server exchange, Ticket-Granting Server exchange, Client–Server exchange)
- Build a simulation with a client, KDC (AS + TGS) and one service, using symmetric encryption and authenticated tickets
- Include timestamps, ticket lifetimes, session keys and authenticators
- Test the normal flow and the attack/failure cases below

Student A – Protocol Design

Student A must individually:

- Draw the message-sequence diagram and label what is encrypted with which key

- Explain what the Ticket-Granting Ticket is for and why the user's long-term key is used only once per session
- Explain the purpose of the authenticator and the timestamp
- Explain how mutual authentication is achieved
- Identify which party can read which part of each message

Student B – Attack and Failure Evidence

Student B must individually:

- Demonstrate a replay of a captured authenticator with and without replay protection
- Demonstrate an expired ticket and a clock-skew failure
- Demonstrate what happens if the KDC is unavailable
- Record accept/reject outcomes and the reason for each
- Explain what a stolen ticket or an offline password-guessing attack on the AS reply would allow

Cross-Understanding Requirement

Student B must be able to explain:

Why does the client never send its password across the network, and what does the service learn about the client?

Student A must be able to explain:

What experimental evidence shows that the timestamp/authenticator mechanism actually prevents replay? This deliberately tests understanding across the assigned responsibilities.

Q6 – Advanced Security Mini-Investigations: RSA, DSA/DSS, ECDSA, ElGamal and Schnorr (7 Marks)

This question provides particularly clear individual evidence.

Student A – Hash Weakness and Signature Forgery

Student A takes primary responsibility for the collision-to-forgery investigation.

The student must:

- Explain the difference between a collision attack and a second pre-image attack
- Using your truncated-hash collision finder from Q1, build two different messages with the same short digest and show that a signature on one is valid for the other (use a signature scheme over the truncated digest)
- Explain why this attack requires the attacker to prepare both messages in advance
- Explain how a collision in MD5/SHA-1 enabled real-world forged certificates and what this means for X.509
- Explain the role of digest length and algorithm choice in deciding what a CA may use
- Recommend a hash policy for signatures and justify it

Student B Verification

Student B must provide a short explanation of:

Why does weakening the hash function weaken the signature, even when the signature algorithm itself (RSA/DSA) is unchanged?

Student B – Kerberos versus X.509 and Non-Repudiation

Student B takes primary responsibility for the framework comparison investigation.
The student must:

- Compare Kerberos and X.509/PKI in a table: key type, trusted third party, online/offline dependence, scalability, revocation, typical environment
- Explain which framework gives non-repudiation and which does not, and why (shared secret vs private key held by one party)
- Analyse the single point of failure of Kerberos and the trust-anchor risk in X.509
- Compare the number of key relationships required in a network of n users for pairwise shared keys, Kerberos with a KDC, and PKI
- Recommend which framework you would use for (i) a campus single sign-on system and (ii) signing legally binding electronic contracts

Student A Verification

Student A must provide a short explanation of:

Why can a Kerberos ticket prove who a user is to a service, yet not prove to a third party later that the user performed an action?

Thus, Q6 provides different primary tasks but mandatory cross-understanding.

Evidence Expected: worked toy example, verification trace, attack/failure cases and a comparison table covering RSA, DSA/DSS, ElGamal and Schnorr.

• Comparison requirement: Compare RSA, DSA/DSS, ElGamal and Schnorr signatures with respect to key material, randomness/nonce requirements, verification, forgery resistance and non-repudiation assumptions.

• Schnorr protocol: Describe the commitment–challenge–response sequence, derive the verification relation for toy parameters, and explain why the challenge must be unpredictable and why nonce reuse is dangerous.

• ElGamal signature scheme: Derive key generation, signing and verification for small toy parameters; test a valid signature and at least two altered-message/signature cases; explain how poor handling of signing randomness can weaken security.

Q7 – Design a Secure Authentication and Integrity Scheme from First Principles (10 Marks)

Problem Context – Secure Over-the-Air Update for Medical Devices

A manufacturer distributes firmware updates to thousands of infusion pumps in hospitals. Each pump must be sure that an update (a) has not been altered, (b) really comes from the manufacturer, and (c) the manufacturer cannot later deny having issued it. Pumps have limited memory, are often offline, and have unreliable clocks. Some pumps must also authenticate to the hospital's central server before accepting a service session.

Stage 1 – Independent Design Proposal

Before discussing the solution with each other:

Student A independently proposes:

- Which security goals each mechanism will address
- Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used and where

- What key or certificate material each party holds
- A possible message flow for one update

Student B independently proposes:

- Which security goals each mechanism will address
- Which primitives (hash, MAC, signature, certificate, Kerberos-style ticket) will be used and where
- What key or certificate material each party holds
- A possible message flow for one update

Each student's initial proposal must be retained in the submission. No marks are deducted merely because an initial design is incorrect. The purpose is to provide evidence of independent design thinking.

Stage 2 – Compare the Two Designs

The pair must now compare the two independent proposals. Discuss:

- Where the designs agree?
- Where they differ?
- Strengths and weaknesses of each
- Missing threats or missing cases (rollback to an old version, stolen signing key, offline pump, wrong clock, revoked key)
- Which ideas should be retained?

Evidence Expected: A short design-comparison record showing how the two initial ideas influenced the final solution.

Stage 3 – Joint Final Design

Using the strongest ideas from the discussion, jointly develop the final scheme. The final design must identify:

- Threat model (who the attacker is and what the attacker can do)
- Security goals and the mechanism that achieves each
- Key and certificate management (issuance, storage, rotation, revocation)
- Message flow / protocol (diagram) for signing, distributing and verifying an update
- Behaviour when the pump is offline or its clock is wrong
- Complete pseudocode for signing and for verification

Stage 4 – Divide the Validation Responsibilities

Student A – Security Argument Validation

Student A must:

- State what each attack would need to succeed (e.g., 2^(n/2) work for a collision, forging a signature, compromising a key)
- Show, goal by goal, why the final design prevents or limits each attack
- Estimate the attacker's effort for your chosen digest and key sizes using the results from Q1 and Q3
- Identify the residual risks that remain after the design

Student B – Experimental Validation

Student B must:

- Design correctness and attack test cases (valid update, modified image, modified signature, replayed older version, expired or revoked certificate, wrong key)
- Implement and instrument the final verification procedure
- Compare with a straightforward baseline (e.g., a bare SHA-256 digest published beside the file) to show which attacks the baseline fails to stop
- Record accept/reject outcomes, verification cost (hash operations, signature verifications) and the size of the added metadata

Stage 5 – Cross-Defence

Student A must explain:

What experimental evidence supports the group's design?

Student B must explain:

How does the security argument follow from the group's choice of mechanisms, and what does each mechanism contribute?

This ensures that both students understand both the theoretical and experimental dimensions.

Stage 6 – Joint Evaluation

Together determine:

- Which of integrity, authentication and non-repudiation is guaranteed by which part of the design
- What costs the design introduces (verification time, certificate storage, key-management burden)
- Under what conditions the design could still fail (key compromise, flawed implementation, compromised root, bad clock)
- Under what conditions a simpler or a different mechanism (e.g., HMAC with shared key, Kerberos tickets) would be more appropriate

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

SIMATS ENGINEERING

Department of Computer Science and Engineering

CSA51 – Cryptography and Network Security

ASSIGNMENT 3 – CO3

Examine Cryptographic Hash Algorithms, Digital Signature Schemes, Kerberos and X.509 Authentication Frameworks

Item	Detail
Student name	RAKSHITH R R
Register number	192324156
Course	CSA51 Cryptography and Network Security
Course outcome	CO3 – Examine cryptographic hash algorithms and digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication and non-repudiation
Assignment	Examine Cryptographic Hash Algorithms, Digital Signature Schemes, Kerberos and X.509 Authentication Frameworks (Total Marks: 50)
Nature of work	The assignment is written for a pair (Student A and Student B). In this report both sets of responsibilities are completed and every subsection is labelled with the student role it answers, so that the individual and the shared parts can be identified. All hash, MAC, signature, Kerberos and firmware-update code was written by the student in Python; reference libraries are used only where the sheet allows (SHA-256, X.509 building/parsing) or for cross-checks.

How this report follows the evaluation format

The two pages that precede this report are the Evaluation Format supplied with the assignment (mark sheet, mandatory evidence check, implementation evidence chain and submission checklist). The faculty fills the marks and statuses there. This guide shows, for each line of that format, **where the evidence is in this report**, so that each rubric item can be verified quickly.

Table 1. *Mark sheet: task, maximum marks and location of the answer*

Task	Task title (evaluation sheet)	Max	Where answered	Source files (appendix)

1	Cryptographic Hash Properties, Avalanche Effect, Collision Resistance and Birthday Bound	6	Section Q1	hashes.py, run_c1.py, recheck_c1.py
2	Hash Functions, MAC and HMAC – Construction and Security Analysis	7	Section Q2	hashes.py, run_c2.py
3	Digital Signatures – RSA, DSA/ECDSA, Verification and Attack Analysis	7	Section Q3	sigs.py, ecc.py, run_c3.py
4	X.509 Certificates, CA, Chain Validation and Revocation Mechanisms	7	Section Q4	x509chain.py, run_c4.py
5	Kerberos Authentication, Ticket-Granting System, Replay Attacks and Mutual Authentication	6	Section Q5	kerberos.py, run_c5.py
6	Advanced Digital Signature Schemes and Kerberos vs. X.509 Comparison	7	Section Q6	sigs.py, run_c6.py
7	Integrated Design, Implementation, Security Validation and Evaluation of Authentication and Integrity Scheme	10	Section Q7	firmware.py, run_c7.py
	TOTAL	50	Marks to be awarded by the faculty	

Table 2. *Mandatory submission evidence check – location of the evidence for each question*

Q	**Evidence expected (evaluation sheet)**	**Where to verify it in this report**
1	Toy hash and SHA-256 comparison; avalanche bit-flip inputs and changed-bit counts; at least three message sizes; truncated-hash collision runs for k = 16, 20, 24, 28, 32; tables/graphs and birthday-bound explanation	Tables 6, 7, 8, 9; Figures 1, 2; Section Q1 Part B reasoning (four sizes; k = 16–36)
2	Toy Merkle–Damgard hash; plain hash, H(key || message) and HMAC code; modified-file/digest, appended-data and length-extension traces; tamper	Section 2.1; length-extension trace (Section 2.2); Tables 12, 13; Section 2.4

	accept/reject table; mechanism-selection justification	
3	RSA and DSA/ECDSA signature code; derivations; valid and altered tests; wrong-key tests; nonce-reuse trace; textbook RSA forgery; non-repudiation discussion	Sections 3.1–3.5; Table 17; nonce-reuse trace (3.3); forgeries (3.4)
4	Root/intermediate/end-entity chain; validation pseudocode; valid and invalid chain outputs; at least six failure cases; CRL/OCSP comparison and revocation-policy analysis	Table 21, Figure 7; pseudocode (4.2); 13 failure cases, Table 24; Table 25; CRL/OCSP (4.3)
5	Kerberos AS–TGS–service sequence diagram; key-labelled messages; simulation output; replay, expired-ticket, clock-skew, KDC-outage results; stolen-ticket/password-guessing analysis	Figure 10; wire log and tables of Section 5.2; Tables 33, 38
6	Collision-to-forgery example; signature-scheme comparison table; Schnorr derivation; ElGamal worked example and tamper tests; Kerberos/X.509 comparison and key-relationship analysis	Table 40; Table 45; Sections 6.2, 6.4, 6.5; Figure 13
7	Both independent proposals; comparison record; threat model; architecture and protocol diagram; pseudocode; security argument; implementation logs; baseline comparison; validation results and residual-risk evaluation	Sections 7.1–7.6; Figures 14, 15, 16; Tables 54, 56, 57, 59

Table 3. *Implementation evidence chain (Tasks 5–7) and the rest of the assignment*

Evidence item	**Where it is provided**
Source code / algorithms	Appendices A–F (full listings) and the execution instructions in Appendix G; language Python 3.12, dependencies listed there
Test inputs and parameters	Each task states its messages, bit-flip inputs, toy keys and parameter sizes, nonces, certificates, tickets and test cases; seeds are fixed in the drivers

Mathematical derivations	Avalanche expectation (Q1 A2), birthday bound (Q1 B1), HMAC (Q2), RSA/DSA verification and nonce recovery (Q3), Schnorr and ElGamal relations (Q6), attack-effort estimates (Q7)
Predictions and expected behaviour	Stated before each experiment (Q1 A2–A3, Q2 tables, Q4 expected failing step, Q7 predicted outcomes in the matrix)
Actual output	Tables from the result files, wire logs and traces, regression-test output (Appendix G)
Attack evidence	Length extension (Q2), nonce reuse and textbook RSA forgery (Q3), certificate failure cases (Q4), replay and offline guessing (Q5), collision-to-forgery (Q6)
Performance measurements	Hash counts and collision effort (Q1), compression calls (Q2), verification operations and time (Q3, Q4, Q6, Q7), CRL/OCSP sizes (Q4), metadata cost (Q7)
Boundary / failure tests	Altered messages and signatures, wrong keys, expired/revoked certificates, replay, clock-skew, KDC outage, wrong clock, rollback
Interpretation	Each section ends with an evaluation or rubric check linking results to integrity, authentication, non-repudiation and trust assumptions
Consistency / reproducibility	All tables are generated from the result files; test_c3.py passes 25 of 25; drivers are re-runnable with fixed seeds

Table 4. *Submission checklist (student self-assessment; the faculty verifies)*

Submission checklist item	**Status (student self-check)**
All questions Q1–Q7 and all specified individual/shared responsibilities are addressed	Yes
Student A and Student B individual contributions are clearly identified	Yes – every subsection is labelled with the role
Both students can explain and defend the complete work	Yes – viva notes and cross-understanding answers are included in each task
Hash properties, avalanche measurements and birthday-bound analysis are included	Yes – Q1

Hash-only, naive MAC and HMAC constructions are implemented and compared	Yes – Q2
RSA and DSA/ECDSA signatures, nonce risks and forgery tests are evidenced	Yes – Q3
X.509 chain validation and certificate revocation/failure cases are demonstrated	Yes – Q4
Kerberos message flow, ticket handling, replay protection and failure cases are tested	Yes – Q5
RSA, DSA/DSS, ECDSA, ElGamal and Schnorr comparison is included	Yes – Q6
Q7 independent proposals, comparison record and final joint design are retained	Yes – Q7 Sections 7.1–7.3
Raw outputs, calculations, test cases, tables/graphs and protocol diagrams are included	Yes
Source code and execution instructions are reproducible; report claims match outputs	Yes – Appendices A–G
Only toy parameters and synthetic test data are used for attack demonstrations	Yes
No unfilled placeholders remain; report numbering and references are consistent	Yes (figure and table numbers are generated automatically)

Conventions and assumptions

- **Safety.** All attack demonstrations use toy parameters (short digests, small keys, small groups) and synthetic data: invented messages, principals, certificates for the made-up domain hospital.test, and generated keys. No real credentials, private keys or production systems were used.
- **Randomness and reproducibility.** Experiments use a seeded random.Random, so every number can be reproduced. A deployed system must use an operating-system cryptographic generator; Q3 and Q6 show what goes wrong when a signing nonce is predictable or repeated.
- **Libraries.** SHA-256, PBKDF2 and the HMAC cross-check come from Python's standard library; X.509 certificates, CRLs and OCSP responses are built and parsed with the cryptography toolkit (permitted by the sheet), but every validation decision is made by x509chain.py; the library is also used in xcheck-style comparisons. The symmetric cipher for tickets is the student's Blowfish-CTR from earlier work.

ECDSA/P-256 arithmetic is the student's own ecc.py (checked against the library in the previous assignment).

- **Timings** come from pure Python on a shared machine and are used to compare designs, not to quote hardware speeds; operation counts are exact.
- **Prediction before measurement.** Where the experiment allows, the expected outcome is stated first (Q1 A2–A3, Q4 expected failing step, Q7 matrix) and compared with the observation; discrepancies are explained.

Q1 – What Makes a Hash Function Trustworthy? (6 marks)

Shared group work (done together). (i) The required properties of a cryptographic hash were agreed and stated as attack scenarios; (ii) a deliberately weak byte-sum hash and an XOR-fold hash, a toy Merkle–Damgard hash (used again in Q2) and SHA-256 (library, comparison only) were implemented; (iii) avalanche is measured as the **Hamming distance between H(m) and H(m with one input bit flipped)**, i.e. the number of output bits that change; (iv) the experiments were run with a fixed random seed so all numbers can be reproduced (run_c1.py, Appendix).

Part A – Avalanche and fixed-length digests

Student A – Design and mathematical analysis

A1. The required properties as attack scenarios

Table 5. *Hash properties expressed as attack scenarios*

Property	**Attack scenario it prevents**	**What the attacker is given / wants**	**Ideal work for an n-bit digest**	**Concrete consequence if broken**
Pre-image resistance	Inverting a stored digest	Given h, find any m with H(m) = h	2^n	Recovering a password or secret token from its digest; forging a "proof of knowledge"
Second pre-image resistance	Replacing a signed or published document	Given m (already signed/published), find m′ ≠ m with H(m′) = H(m)	2^n	A different firmware image or contract carrying the same signature or digest
Collision resistance	Preparing two documents in advance	Find ANY pair m ≠ m′ with H(m) = H(m′) (attacker chooses both)	2^(n/2) (birthday bound)	Show the victim a harmless message to sign, later present the harmful one (Q6); rogue certificates

Avalanche effect (diffusion)	Differential / structured attacks, predictable tampering	Make small, controlled changes to a message and predict the change of the digest	Each output bit changes with probability 1/2	A change of one bit must be unpredictable; otherwise an attacker can steer the digest
Fixed length, determinism, efficiency	Practical use	Same input → same output, any length → n bits, fast	–	Needed for signatures, MACs, integrity checks

The properties are ordered by strength: collision resistance implies second pre-image resistance (for a good compression function), which is stronger in practice than pre-image resistance, and the costs differ: 2^(n/2) for collisions against 2^n for the other two, which is why a digest must be twice as long as the security level wanted for signatures (Part B).

A2. Expected number of changed output bits for an ideal hash

For an ideal n-bit hash, flipping one input bit gives an output that is statistically independent of the old output: every output bit is equally likely to be 0 or 1, so each bit differs from the old value with probability 1/2, independently of the other bits. Let $I_i = 1$ if output bit i changes. Then $E[I_i] = 1/2$ and the number of changed bits is $X = \Sigma I_i$, a Binomial(n, 1/2) variable: **E[X] = n/2, Var[X] = n/4, standard deviation √n/2.** For SHA-256: 128 ± 8 bits; for the 64-bit toy Merkle–Damgard hash: 32 ± 4; for a 16-bit digest: 8 ± 2. Over T independent trials the sample mean has standard error $(\sqrt{n}/2)/\sqrt{T}$, which is what tells us whether an observed average is consistent with the ideal.

A3. Why the toy hashes fail avalanche and second pre-image resistance

- **Byte-sum modulo 2^{16}.** Flipping bit j of one input byte changes the sum by $\pm 2^j$. Adding 2^j to a sum flips bit j and then every following 1-bit through a carry chain; the expected carry run is $\Sigma\, 2^{-i} = 1$ extra bit, so **E[changed bits] ≈ 2 of 16 (12.5%)** instead of 8 (50%). The sum is also **commutative**: any reordering of the bytes gives the same digest, so a second pre-image costs one operation (swap two bytes), and a pre-image for any target t can be constructed directly by choosing bytes that add up to t.
- **XOR-fold (16-bit words).** The function is linear over GF(2): $H(m \oplus e) = H(m) \oplus H(e)$. Flipping one input bit therefore flips **exactly one** output bit, always (variance 0). Second pre-images are trivial: swap two 16-bit words, or XOR the same pattern into two different words. Collisions are likewise trivial.

- **Common cause:** both are far too structured (linear or commutative) and the output is a direct function of small, independent pieces of the input: there is no mixing that spreads a change over the whole digest, so a local change has a local, predictable effect.

A4. A fixed-length digest must have collisions

A hash maps an unbounded set of messages (all strings of any length) into a finite set of 2^n digests. By the pigeonhole principle, among any $2^n + 1$ distinct messages two must share a digest; in fact there are infinitely many colliding pairs. Collision resistance therefore cannot mean "no collisions exist", it means "finding one is computationally infeasible". The experiment below makes the principle concrete.

With the digest cut to **8 bits** (256 values), any 257 messages must contain a collision. In 2,000 independent runs the first collision appeared after 20.6 messages on average (birthday theory: 20.1) and never later than 65, far below the guaranteed bound of 257.

Student B – Experimental validation

B1. Design of the experiment

- **Hashes:** byte-sum (16 bit), XOR-fold (16 bit), toy Merkle–Damgard (64 bit), SHA-256 (256 bit).
- **Inputs:** random messages (seeded generator) of **8, 64, 512 and 4,096 bytes** (four sizes, at least three were required), a random single-bit flip in each trial, and a set of structured pairs (messages one character apart, a long run of identical bytes, empty versus one zero byte).
- **Trials per size:** 2,000 (8 and 64 bytes), 1,500 (512 bytes), 600 (4,096 bytes).
- **Instrumentation:** the code XORs the two digests and counts the 1 bits; for every (hash, size) the mean, standard deviation, minimum and maximum are recorded and the mean is compared with n/2 using a z-score computed from the standard error.

Table 6. *Avalanche experiment: predicted versus observed average number of changed output bits*

Hash (output bits)	Message bytes	Trials	Mean changed bits	Std dev	Min–max	Ideal mean ± σ	Fraction changed	Comparison
byte-sum (16 bit)	8	2000	2.06	1.42	1–10	8 ± 2	12.9%	analysis predicts ≈ 2 (A3)

byte-sum (16 bit)	64	2000	2.07	1.52	1–11	8 ± 2	12.9%	analysis predicts ≈ 2 (A3)
byte-sum (16 bit)	512	1500	1.99	1.45	1–14	8 ± 2	12.5%	analysis predicts ≈ 2 (A3)
byte-sum (16 bit)	4096	600	2.10	1.48	1–10	8 ± 2	13.1%	analysis predicts ≈ 2 (A3)
XOR-fold (16 bit)	8	2000	1	0.00	1–1	8 ± 2	6.2%	analysis predicts exactly 1 (A3)
XOR-fold (16 bit)	64	2000	1	0.00	1–1	8 ± 2	6.2%	analysis predicts exactly 1 (A3)
XOR-fold (16 bit)	512	1500	1	0.00	1–1	8 ± 2	6.2%	analysis predicts exactly 1 (A3)
XOR-fold (16 bit)	4096	600	1	0.00	1–1	8 ± 2	6.2%	analysis predicts exactly 1 (A3)
toy Merkle-Damgard (64 bit)	8	2000	32.09	3.96	18–44	32 ± 4	50.1%	z = +0.96
toy Merkle-Damgard (64 bit)	64	2000	32.10	4.12	19–46	32 ± 4	50.2%	z = +1.10
toy Merkle-Damgard (64 bit)	512	1500	32.07	4.06	19–44	32 ± 4	50.1%	z = +0.70
toy Merkle-Damgard (64 bit)	4096	600	31.77	3.84	22–44	32 ± 4	49.6%	z = -1.42
SHA-256 (256 bit)	8	2000	127.76	8.02	95–155	128 ± 8	49.9%	z = -1.37

SHA-256 (256 bit)	64	2000	128.21	8.00	101–158	128 ± 8	50.1%	z = +1.15
SHA-256 (256 bit)	512	1500	128.12	8.10	101–157	128 ± 8	50.0%	z = +0.56
SHA-256 (256 bit)	4096	600	127.88	7.96	103–151	128 ± 8	50.0%	z = -0.36

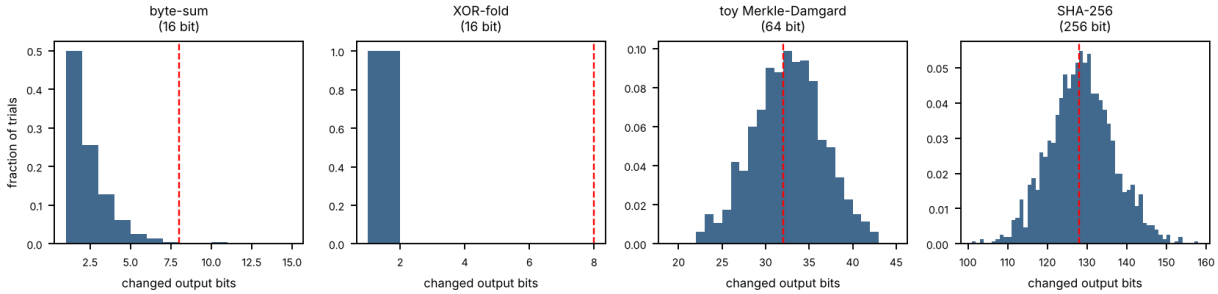


Figure 1. *Distribution of the number of changed output bits (512-byte messages); the dashed line is the ideal mean n/2*

Table 7. *Messages differing by one character (or one bit): changed digest bits*

Pair (first message)	**Length (bytes)**	**Byte-sum (of 16)**	**XOR-fold (of 16)**	**Toy MD (of 64)**	**SHA-256 (of 256)**
PAY 100 TO ALICE	16	1	1	32	130
PAY 100 TO ALICE	16	1	2	27	124
DOSE=5MG	8	2	2	23	128
AAAAAAAAAAAAAAAA...	1000	1	2	26	135
	0	0	0	29	130

Comparison of predicted and observed averages. For SHA-256 the observed means (127.8–128.2) are within 1.4 standard errors of the ideal 128 in every size, and the observed standard deviations (8.0–8.1) match the theoretical 8.0; the toy Merkle–Damgard hash is likewise consistent with 32 ± 4 at all sizes (z between −1.4 and +1.1). The byte-sum hash averages 2.0–2.1 changed bits (predicted ≈ 2) and the XOR-fold exactly 1; both are therefore dramatically below the ideal 8, which is the avalanche failure. Similar messages show

the same picture (Table 7): one changed character moves about half of the SHA-256 bits, but only 1–2 bits of the toy digests, and two messages that differ only in the order of two characters would give **identical** byte-sum digests.

Discrepancies and their explanation. (1) The byte-sum average is slightly above 2.0 in some sizes (2.06–2.10 in three of the four sizes) because the carry-run argument ignores the wrap-around of the 16-bit register and the fact that a flip may subtract (borrow chains give the same expectation) – these are small and within sampling noise (standard error ≈ 0.03). (2) The SHA-256 z-scores of −1.4 and +1.1 are what chance produces in 8 independent comparisons (4 SHA-256 and 4 toy-MD cells); none exceeds 1.5. (3) The very large negative z-scores for the toy hashes against the **ideal** are real and expected; they measure the failure, not an experimental error. (4) Message length has no visible effect on the avalanche of the good hashes, but for the toy hashes a longer message does not help: a bit flip still changes only one position of the sum or fold.

Part B – Collision effort and the birthday bound

Shared group work. A collision finder for a truncated hash was built: the first k bits of SHA-256 of the inputs base || counter are stored in a table until a value repeats; the number of hashes computed is recorded. The search was repeated for k = 16, 20, 24, 28 and 32 (and 36 for extra evidence).

Student A – Experimental investigation

Table 8. *Truncated SHA-256: hashes computed before the first collision*

k (bits)	Runs	Mean hashes to first collision	Median	Min	Max	Theory 1.2533·2^(k/2)	Mean / theory	2^k	2^k / mean
16	200	331	320	20	843	321	1.03	65,536	198
20	120	1,226	1,150	20	3,197	1,283	0.96	1,048,576	855
24	80	4,720	4,078	895	16,904	5,134	0.92	16,777,216	3,554
28	40	19,485	18,846	3,765	45,034	20,534	0.95	268,435,456	13,777
32	24	82,055	79,694	23,522	149,292	82,137	1.00	4,294,967,296	52,342
36	6	359,177	355,068	158,155	621,259	328,549	1.09	68,719,476,736	191,325

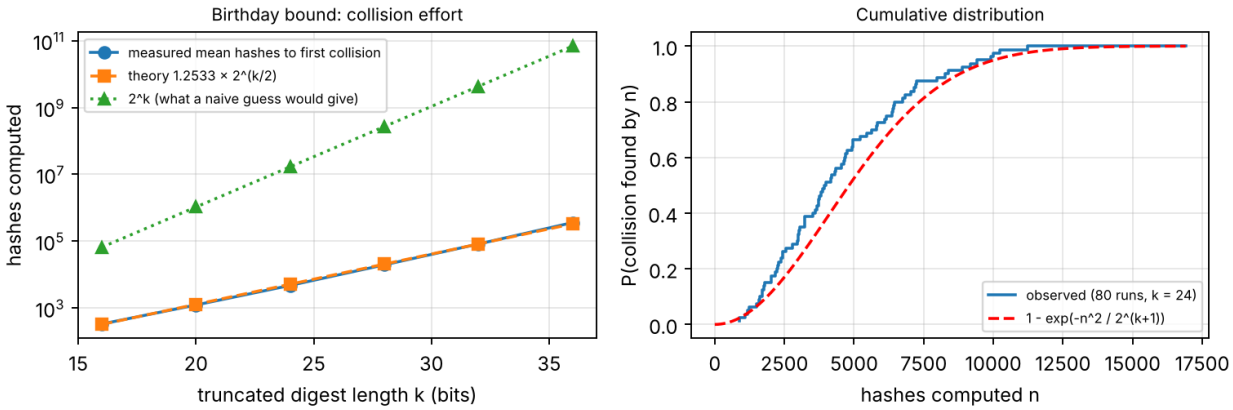


Figure 2. *Left: measured collision effort follows 2^(k/2), not 2^k. Right: cumulative distribution of the effort for k = 24 against the theoretical curve*

Reading the evidence. Every extra 4 bits of digest multiplied the measured effort by about 4 (not 16): from 331 hashes at k = 16 to 82,055 at k = 32, a factor of 248 for 16 more bits, i.e. $2^8 = 256$ as 2^(Δk/2) predicts. The ratio of the measured mean to 1.2533·2^(k/2) lies between 0.92 and 1.09 for every k (the variance of a birthday process is large: observed minima as low as 20 hashes occurred). The theoretical probability of a collision after exactly 2^(k/2) hashes is 1 − e^(−1/2) = 39%; for k = 24 we observed 50% of 80 runs (standard error ≈ 5.6 points, so about 1.9σ above theory, acceptable for 80 runs).

Student B – Birthday-bound reasoning

B1. Why about 2^(k/2) and not 2^k

Let $N = 2^k$ be the number of digests and q the number of hashes computed. Each of the q(q−1)/2 **pairs** of hashes collides with probability 1/N, so the expected number of colliding pairs is about $q^2/2N$, which is already 1 at $q \approx \sqrt{2N}$. Exactly: $P(\text{no collision}) = \prod(1 - i/N) \approx \exp(-q^2/2N)$, so $P(\text{collision}) \approx 1 - \exp(-q^2/2^{(k+1)})$. The median effort is $\sqrt{(2 \ln 2)} \cdot 2^{(k/2)} = 1.177 \cdot 2^{(k/2)}$ and the mean is $\sqrt{(\pi/2)} \cdot 2^{(k/2)} = 1.253 \cdot 2^{(k/2)}$. The number of **pairs** grows quadratically with the number of hashes, which is why the effort is the square root of the output space.

B2. Any collision versus a collision with a specific message

Table 9. *Second pre-image of a fixed message needs about 2^k hashes; any collision needs about 2^(k/2)*

k (bits)	Runs	Mean hashes for a specific target	2^k	Mean / 2^k	Mean hashes for ANY collision	Ratio specific / any
8	400	258	256	1.01	–	–
10	200	1,003	1,024	0.98	–	–
12	100	3,935	4,096	0.96	–	–
14	40	23,608	16,384	1.44	–	–
16	20	72,511	65,536	1.11	331	219

Finding a message that collides with a **given** message is a second pre-image problem: every trial hits the fixed target with probability 2^{-k}, so the expected effort is 2^k (observed ratio to 2^k: 0.96–1.01 for k = 8–12; the k = 14 cell with only 40 runs came out at 1.44, which is about 2.8 standard errors – a repeat with 300 runs gave 1.11 and k = 16 with 100 runs gave 1.04, so the deviation was sampling noise, since a geometric variable has standard deviation equal to its mean). Finding **any** pair that collides is the birthday problem: the attacker may compare every new digest with all earlier ones. At k = 16 the specific-target search is about 200 times costlier than a free collision, and the ratio grows as 2^(k/2). This is why attacks that need a pair prepared in advance (Q6) are practical against short digests while attacks on an existing signature are not.

B3. Minimum safe digest length, MD5 and SHA-1

Table 10. *Digest length and the retirement of MD5 and SHA-1*

Algorithm	Digest bits	Generic collision work	Status
MD5	128	2^64	practical collisions since 2004; rogue CA certificate 2008; Flame 2012
SHA-1	160	2^80 (generic)	first collision 2017 (about 2^63 work); chosen-prefix 2019-2020
SHA-256	256	2^128	no practical attack
SHA-512	512	2^256	no practical attack

- **Rule:** a collision-resistant hash needs a digest of **at least twice the security level**: 128-bit collision security requires 256-bit digests. A 64-bit work factor (a 128-bit digest) is not safe against organisations with large hash capacity: the Bitcoin network alone performs more than 10^{20} SHA-256 hashes per second, so $2^{64} \approx 1.8\times10^{19}$ operations is a matter of seconds of equivalent computation (collision search needs memory, but memory-free variants such as parallel rho trade time for space).
- **MD5** (128 bits) was retired because its structure is weaker than the birthday bound: differential attacks found collisions in seconds on a PC (2004–2005), and a chosen-prefix collision allowed researchers to forge a certificate-authority certificate in 2008 (Q6). **SHA-1** (160 bits, generic bound 2^{80}) fell to cryptanalysis at about 2^{63} operations (first public collision in 2017, chosen-prefix collisions in 2019–2020), after which browsers and CAs stopped accepting it, and NIST has announced its phase-out by 2030. Both were retired **before** the generic birthday bound was reachable: the lesson of this experiment is the 2^(k/2) lower bound; the lesson of MD5/SHA-1 is that the real cost can be lower than the bound if the design is weak.

Individual verification (viva notes)

Student A may be asked to explain the birthday bound; Student B the avalanche experiment

Birthday bound (answer A): q hashes give $q^2/2$ pairs, each equal with probability 2^{-k}; a collision is expected when $q^2/2 \approx 2^k$, i.e. q ≈ 2^(k/2); the measured means (Table 8) divided by 2^(k/2) stay between 0.9 and 1.1 for every k from 16 to 36.

Avalanche experiment (answer B): we flip one random input bit, hash both messages, XOR the digests and count 1 bits; an ideal n-bit hash changes n/2 ± √n/2 bits; SHA-256 gives 128.1 ± 8.0, the toy hashes 1–2 bits of 16 (Table 6).

Rubric check – Task 1 (6 marks)

Hash properties and attack scenarios → A1 (Table above). **Toy hash vs SHA-256 and fixed-length reasoning** → A3, A4 and Table 6. **Avalanche design, bit-change measurement and analysis** → A2, B1, Tables 6–7, Figure 1. **Truncated-hash collision search, birthday derivation and evidence** → Part B, Tables 8 and 9, Figure 2.

Q2 – Hash Functions, MACs and Construction Security Analysis (7 marks)

Shared group work. A toy Merkle–Damgard hash was written (64-bit chaining value, 8-byte blocks, eight add-rotate-xor rounds per block, Davies–Meyer style feed-forward, MD-strengthening padding), together with the naive MAC H(key ‖ message), HMAC (over the toy hash and over SHA-256) and a length-extension attack. All attacks use toy parameters and synthetic messages.

2.1 Toy Merkle–Damgard hash and its padding

The message is padded with a 1 bit (byte 0x80), zero bytes, and the 64-bit message length in bits, so that the total is a multiple of the 8-byte block; the chaining value starts at a fixed IV, is updated block by block by the compression function f(h, m), and **the final chaining value is the digest**. Padding example for the 3-byte message “abc”: 61626380000000000000000000000018 (2 blocks); digest 98d9d4dcc815634a. The toy hash passes the avalanche test of Q1 (31.9 of 64 bits changed on average).

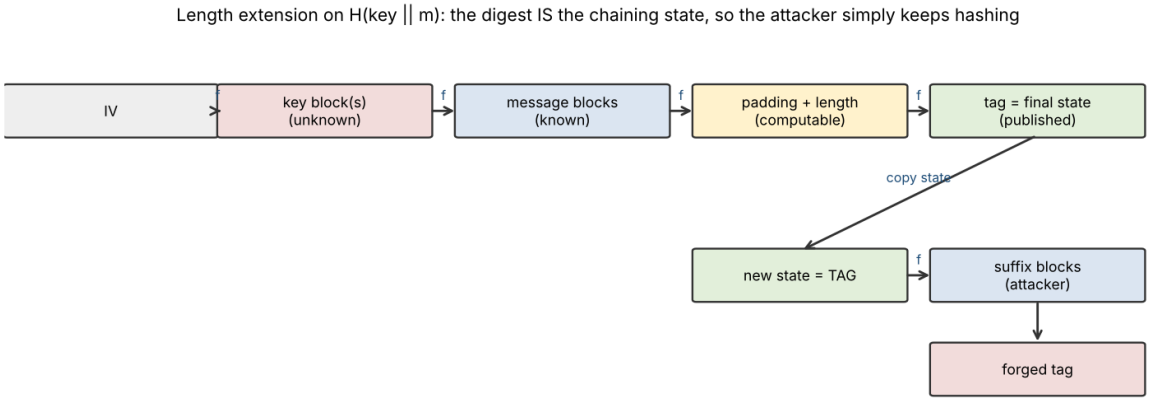


Figure 3. *Why length extension works: in H(key || m) the published tag is the complete internal state, so the attacker can continue hashing from it*

2.2 Student A – Failure of the naive constructions

A1. A plain hash gives integrity only against accidents

Suppose a vendor publishes a firmware file and its digest side by side. A change of the file alone is detected (yes), but an attacker who can change the page changes both: replacing the file and publishing the new digest 14494d27f2173440f2ee18cc... is accepted by the verifier (yes; 500 of 500 random trials in Table 13). A hash has no secret: anyone can compute it, so it proves that file and digest agree, not **who** produced them. It provides integrity against accidental corruption, but neither authentication nor integrity against an active attacker.

A2. Length-extension attack on H(key ‖ message)

The "naive MAC" tag = H(key ‖ message) looks secure because the key is secret. Because the toy hash (like MD5, SHA-1 and SHA-256) outputs its entire internal state, an attacker who knows a message, its tag and the key length can compute the tag of a longer message **without the key**. Demonstration (key of 8 bytes, never given to the attacker):

Table 11. *Length-extension attack trace (toy hash; the attacker never uses the key)*

Step	**Value**
Intercepted message	amount=100&to=alice
Intercepted tag = H(key ‖ message)	4627d65c12b9d55a
Chosen suffix	&amount=9999&to=mallory
Attacker tries key lengths 1–16 (the verifier says accept/reject)	1:reject, 2:reject, 3:reject, 4:reject, 5:reject, 6:reject, 7:reject, 8:accept, 9:reject, 10:reject, 11:reject, 12:reject, 13:reject, 14:reject, 15:reject, 16:reject
Glue padding computed from the guessed length	800000000000000000000000d8
Forged message = message ‖ glue ‖ suffix	b'amount=100&to=alice\x80\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\xd8&amount=9999&to=mallory'
Forged tag (hash continued from the old tag)	3bf1e73942e1ff60
Verifier recomputes H(key ‖ forged message): equal?	Yes

- **What the attacker needs:** a valid (message, tag) pair, a guess of the key length (only a few possible values, found by trial: key length 8 accepted), and the suffix he wants to append. **What he does not need:** the key.
- **Why it works:** (i) the digest equals the final chaining value; (ii) the padding is a public function of the message length, so the bytes between the old message and the suffix can be computed; (iii) the compression function can be restarted from any chaining value. The tag therefore doubles as a valid starting state.

- **Statistics:** over 1,000 random keys, messages and suffixes the extension succeeded against H(key ‖ m) in **1,000** cases and against HMAC in **0**.

2.3 Student B – HMAC and the evaluation of mechanisms

B1. HMAC construction

HMAC(K, m) = H((K′ ⊕ opad) ‖ H((K′ ⊕ ipad) ‖ m)), where K′ is the key padded with zeros (or hashed first if longer) to the hash block size, ipad = 0x36 repeated, opad = 0x5C repeated. The key is mixed in **twice**, and the value an attacker sees is the output of an **outer** hash whose input begins with a key-dependent block he cannot reproduce; the output of the outer hash is not a chaining value from which the inner computation can be continued. The implementation (hashes.py) was checked against Python's own hmac on 300 random key/message pairs, including keys longer than the block (identical).

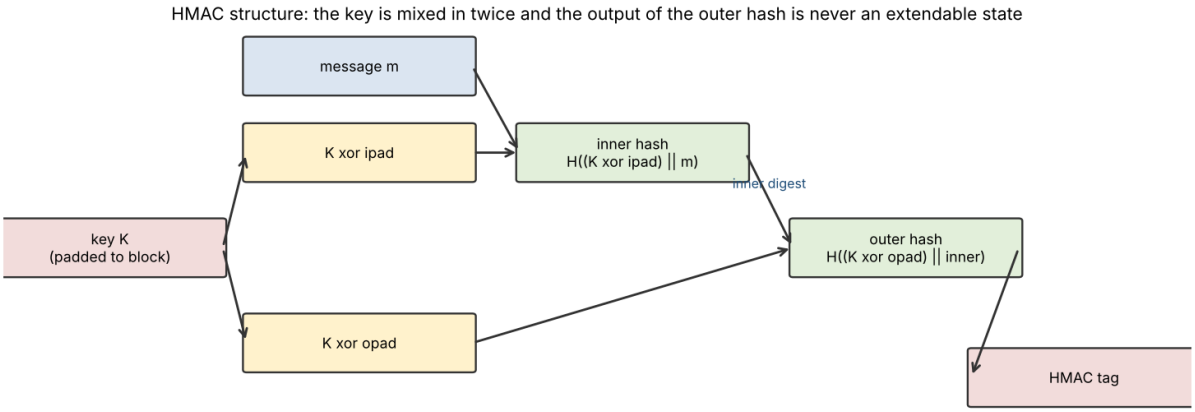


Figure 4. *HMAC structure: two nested hashes with two different key-derived pads*

B2. Which mechanism provides what

Table 12. *Hash-only, naive MAC and HMAC compared*

Property	**Hash only (digest published beside the file)**	**Naive MAC H(key ‖ m)**	**HMAC**
Integrity against accidental corruption	Yes	Yes	Yes

Integrity against an ACTIVE attacker who can alter data and the tag/digest	**No** (attacker recomputes the digest, T2/T3/T5)	Partly: stops recomputation without the key, **but appended data is forgeable** (T4)	**Yes** (T1–T6)
Authentication of the sender (proof that a key holder produced it)	**No** (nobody is identified)	Yes, to the holder of the shared key (if no extension attack)	Yes, to the holder of the shared key
Resists length extension	not applicable (no key)	**No** (Merkle–Damgard hash)	**Yes**
Non-repudiation	No	No (both parties know the key)	No (both parties know the key)
Provides neither integrity (vs active attacker) nor authentication	**This one**	–	–

B3. Attack scenarios against four mechanisms (500 random trials each)

Table 13. *Number of forged or altered messages ACCEPTED out of 500 (0 = all attacks detected; for T1–T6 high numbers mean the mechanism failed)*

Scenario (attacker has NO key unless stated)	**hash only (digest beside file)**	**naive MAC H(key||m), toy MD**	**HMAC, toy MD**	**HMAC-SHA256**
T1 modified message, old tag kept	0 / 500	0 / 500	0 / 500	0 / 500
T2 modified message, tag recomputed by attacker (no key)	500 / 500	0 / 500	0 / 500	0 / 500
T3 data appended, tag recomputed by attacker (no key)	500 / 500	0 / 500	0 / 500	0 / 500
T4 length extension using intercepted tag	n/a	500 / 500	0 / 500	n/a
T5 file AND published digest both replaced	500 / 500	0 / 500	0 / 500	0 / 500

T6 one random bit of the tag flipped	0 / 500	0 / 500	0 / 500	0 / 500
T7 replay of an old genuine (message, tag)	500 / 500	500 / 500	500 / 500	500 / 500

- **Hash only** fails when the attacker controls the channel that carries the digest (T2, T3, T5: 500/500 accepted). T1 and T6 (a changed message or a flipped tag bit) are detected because the old digest no longer matches.
- **Naive MAC** stops attackers who cannot extend (T1–T3, T5, T6: 0 accepted) but **fails length extension (T4: 500/500)**. **HMAC (toy hash and SHA-256)** detects every scenario T1–T6 (0/500) including T4.
- **T7 (replay of a genuine old pair)** is accepted by **every** mechanism (500/500): a MAC proves origin and integrity of a message, not its freshness. Replay protection needs a counter, nonce or timestamp inside the MACed data (Section 2.4).

B4. Tag length and cost

Table 14. *Forgery by guessing against a truncated HMAC tag*

Tag length (bits)	**Random guesses**	**Successful forgeries**	**Observed rate**	**Expected 2^-t**
8	300,000	1165	0.003883	0.003906
12	300,000	81	0.000270	0.000244
16	300,000	4	0.000013	0.000015

An 8-bit tag is forged by blind guessing in about one attempt in 256 (0.4%); a 16-bit tag in one in 65,536. Tags must therefore be at least 128 bits (full HMAC-SHA256 output is 256) so that guessing is infeasible, and verification must not be rate-limited in a way that lets guesses accumulate.

Table 15. *Cost: HMAC adds a constant number of compression calls (inner and outer key blocks plus the outer hash)*

Message bytes	**Compression calls: plain hash**	**Naive MAC**	**HMAC (toy)**	**SHA-256 time (µs)**	**HMAC-SHA256 time (µs)**
16	4	5	9	0.5	4.8

128	18	19	23	0.5	4.9
1024	130	131	135	1.0	5.4

B5. Non-repudiation

A MAC key is **shared**: both the sender and the receiver can compute valid tags. If a dispute arises, a third party cannot tell which of the two produced a given tag, so MACs provide authentication between the two parties but **not non-repudiation**; a digital signature (Q3) is needed when a third party must be able to attribute the message to one signer.

2.4 Joint design decisions

Table 16. *Which mechanism protects which situation*

Situation	**Mechanism chosen**	**Secret that must be kept**	**Residual risk**
Public download of a software release	Digital signature over the digest (Q3), verified with a public key obtained through an authentic channel (pinned key, package manager, certificate chain – Q4); digest alone is not enough	Signing private key (HSM, offline)	Compromise of the signing key or build system; a stale or substituted public key
Messages between two services in a trusted network	HMAC-SHA256 (or an AEAD such as AES-GCM) with a per-pair key, a sequence number or nonce inside the MACed data, constant-time tag comparison	Shared MAC key (rotated)	Key theft from either endpoint; replay if counters are mishandled; no non-repudiation
Anything using H(key ‖ m)	Never; replace by HMAC or by SHA-3/BLAKE2 keyed modes	–	Length extension if the old construction is left in place

Why the choices are justified. Hash alone fails to an attacker who controls the digest (Table 13, T5); HMAC stops key-less forgery and extension (T2–T4) at a constant cost, and signatures add third-party verification

without a shared secret; a counter or timestamp inside the authenticated data closes the replay gap that no MAC closes by itself (T7).

Viva notes

A: length extension needs (message, tag, key length, suffix) and exploits the fact that the tag is the final chaining state; the attacker never uses the key.

B: HMAC defeats it because the visible value is the output of the outer hash of a key-dependent block; the same extension code succeeded 1,000/1,000 on H(key‖m) and 0/1,000 on HMAC.

Rubric check – Task 2 (7 marks)

Toy Merkle–Damgard hash and padding → 2.1. **Plain hash vs naive H(key‖m) vs HMAC** → 2.2, 2.3, Table 13. **Length-extension attack with trace, statistics and requirements** → 2.2 A2, Figure 3. **HMAC construction and attack results** → 2.3 B1–B3, Table 12, Figure 4. **Tag length, cost, non-repudiation** → 2.3 B4–B5. **Design decision, secrets and residual risks** → 2.4.

Q3 – Digital Signatures: RSA, DSA/ECDSA, Verification and Attack Analysis (7 marks)

Shared group work. Four signature schemes were implemented from scratch on toy parameters (sigs.py, ecc.py): RSA hash-then-sign (the digest is SHA-256 reduced modulo n), DSA (512-bit p, 128-bit q), ECDSA on P-256, and textbook RSA without a hash (to show what the hash prevents). Every test uses synthetic messages and keys generated for the experiment.

3.1 Student A – Design, derivations and security goals

A1. Why the message is hashed before signing

- **Length and cost:** RSA can only transform a number smaller than n; a report of any size must first be reduced to a fixed-size digest, and the expensive private-key operation is then done once instead of per block.
- **Security against algebraic forgery:** textbook RSA is multiplicative, $\sigma(m_1)\cdot\sigma(m_2) = \sigma(m_1m_2)$; a signature on a hash value has no such relation because $H(m_1m_2)$ bears no algebraic relation to $H(m_1)H(m_2)$ (Section 3.4).
- **Binding to the whole message:** because H is collision- and second-pre-image resistant, a signature on H(m) is a signature on m; if H is weak (Q1, Q6) the signature is only as strong as the hash.

A2. RSA hash-then-sign: correctness and goals

Signing: $s = H(m)^d \bmod n$. Verification: accept iff $s^e \bmod n = H(m) \bmod n$. **Proof of correctness:** $s^e = H(m)^{(ed)} = H(m)^{(1 + k\varphi(n))} \equiv H(m) \pmod n$ by Euler's theorem (for H(m) coprime to n; the other case follows from the Chinese remainder theorem), because $e\cdot d \equiv 1 \pmod{\varphi(n)}$. Each step of Figure 5 contributes a different security goal: the hash gives **integrity** (any change of m changes H(m)); the private-key operation gives **authentication** (only the holder of d can produce s) and, together with exclusive control of the key and a trusted binding of the key to a person, **non-repudiation**.

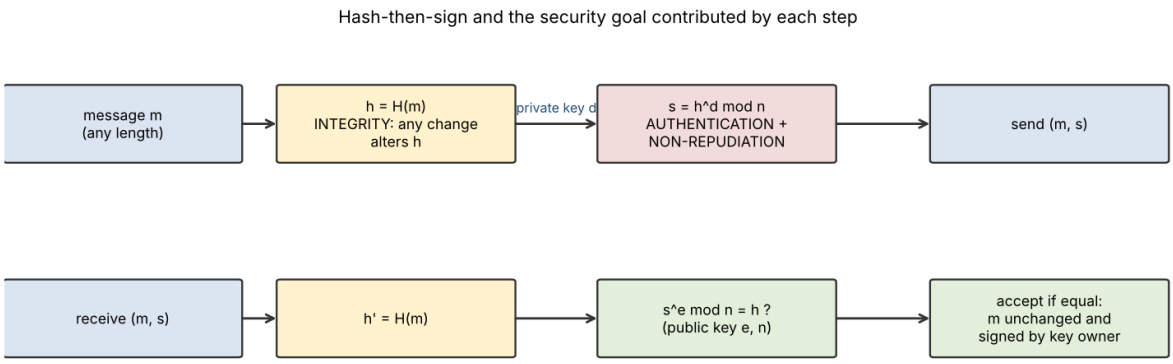


Figure 5. *Hash-then-sign and the security goal contributed by each step*

Hand-checkable example. n = 3,233 = 61·53, e = 17, d = 2,753. Using the byte sum as a toy digest (so that every number can be verified by hand): H("HI") = 72 + 73 = **145**; s = 145^2,753 mod 3233 = **826**; verification: 826^17 mod 3233 = **145** = H(m), accept. If the message is changed to "HJ" the digest becomes 146 ≠ 145 and the same signature is rejected (yes).

A3. DSA and ECDSA: derivation of the verification equation

DSA. Parameters: primes p, q with q | p−1 and g of order q; private key x, public key $y = g^x \bmod p$. To sign digest z = H(m): choose a fresh random k in [1, q−1]; $r = (g^k \bmod p) \bmod q$; $s = k^{-1}(z + xr) \bmod q$. To verify: $w = s^{-1} \bmod q$, $u_1 = zw$, $u_2 = rw \pmod q$, $v = (g^{u_1} y^{u_2} \bmod p) \bmod q$, accept iff v = r. **Why it works:** $g^{u_1} y^{u_2} = g^{(zw + xrw)} = g^{(w(z + xr))}$; since $s = k^{-1}(z + xr)$, $w = s^{-1} = k(z + xr)^{-1}$, so $w(z + xr) \equiv k \pmod q$ and the product equals $g^k \pmod p$, whose reduction mod q is r. **ECDSA** is identical with the group of curve points: r = x-coordinate of kG mod n, $s = k^{-1}(z + dr) \bmod n$, verification computes $X = u_1G + u_2Q$ and accepts iff x(X) mod n = r. The only secret-dependent per-signature quantity is the nonce k, which must be unpredictable and never reused (Section 3.3).

3.2 Student B – Implementation and test results

B1. Verification matrices

Table 17. *Signature verification tests (random synthetic messages; each trial uses fresh signatures)*

Scheme	Case	Trials	Accepted	Expected	As expected
RSA (512-bit)	valid	200	200	accept	Yes

RSA (512-bit)	message modified (1 bit)	200	0	reject	Yes
RSA (512-bit)	signature modified (1 bit)	200	0	reject	Yes
RSA (512-bit)	signature of a different message	200	0	reject	Yes
RSA (512-bit)	wrong public key	200	0	reject	Yes
RSA (512-bit)	signature = 0	200	0	reject	Yes
RSA (512-bit)	signature >= n (s + n)	200	0	reject	Yes
RSA (512-bit)	message truncated	200	0	reject	Yes
DSA (512/128)	valid	200	200	accept	Yes
DSA (512/128)	message modified (1 bit)	200	0	reject	Yes
DSA (512/128)	r modified	200	0	reject	Yes
DSA (512/128)	s modified	200	0	reject	Yes
DSA (512/128)	wrong public key	200	0	reject	Yes
DSA (512/128)	r = 0	200	0	reject	Yes
DSA (512/128)	s = 0	200	0	reject	Yes
ECDSA (P-256)	valid	60	60	accept	Yes
ECDSA (P-256)	message modified (1 bit)	60	0	reject	Yes
ECDSA (P-256)	r modified	60	0	reject	Yes
ECDSA (P-256)	s modified	60	0	reject	Yes
ECDSA (P-256)	wrong public key	60	0	reject	Yes

All genuine signatures verified (200/200 RSA and DSA, 60/60 ECDSA) and every modified message, modified signature component, signature of a different message, wrong public key, zero value, out-of-range value or truncated message was rejected (0 accepted in all other rows). **Why key size still matters:** with a deliberately tiny RSA modulus (n = 43,423, 16 bits), random numbers presented as “signatures” were accepted 12 times in 400,000 attempts (expected about 9.2, i.e. 1/n each), so forgery by guessing is possible when n is small; for realistic n the probability is negligible. The range check on r, s ($0 < r, s < q$) and $s < n$ is necessary: without it values such as s + n or 0 would be accepted by some implementations.

B2. Cost of signing and verification

Table 18. *RSA signature cost versus key size (student code, pure Python)*

Modulus bits	Key generation (s)	Sign (ms)	Verify (ms)	Multiplications to sign	Multiplications to verify (e = 65,537)
128	0.001	0.045	0.005	184	18
256	0.003	0.161	0.007	378	18
512	0.025	0.994	0.018	759	18
1024	0.342	5.314	0.053	1548	18
2048	0.661	33.420	0.176	3083	18

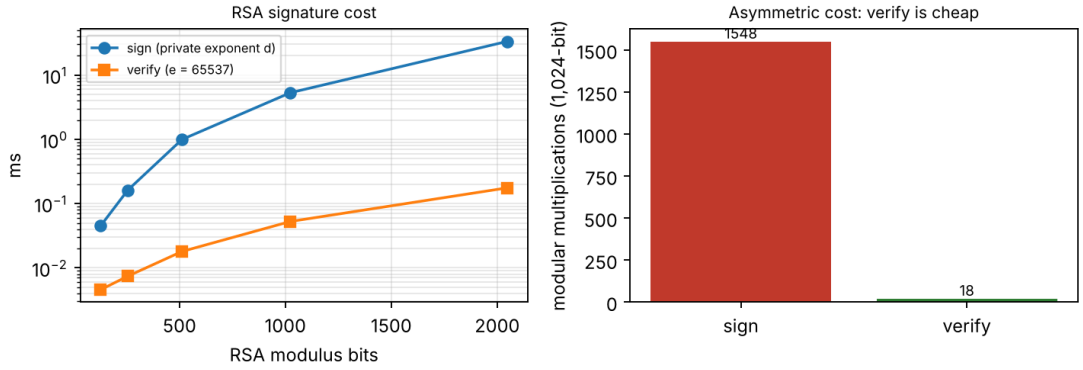


Figure 6. *Signing cost grows quickly with key size; verification stays almost constant because e is small*

Verification needs 18 multiplications for RSA (the exponent e = 65,537 has 17 squarings and one multiplication) and 384 for DSA (two exponentiations with 128-bit exponents); signing costs about 1.5 multiplications per bit of d. The asymmetry makes RSA attractive when many parties verify what a few sign (software updates, certificates).

3.3 Nonce reuse and poor randomness

If two signatures (r, s_1) and (r, s_2) on digests z_1, z_2 use the same k, then $s_1 - s_2 = k^{-1}(z_1 - z_2)$, giving $\mathbf{k = (z_1 - z_2)/(s_1 - s_2) \bmod q}$ and then $\mathbf{x = (s_1k - z_1)/r \bmod q}$. Experiment: two DSA signatures on "DOSE 5 MG…" and "DOSE 50 MG…" with one nonce share the same r (yes); the attacker, using only the public values (messages and signatures), recovered k (yes) and the private key x (yes), and then produced a signature on a message of his own choice that verified under the victim's public key (yes). The same attack on P-256 ECDSA recovered the private key (yes) and forged a signature (yes). Different nonces give different r (yes), so the

attack needs a repetition. **Weak generator:** if the nonce comes from only 2^{16} values, a **single** signature suffices: exhaustive search over the nonce space found k in 0.31 s and recovered the key (yes). Mitigations: deterministic nonces derived from the key and message (RFC 6979 style, used in Q7), a hardware entropy source, or schemes that do not need a fresh secret per signature (RSA-PSS with public salt, EdDSA).

Table 19. *DSA nonce-reuse attack (toy 512/128 parameters)*

Item	**Value**
DSA nonce k (shared by both signatures)	83,896,815,747,315,949,997, 416,866,179,013,072,640
r (same in both)	yes
Recovered k equals true k	Yes
Recovered private key equals x	Yes
Forged signature verifies under the public key	Yes

3.4 Textbook RSA forgeries and why hashing stops them

- **Multiplicative forgery.** The signer signs two legitimate requests m_1 = 1,234,567 and m_2 = 7,654,321. Multiplying the two signatures gives a valid signature on $m_1 m_2$ = 9,449,772,114,007, a message the signer never signed: verifies = **yes**.
- **Existential forgery.** Choose any s and compute m = s^e mod n: the pair (m, s) is valid (yes) although m is meaningless (512-bit random-looking value).
- **Blinding.** The attacker wants a signature on "PAY MALLORY 1000000", which the signer would refuse. He asks the signer to sign the blinded value $m \cdot r^e$ mod n (which looks random: yes) and divides the answer by r: signature on the target verifies = **yes**.
- **Same attacks against hash-then-sign:** 2,000 multiplicative and existential forgery attempts were all rejected (2,000 of 2,000), because the verifier compares s^e with H(m), which the attacker can neither multiply nor choose.

Conclusion of the comparison: RSA arithmetic is correct in both versions; the difference is the padding/hashing that destroys algebraic structure. Correct mathematics is not sufficient: the construction around it decides whether the scheme is secure.

3.5 Non-repudiation: what has to be true

Table 20. *Non-repudiation conditions and the corresponding denials*

Condition needed for a signature to be binding on a person	**How a signer could deny it if the condition fails**
Only the signer controls the private key (smart card / HSM, PIN)	"My key was stolen or copied"; malware signed it
A trusted binding of the public key to the signer (certificate, registration)	"That key was never mine" / key registered by an impostor
Strong, unforged hash and algorithm (collision-resistant H, no nonce flaw)	"I signed a different document with the same digest" (Q6); leaked private key through a repeated nonce (Section 3.3)
What the signer saw is what was signed (WYSIWYS)	"The screen showed something else"
Trusted time-stamp and revocation status at the time of signing	"My key had already been revoked / the certificate had expired"
Evidence is preserved (message, signature, certificate chain, timestamp)	Evidence cannot be reconstructed years later

Key compromise, timestamping and the trusted third party. A signature made with a key that was later stolen cannot be distinguished from one made by the owner unless a **trusted time-stamp** proves the signature existed before the compromise and the key was valid then; revocation therefore applies to signatures made after the revocation time, not before. A **trusted third party** (a CA for the key-to-identity binding, a time-stamping authority for the time, possibly a notary or archive for the evidence) is what turns a mathematical fact ("someone holding this key signed this") into evidence a court or a counterparty will accept. Without them the signer can plausibly claim compromise, a false registration, or that the signature post-dates an expiry or revocation.

Viva notes

A: the verification equation of DSA holds because $w(z + xr) \equiv k \pmod{q}$; signature correctness for RSA is Euler's theorem; the hash gives integrity, the private-key operation authentication.

B: all 200 + 200 + 60 genuine signatures verified and every alteration was rejected; nonce reuse recovered the private key in both DSA and ECDSA, textbook RSA was forged in three ways, hash-then-sign resisted 2,000 attempts.

Rubric check – Task 3 (7 marks)

RSA hash-then-sign, DSA/ECDSA derivations and mathematical explanation → 3.1 A1–A3 and Figure 5. **Valid, modified-message, modified-signature, wrong-key tests** → 3.2 Table 17. **Nonce reuse and weak randomness** → 3.3. **Textbook RSA forgery vs hashing** → 3.4. **Non-repudiation assumptions** → 3.5. **Operation counts and key-size effect** → 3.2 B2, Figure 6.

Q4 – X.509 Certificates, Chain Validation and Revocation Mechanisms (7 marks)

Shared group work. A three-level hierarchy (offline root CA → issuing CA → end-entity server certificate) was generated for the synthetic domain hospital.test with ECDSA P-256 keys using the cryptography toolkit (permitted by the sheet) for **building and parsing** certificates, CRLs and OCSP responses. The **validation algorithm itself was written by the students** (x509chain.py): every accept/reject decision, the order of the checks and the reason strings are our own code.

4.1 Shared work – X.509 structure, hierarchy and chain of trust

Table 21. *Fields and extensions of the three certificates built for the experiment*

X.509 v3 field	Root CA	Issuing (intermediate) CA	End-entity (server)
version	v3	v3	v3
serial	0x1	0x10	0x1001
signature algorithm	ecdsa-with-SHA256	ecdsa-with-SHA256	ecdsa-with-SHA256
issuer	Medical Root CA	Medical Root CA	Medical Issuing CA
subject	Medical Root CA	Medical Issuing CA	pump-gateway.hospital.test
not before	2021-05-16	2024-12-06	2026-10-07
not after	2040-07-15	2029-11-10	2027-01-05
public key	EC P-256 (secp256r1)	EC P-256 (secp256r1)	EC P-256 (secp256r1)
basicConstraints	CA=yes, pathLen=1	CA=yes, pathLen=0	CA=no, pathLen=None
keyUsage	keyCertSign, cRLSign	keyCertSign, cRLSign	digitalSignature
DER size (bytes)	407	410	475

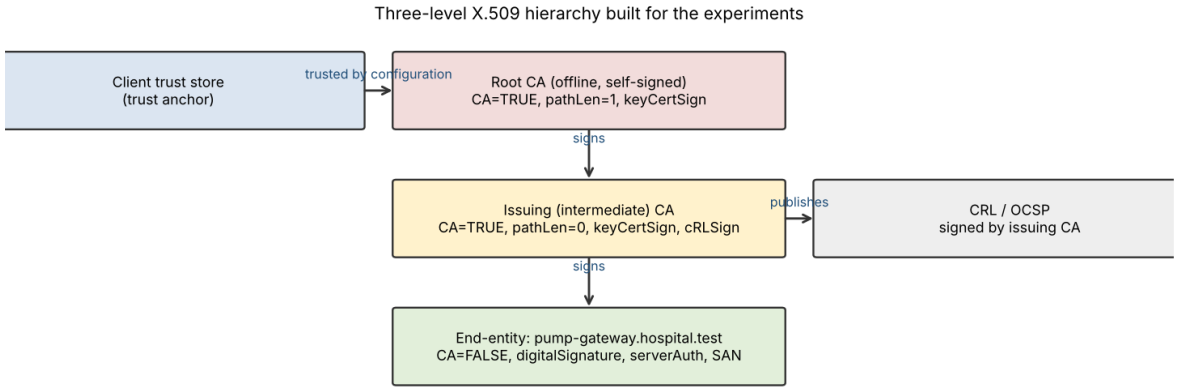


Figure 7. *Three-level hierarchy used in the experiments; the client trusts the root by configuration*

Fields and what they protect. *Serial number* uniquely identifies a certificate for revocation; *issuer* and *subject* names link a certificate to its signer and owner; the *validity period* limits how long a stolen or misissued certificate is usable; the *subject public key* is the key being certified; the *signature algorithm and value* let anyone verify that the issuer vouched for the exact byte string (the TBSCertificate) – a one-byte change breaks the signature (test F4). **Extensions:** *basicConstraints* (CA flag, path length) says whether the key may sign other certificates and how deep a chain below it may go; *keyUsage* limits the technical purposes (digitalSignature, keyCertSign, cRLSign); *extendedKeyUsage* restricts the application (serverAuth); *subjectAltName* carries the DNS names the certificate is valid for. Critical extensions that a client does not understand must cause rejection.

Why a chain of trust. A client cannot know millions of server keys. It holds a small number of **trust anchors** (root certificates) configured out of band; the root signs issuing CAs, which sign end-entity certificates, so one signature chain transfers trust from the anchor to the server key. The root key is kept **offline** (rarely used) and the exposed issuing CA can be replaced if compromised; path-length and name constraints limit what a subordinate CA may do. A certificate binds a **name** to a **key**, it does not say the owner is honest.

4.2 Student B – Trust model and validation logic

VALIDATE(chain = [leaf, CA_1 ... root], trust_store, hostname, now, revocation_info, policy):

1. for each i: if issuer(chain[i]) != subject(chain[i+1]) -> reject "issuer/subject"
2. for each i: if signature of chain[i] does not verify with key(chain[i+1]) (root: self-signature) -> reject "signature"

3. for each certificate: if now < notBefore or now > notAfter -> reject "validity"
4. for each certificate above the leaf: require basicConstraints CA = TRUE; pathLen(chain[i]) >= number of CA certificates below it -> reject "basicConstraints"
5. each issuer has keyUsage keyCertSign; leaf has digitalSignature and EKU serverAuth -> reject "keyUsage"
6. hostname matches a subjectAltName entry (exact or single-label wildcard) -> reject "name"
7. for each certificate below the root: obtain CRL/OCSP status; revoked, stale, or unavailable under FAIL-CLOSED -> reject "revocation"
8. the last certificate is (byte-for-byte) in the trust store -> reject "trust anchor"
accept

Table 22. *Validation steps and the attacks they prevent*

Step	Check	Attack or fault it prevents
1	Issuer name = next subject	Chains assembled from unrelated certificates
2	Signature of every link	Forged or modified certificates (F3b, F4)
3	Validity periods of all certificates	Use of expired or not-yet-valid certificates (F1, F2, F12)
4	CA flag and path length	A leaf or a constrained CA minting new certificates (F6, F8)
5	keyUsage and extended key usage	A key certified for one purpose used for another (F9, F10)
6	Hostname vs subjectAltName	A valid certificate for another site presented for this one (F5)
7	Revocation status	Use of a compromised or withdrawn certificate before it expires (F7)
8	Anchor in the trust store	A perfectly valid chain from an unknown, attacker-run CA (F11)

Why trust in the root cannot itself be verified cryptographically

A chain ends in a **self-signed** root: its signature verifies under its own public key, which proves nothing, because anybody can generate a key pair and issue a self-signed certificate that claims any name. Cryptography can only **transfer** trust from a key that is already trusted to the key it certifies; the first key must be trusted by a decision that is not cryptographic: it is shipped with the operating system, browser or device ROM, admitted under audited CA policies, or installed by an administrator. Test F11 shows this: a chain that is

cryptographically perfect is rejected because its root is not in the trust store. The consequence is that the trust store is a high-value target and that one wrongly trusted root affects every name.

How X.509 binds an identity to a public key, and why that makes a signature meaningful

A signature only proves that **some private key** produced it. The certificate states, and the CA signs, that key K belongs to subject S (a server name, a manufacturer, a person) after the CA has checked the identity. Validation (steps 1–8) establishes that the statement is authentic, current, within the CA's permitted scope, not revoked and anchored in a trusted root. Only then can "signed by K" be read as "signed by S"; this is what gives a signature the meaning needed for authentication and, together with exclusive key control (Q3), for non-repudiation. F5 shows the binding is to a **name**: a valid certificate for another name is refused.

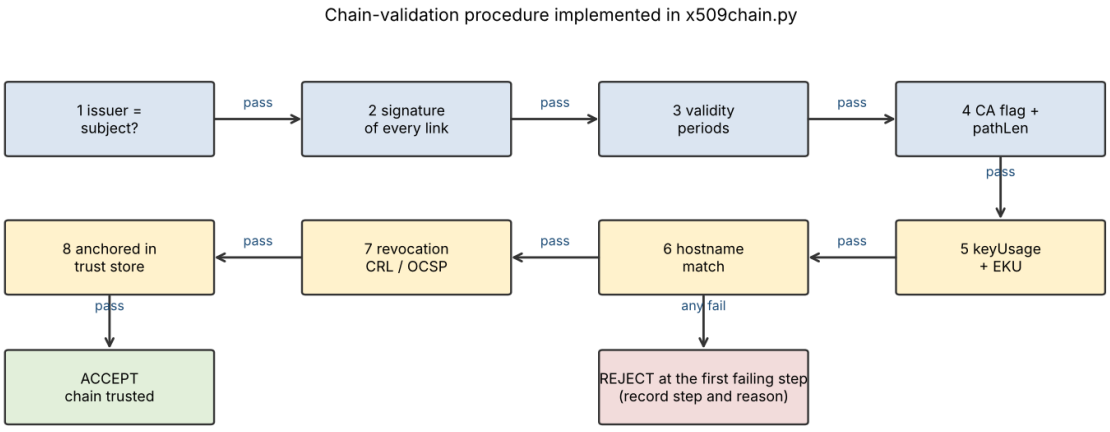


Figure 8. *Chain-validation procedure implemented in x509chain.py*

Table 23. *Trace of a valid chain (accepted)*

Step	Result	Detail
1 issuer/subject match	ok	3 certificates link up
2 signature	ok	all signatures verify
3 validity period	ok	all valid on 2026-11-06
4 basicConstraints	ok	CA flags and path length respected
5 keyUsage	ok	issuers may sign certificates; leaf may authenticate a server
6 name match	ok	"pump-gateway.hospital.test" matches ['pump-gateway.hospital.test']
7 revocation	ok	CRL checked: not revoked

8 trust anchor	ok	anchored at trusted root "Medical Root CA"

4.3 Student A – Failure-case experiments and revocation

Failure cases (13 test cases)

Table 24. *All 13 failure cases were rejected at the expected step (13 of 13)*

ID	Case	Expected failing step	Observed step	Result	Reason reported by the validator
F1	Expired end-entity certificate	3 validity period	3 validity period	rejected	"pump-gateway.hospital.test" expired on 2026-10-02
F2	Certificate not yet valid	3 validity period	3 validity period	rejected	"pump-gateway.hospital.test" not valid before 2026-11-16
F3a	Wrong issuer: names do not link (issued by "Rogue CA")	1 issuer/subject match	1 issuer/subject match	rejected	issuer of "pump-gateway.hospital.test" is "Rogue CA", next certificate is "Medical Issuing CA"
F3b	Wrong issuer key: right issuer name, signed with attacker key	2 signature	2 signature	rejected	signature of "pump-gateway.hospital.test" does not verify under "Medical Issuing CA"
F4	Tampered certificate field (one byte of the subject changed)	2 signature	2 signature	rejected	signature of "pump-gatewaz.hospital.test" does not verify under "Medical Issuing CA"
F5	Hostname mismatch (certificate for another name)	6 name match	6 name match	rejected	"billing.hospital.test" not in subjectAltName ['pump-gateway.hospital.test']
F6	End-entity certificate used as a CA	4 basicConstraints	4 basicConstraints	rejected	"pump-gateway.hospital.test" has CA=FALSE but issued another certificate

F7	Revoked certificate (serial in the issuer's CRL)	7 revocation	7 revocation	rejected	serial 1001 is listed in the CRL of "Medical Issuing CA"
F8	Path length exceeded (CA below an intermediate with pathLen = 0)	4 basicConstraints	4 basicConstraints	rejected	pathLen 0 of "Medical Issuing CA" exceeded (1 intermediate CA(s) below)
F9	Key usage violation (leaf without digitalSignature)	5 keyUsage	5 keyUsage	rejected	leaf lacks digitalSignature
F10	Extended key usage violation (clientAuth only)	5 keyUsage	5 keyUsage	rejected	leaf EKU does not include serverAuth
F11	Chain anchored at an untrusted root	8 trust anchor	8 trust anchor	rejected	"Other Root CA" is not in the trust store
F12	Expired intermediate CA	3 validity period	3 validity period	rejected	"Medical Issuing CA" expired on 2026-11-01

Notes: F3a (issuer name mismatch) and F3b (right name, wrong signing key) are different failures caught at steps 1 and 2; F4 changes one byte of the subject and is caught only because the signature covers every byte; F6 shows that a leaf certificate cannot be turned into a CA; F8 shows pathLen = 0 stopping a rogue sub-CA; F11 shows that a chain can be cryptographically perfect and still untrusted. The anchor check comes last in our implementation, so the revocation data of the foreign CAs had to be supplied in F11 to reach step 8 (a production client checks the anchor early).

Revocation: CRL versus OCSP and the effect of policy

Table 25. *Revocation is detected only if it is checked, and the failure policy matters*

Situation (revoked or good leaf)	**Result**	**Failing step**	**Reason**
Revoked leaf, revocation checking NOT performed	ACCEPTED	-	accepted
Revoked leaf, CRL checked	rejected	7 revocation	serial 1001 is listed in the CRL of "Medical Issuing CA"

Revoked leaf, CRL server unreachable, FAIL-OPEN policy	ACCEPTED	-	accepted
Revoked leaf, CRL server unreachable, FAIL-CLOSED policy	rejected	7 revocation	revocation status unavailable and policy is FAIL-CLOSED
Stale CRL (older than nextUpdate), revoked leaf not yet listed	rejected	7 revocation	CRL is stale (past nextUpdate)
Revoked leaf, OCSP response checked	rejected	7 revocation	OCSP responder reports REVOKED
Good leaf, OCSP response checked	ACCEPTED	-	accepted

Table 26. *CRL growth; an OCSP response is constant (264 bytes good, 282 bytes revoked; request 69 bytes)*

Revoked entries in the CRL	**CRL size (bytes)**	**Build time (ms)**	**Parse + verify + lookup (ms)**
0	204	0.1	0.09
10	429	0.1	0.10
100	2,411	0.5	0.13
1,000	22,211	5.3	0.44
10,000	220,213	175.1	4.03

What a fail-open revocation policy means

A **fail-open** client treats a certificate as good when it cannot obtain revocation information (responder down, network blocked, response missing). An attacker who has stolen a key and whose certificate was revoked can therefore simply block the revocation traffic: in Table 25 the revoked certificate was accepted under fail-open and rejected under fail-closed. Revocation then protects only against attackers who cannot interfere with the network, which is rarely the assumed attacker. **Fail-closed** is safe but turns every outage of the CA infrastructure into an outage of the service; the compromise is stapled responses, cached lists with a defined lifetime, and short-lived certificates that need no revocation.

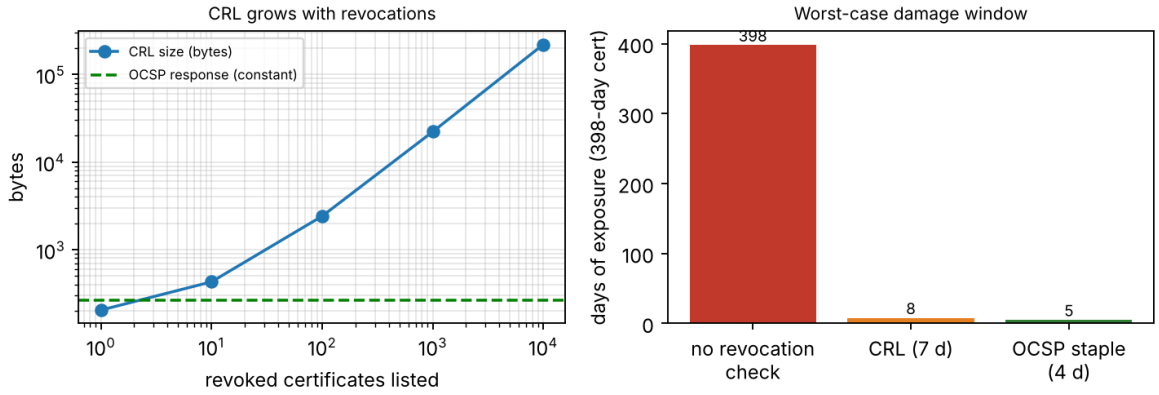


Figure 9. *Left: CRL size grows with the number of revocations while an OCSP response stays constant. Right: worst-case damage window of a mis-issued 398-day certificate*

Table 27. *CRL, OCSP and stapling compared*

Property	**CRL**	**OCSP (online responder)**	**OCSP stapling**
Freshness	Up to nextUpdate (hours–days)	Near real-time (response validity minutes–days)	Validity of the stapled response (about 4 days)
Size / bandwidth	Grows with entries (measured: about 22 B per entry)	Small and constant (264 bytes here)	Small, carried inside the handshake
Privacy	No per-site leak (list fetched in bulk)	The CA learns which site the client visits	No leak: the server fetches the response
Availability	Cache lets the client work offline until nextUpdate	Needs the responder at connection time; failure forces a fail-open/closed decision	Server must refresh; client sees it with the handshake
Client cost	Download and parse large lists	One extra round trip	None extra

4.4 Joint engineering decision: exposure window and mitigations

If a certificate is mis-issued or its key stolen, the damage lasts until clients stop accepting it. For clients that never check revocation this is the **remaining lifetime**; for clients that check, it is the detection delay plus the freshness of the revocation data. The values below are a worst-case model (days):

Table 28. *Worst-case exposure window in days (model: min(lifetime, detection delay + freshness))*

Certificate lifetime	**No revocation checking**	**CRL (7-day nextUpdate), detected after 1 day**	**OCSP stapling (4-day validity), detected after 1 day**	**CRL (7 days), detected after 7 days**
398-day certificate	398	8	5	14
90-day certificate	90	8	5	14
7-day certificate	7	7	5	7

Table 29. *Operational mitigations*

Risk	**Mitigation**	**Residual risk**
Compromise of the issuing CA	Offline root, HSM-protected issuing key, path-length and name constraints, rapid revocation of the intermediate	Time to remove/replace the intermediate in all clients
Mis-issuance by a CA	Certificate-transparency logs and monitoring, CAA records, short-lived certificates (90 days or less)	Detection depends on someone monitoring
Revocation unavailable	OCSP stapling with must-staple, cached CRLs, short-lived certificates that need no revocation	Fail-open policy accepts a revoked certificate (Table 25)
Stolen end-entity key	Keys in hardware, revoke immediately, forward-secret key exchange (CO2)	Window until revocation is seen by clients

Evaluation (Q4)

A signature chain is worthless unless **every** step is performed: the 13 failure cases fail at 8 different steps, and revocation was missed whenever it was not checked (Table 25). The practical recommendation is short-lived certificates plus stapled revocation, with a fail-closed policy for high-assurance systems (accepting the availability cost) and a documented fail-open exception only where availability dominates.

Viva notes

A: which extension makes a leaf unable to issue certificates? basicConstraints CA = FALSE; why is the root offline? it is the single point of trust and is rarely needed.

B: show the validator rejecting F4: one byte of the subject changed → signature check fails at step 2 although names, dates and extensions look correct.

Rubric check – Task 4 (7 marks)

X.509 v3 fields and extensions, chain of trust → 4.1, Table 21, Figure 7. **Chain-validation procedure** → 4.2, Figure 8. **At least five failure cases** → 13 cases, Table 24. **CRL vs OCSP** → 4.3, Figure 9. **Revocation policy, compromise and mis-issuance** → 4.3–4.4.

Q5 – Kerberos: Ticket-Granting, Replay Protection and Mutual Authentication (6 marks)

Shared group work. A simplified Kerberos v5-style system was built (kerberos.py): an Authentication Server (AS) and Ticket-Granting Server (TGS) forming the KDC, a service, and a client. Tickets, authenticators and replies are sealed with the students' own Blowfish-CTR + HMAC-SHA256 (encrypt-then-MAC); the long-term user key is derived from the password with PBKDF2. The experiments use synthetic principals (alice, bob, ehr/server) and a simulated clock.

5.1 Student A – Protocol design and what each message protects

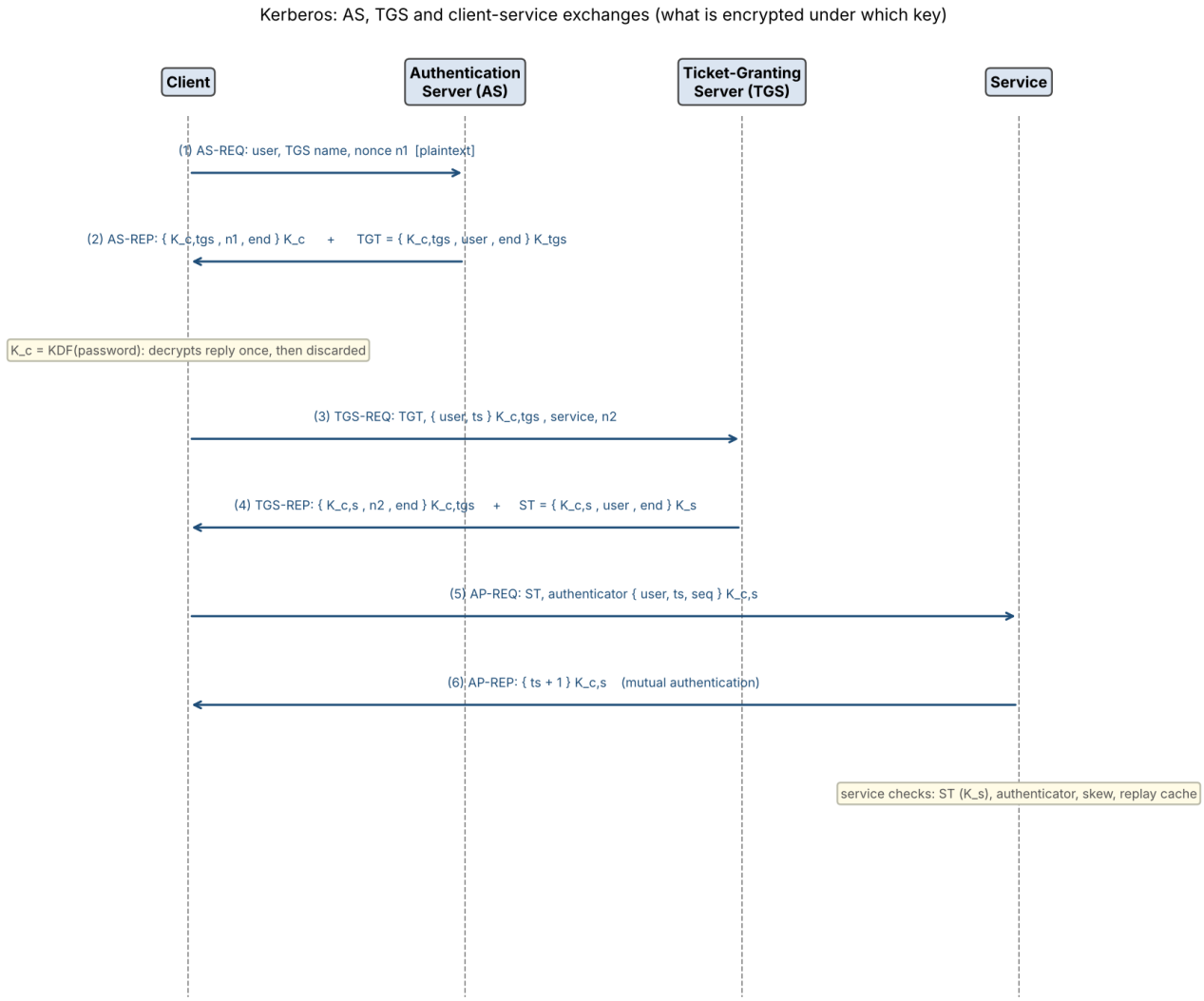


Figure 10. *Kerberos message flow with the key used to protect each part (K_c = key derived from the user's password, K_tgs and K_s = long-term keys of the TGS and the service)*

Table 30. *Kerberos messages, protection and purpose*

Message	Contents	Protected by	Purpose / what it prevents
(1) AS-REQ	user name, TGS name, nonce n1	plaintext (nothing secret)	Asks for a TGT; the password is never sent

(2) AS-REP	{K_c,tgs, n1, end}K_c and the TGT {K_c,tgs, user, end}K_tgs	K_c (password-derived) and K_tgs	Only the genuine user can read K_c,tgs; the client cannot read or alter the TGT; n1 proves the reply is fresh
(3) TGS-REQ	TGT, authenticator {user, ts}K_c,tgs, service name, nonce n2	K_c,tgs	Proves the requester holds the TGS session key (not just a stolen TGT)
(4) TGS-REP	{K_c,s, n2, end}K_c,tgs and the service ticket {K_c,s, user, end}K_s	K_c,tgs and K_s	Gives client and service a fresh shared session key; the service ticket is opaque to the client
(5) AP-REQ	service ticket, authenticator {user, ts, seq}K_c,s	K_s and K_c,s	Proves to the service who the client is, now
(6) AP-REP	{ts + 1}K_c,s	K_c,s	**Mutual authentication:** only a service that could open the ticket knows K_c,s

- **Why the password never travels:** K_c = KDF(password) is computed on the client, used once to open the AS reply and then discarded; the capture of all six messages (652 bytes) contained neither the password nor the derived key (Section 5.2).
- **Why a ticket alone is not enough:** the ticket is not bound to the holder by itself; the holder must also produce an authenticator under the session key inside the ticket, which a thief who only copied the ticket cannot do.
- **What each party learns:** the service learns the client's name, the session key and the ticket lifetime – never the password, K_c or K_c,tgs; the client never learns K_s or K_tgs; the KDC knows every long-term key, so its database is the crown jewel.
- **Why timestamps and a replay cache:** the authenticator contains a timestamp and sequence number; the service accepts it only within ±5 minutes of its own clock and only once (the replay cache), because anyone on the network can copy an authenticator in transit.

Purpose of the TGT and why the long-term key is used only once per session

The **ticket-granting ticket (TGT)** is a reusable credential: after one password-based exchange with the AS the client holds a TGT and the session key K_c,tgs, and can obtain tickets for any number of services without

typing the password again or keeping the password-derived key. The long-term key K_c = KDF(password) is therefore needed **once** (to open the AS reply) and is then erased; everything afterwards is protected by short-lived session keys. This limits the exposure of the password-derived key to a single moment, reduces how much traffic an attacker can collect that is encrypted under it (the offline guessing target in Section 5.2), and means a stolen TGT is valuable only for its lifetime.

Purpose of the authenticator and the timestamp; mutual authentication

A ticket is a static object that can be copied from the network. The **authenticator** {user, timestamp, sequence}K_c,s is created fresh for every request under the session key inside the ticket: it proves that the sender **knows the session key right now** and, because it contains a current timestamp, that it was not recorded long ago; the sequence number and a replay cache stop reuse inside the allowed skew. **Mutual authentication:** the service returns {ts + 1}K_c,s, which only a party able to open the ticket (that is, holding K_s) can compute; the client therefore knows it is talking to the genuine service and not an impostor who merely relays messages.

Table 31. *Which party can read which part of each message*

Part of the exchange	Client	KDC (AS/TGS)	Service	Eavesdropper
(2) {K_c,tgs, n1, end}K_c	reads (after deriving K_c)	created it	no	no (offline guessing if the password is weak)
(2) TGT {K_c,tgs, user, end}K_tgs	holds it, cannot read it	reads	no	no
(3) authenticator {user, ts}K_c,tgs	creates/reads	reads (key from the TGT)	no	no
(4) {K_c,s, n2, end}K_c,tgs	reads	created it	no	no
(4) service ticket {K_c,s, user, end}K_s	holds it, cannot read it	created it (knows K_s)	reads	no
(5) authenticator {user, ts, seq}K_c,s	creates/reads	not on the path	reads	no
(6) {ts + 1}K_c,s	reads and checks	not on the path	creates	no

5.2 Student B – Simulation and experiments

Table 32. *Normal login and service access (wire log of the simulation)*

Message	Direction	Content	Size on the wire (bytes)
(1) AS-REQ	client → AS	plaintext: user, TGS name, nonce n1	15
(2) AS-REP	AS → client	{K_c,tgs, n1, end}K_c + TGT = {K_c,tgs, user, end}K_tgs	332
(3) TGS-REQ	client → TGS	TGT, {user, ts}K_c,tgs, service name, nonce n2	261
(4) TGS-REP	TGS → client	{K_c,s, n2, end}K_c,tgs + ST = {K_c,s, user, end}K_s	253
(5) AP-REQ	client → service	ST, {user, ts, seq}K_c,s	241
(6) AP-REP	service → client	{ts + 1}K_c,s (mutual authentication)	79

The service identified the client as **alice**, the AP-REP verified (mutual authentication = yes), the service ticket lifetime was 1 hour and the TGT lifetime 10 hours. Scanning the 652 captured bytes for the password and for the long-term key: password found = no, key found = no.

Replay of a captured authenticator

Table 33. *A captured authenticator replayed at random delays (the genuine request was accepted first in both runs)*

Replay delay	WITHOUT replay cache (300 replays in total)	WITH replay cache
1-60 s	ACCEPTED: 21	replay detected: authenticator already seen: 42
61-300 s	ACCEPTED: 132	replay detected: authenticator already seen: 122
301-600 s	clock skew too great: 147	clock skew too great: 136

Without a cache the replay is **accepted for 5 minutes** (153 of 153 replays inside the skew window succeeded) and rejected afterwards only because of the clock-skew test. With the cache, all 164 in-window replays were rejected as already seen. An attacker who modifies the timestamp inside the authenticator breaks the MAC and is rejected (authenticator rejected: cannot be built without the session key). The cache costs one entry per request while the authenticator could still be accepted; our code keeps entries for 300 s, but strictly an authenticator stamped up to 300 s in the future stays acceptable for up to 600 s, so a production cache should keep entries for twice the skew (a small price).

Expiry, clock skew and KDC unavailability

Table 34. *Ticket and TGT lifetimes (service ticket 1 h; TGT 10 h)*

Event	**Outcome**	**Reason**
service ticket used 60 s after issue	accepted	-
service ticket used 3599 s after issue	accepted	-
service ticket used 3601 s after issue	rejected	ticket expired
service ticket used 7200 s after issue	rejected	ticket expired
TGT used 35999 s after issue (new ticket request)	accepted	-
TGT used 36001 s after issue (new ticket request)	rejected	TGT expired

Table 35. *Clock skew: the authenticator is accepted only if $|ts - \text{server time}| \leq 300$ s*

Client clock offset (s)	**TGS exchange**	**Service access**
-600	accepted	rejected: clock skew too great
-301	accepted	rejected: clock skew too great
-299	accepted	accepted
0	accepted	accepted
299	accepted	accepted
301	accepted	rejected: clock skew too great
600	accepted	rejected: ticket not yet valid

The +600 s case is refused with "ticket not yet valid" because the ticket was stamped with the client's clock. Kerberos therefore needs **synchronised clocks** (NTP with authentication); a wrong clock makes the system fail closed, which is safe but disruptive.

Table 36. *KDC unavailable*

Situation	**Action**	**Outcome**	**Why**
1. KDC down before login	login (AS exchange)	FAILS	KDC unreachable
2. KDC goes down after the service ticket was obtained	use existing service ticket	WORKS	service never contacts the KDC
3. KDC down, TGT still valid	request a ticket for a new service (TGS exchange)	FAILS	KDC unreachable
4. KDC still down, ticket expired	existing ticket after its lifetime	FAILS	ticket expired

A KDC outage blocks **new logins and new service tickets** but not use of tickets already held, until they expire: short lifetimes limit damage but raise dependence on KDC availability. Mitigation: replicated KDCs, tickets long enough to ride out short outages, monitoring.

Stolen tickets, offline password guessing and mutual authentication

Table 37. *Stolen ticket versus stolen ticket plus session key*

Attack	**Result**
Attacker copies a service ticket from the network and forges authenticators without the session key (100 attempts)	100 of 100 rejected
Attacker also stole the session key from the client's memory/cache	IMPERSONATION SUCCEEDS (until ticket expiry)
Same stolen ticket and key, used after the ticket expired	ticket expired

Table 38. *Offline guessing against the AS reply (PBKDF2, 1,000 iterations)*

Target (AS reply captured, no pre-authentication)	**Result of a 2,000-word dictionary attack**	**Guesses tried**	**Time (s)**	**Guesses per second**
weak password "sunshine7"	found "sunshine7"	786	0.211	3,720

strong random password	not found	2000	0.519	3,855

Without pre-authentication the AS hands the encrypted reply to **anyone** who asks for a user, and the reply can be attacked offline: the weak password sunshine7 was found in 786 guesses (0.2 s); a strong random password was not found. At our measured rate (3,720 guesses/s in Python) an 8-letter lower-case password (2.1×10^{11} possibilities) would take about 1.8 years on one core, but dedicated hardware is thousands of times faster, so password strength is what matters; a 10-character mixed password would need about 7,162,250 core-years. With **pre-authentication** an unsolicited request is refused (refused: pre-authentication failed), but a pre-authentication blob sniffed from the network can still be attacked offline (sunshine7 recovered). A slower key-derivation function raises the cost per guess (0.25 ms at 1,000 iterations, 25 ms at 100,000, i.e. 100× more work per guess). Mitigations: strong passwords or passphrases, high-cost KDF, public-key pre-authentication (PKINIT) or smart cards/second factor, monitoring of AS requests.

Mutual authentication. In 100 runs the genuine service's AP-REP was accepted 100 times, and an impostor service without K_s (which sends a reply sealed with a guessed key) was accepted 0 times: only a party that could open the service ticket can answer correctly.

5.3 Joint conclusions

Table 39. *Kerberos weaknesses and controls*

Weakness	Evidence	Control
Replay inside the skew window	Table 33: 153/153 replays accepted without cache	Replay cache (0 accepted with it)
Weak passwords	Table 38: cracked in 786 guesses	Strong secrets, slow KDF, PKINIT / second factor
Clock dependence	Skew table	Authenticated time sync, ±5 min policy
KDC is a single point of failure and of compromise	Outage table; KDC holds all keys	Replicas, hardware protection, krbtgt key rotation
Stolen session key and ticket	Impersonation until expiry	Short lifetimes, protect credential caches, monitor

Cross-understanding answers

Student B – Why does the client never send its password, and what does the service learn? The password is only used locally to derive K_c, which opens the AS reply; proof of identity is given by the ability to decrypt that reply and later by authenticators under session keys, so nothing replayable that contains the password exists (a scan of the captured traffic found neither the password nor K_c). The service learns only the client's **name**, the **session key** K_c,s, and the **ticket start/end time**; it does not learn the password, K_c or K_c,tgs, and it cannot impersonate the client to a different service.

Student A – What evidence shows that the timestamp/authenticator mechanism prevents replay? Table 33: without a replay cache 153 of 153 replays within five minutes were accepted and the rest rejected by the skew test; with the cache all 164 in-window replays were rejected as "authenticator already seen", and the genuine request was accepted exactly once. Altering the timestamp inside a captured authenticator fails the integrity check, and a thief with only the ticket produced 0 valid authenticators in 100 attempts.

Rubric check – Task 5 (6 marks)

AS, TGS and service message-flow diagram with key labels → Figure 10 and 5.1. **Ticket, authenticator, session key and mutual authentication** → 5.1 and 5.2. **Replay, expired ticket, clock-skew, KDC-failure and offline-password tests** → 5.2, Tables 33, 38. **Weaknesses and controls** → 5.3.

Q6 – Advanced Signature Schemes and Kerberos vs X.509 Comparison (7 marks)

Shared group work. A signature scheme is only as strong as its hash, its randomness and its parameter checks. This task (i) demonstrates a collision-based forgery, (ii) implements and tests ElGamal and Schnorr signatures on toy parameters, (iii) compares all schemes studied, and (iv) compares Kerberos with X.509 as authentication infrastructures.

6.1 Student A – Collision-to-forgery over a truncated digest

Setting. A signature scheme signs a **k-bit truncated** SHA-256 digest with RSA (128-bit toy modulus). **Attack (Yuval's birthday attack):** the attacker writes a benign text B ("authorises a payment of 100 euros to Alice…") and a malicious text E ("… 90000 euros to the account of Mallory…"). He produces 2^m **whitespace variants** of each (single or double spaces at m positions: the text looks the same to a human), computes the truncated digests, and looks for a benign variant and an evil variant with the same digest. The victim is then shown and signs the benign variant; the signature is attached to the evil variant (Figure 11).

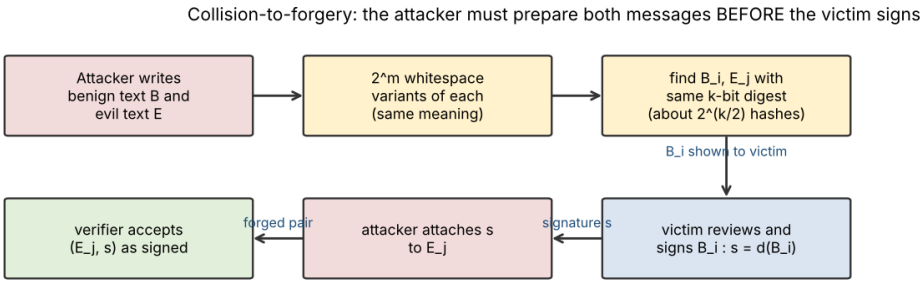


Figure 11. *Collision-to-forgery: both messages must be prepared before the victim signs*

Table 40. *Forgery experiments: every attack produced a message pair the victim never signed that verifies under the signature*

Digest	Whitespace	Variants per	Independent attacks	Successful forgeries	Mean hashes computed	Birthday scale 2^(k/2)	2^k (naive guess)

bits k	position s m	message 2^m					
24	16	65,536	10	10	65,740	4,096	16,777,216
32	19	524,288	4	4	530,136	65,536	4,294,967,296

Table 41. *One complete forgery trace (k = 32)*

Item	Value
Digest length	32 bits (32-bit digest, 128-bit RSA modulus)
Benign text (signed by the victim)	On behalf of the hospital network the finance office hereby authorises a payment of 100 euros to the supplier Alice Medical Ltd for invoice 4471 and confirms that no further payment is due ref-854401
Malicious text (never signed)	On behalf of the hospital network the finance office hereby authorises a payment of 90000 euros to the account of Mallory Trading Ltd for invoice 4471 and confirms that no further payment is due ref-854401
Common truncated digest	0x9c9abc13
Signature s (attached to both)	0x68577ee237110e8882933cd60447e45f
Benign text verifies	Yes
Malicious text verifies with the same signature	Yes
Hashes computed / time	526,032 / 2.7 s

Why the attacker prepares both messages in advance. A collision needs the attacker to control **both** texts; he cannot do this for a message someone else has already signed. For an existing signed message he would need a **second pre-image**, which costs about 2^k. Measured at k = 20: finding a second pre-image took on average 1,791,441 hashes while a free collision took 1,446, a ratio of about 1,239. Hence the attack is practical whenever the attacker can **induce** the victim to sign his chosen benign text (contracts, invoices, certificate requests) and the digest is short; with a 256-bit digest the same method needs about 2^{128} hashes. This is how a

rogue CA certificate was obtained from a CA that still used MD5 (2008): two certificate requests with equal MD5 hashes were prepared and the CA signed the benign one.

Collision attack versus second pre-image attack

A **collision attack** finds **any** two messages $m \neq m'$ with $H(m) = H(m')$, both chosen by the attacker, at cost about 2^(k/2). A **second pre-image attack** is given a particular message m (already signed or published) and must find $m' \neq m$ with $H(m') = H(m)$, at cost about 2^k. The experiments above show the difference concretely (ratio about 1,239 at k = 20) and the forgery works because the attacker supplies **both** texts; a signature that already exists on a document the attacker did not prepare is protected by the much stronger second pre-image barrier.

Real-world forged certificates and what this means for X.509

A certificate is signed by signing the hash of its to-be-signed part, so a hash collision lets the attacker prepare two certificate bodies with the same digest. In 2008 researchers produced a pair of MD5-colliding certificate requests: a harmless website certificate that a commercial CA signed (its serial number and validity time were predictable), and a second body with the same MD5 hash that declared itself a **CA certificate**; the CA's signature on the first was valid for the second, giving them a rogue intermediate CA able to issue certificates for any site. A similar MD5 collision technique was used in the Flame malware (2012) to forge a code-signing certificate. For X.509 this means that every certificate in a chain is only as trustworthy as the hash used in its signature: the CA must not use a hash with known collision attacks, must randomise what the requester can predict (serial numbers), and clients must reject chains signed with such hashes (browsers stopped accepting MD5 and SHA-1 certificates).

Digest length, algorithm choice and a hash policy for signatures

Table 42. *Recommended hash policy*

Decision	Policy recommended	Justification (evidence)
Hash for new signatures and certificates	SHA-256 or stronger (SHA-384/512, SHA-3); **never MD5 or SHA-1**	Collision work 2^128 for 256 bits; measured birthday law 2^(k/2) (Q1); forged certificates and 32-bit demonstration (Section 6.1)
Minimum digest length	256 bits for signatures (so that collision resistance is about 128 bits)	A 128-bit digest gives only 2^64 collision work, within reach of well-resourced attackers

What a CA may sign	Only SHA-2/SHA-3 based signature algorithms; randomised serial numbers (at least 64 bits of entropy); no requester-controlled predictable fields	Prevents the prepared-pair attack of the 2008 forgery
Legacy signatures	Accept MD5/SHA-1 signatures only for historical verification with a timestamp, never for new trust decisions	Existing collisions make SHA-1 signatures forgeable for chosen-prefix attackers
Agility	Algorithm identifiers inside signed data, ability to retire a hash quickly	MD5 and SHA-1 had to be retired in the field

Defences: collision-resistant digests of at least 256 bits; randomised hashing (the signer adds a fresh random salt that the attacker cannot predict before preparing the pair); the signer must not sign a document whose bytes the attacker fully controls without adding his own random element (CAs randomise the serial number); and rejecting hashes with known collision attacks (MD5, SHA-1).

6.2 Student B – ElGamal and Schnorr signatures

B1. ElGamal signatures

Parameters: prime p, generator g; private key x, public key $y = g^x \bmod p$. To sign digest $h = H(m)$: choose k with $\gcd(k, p-1) = 1$; $r = g^k \bmod p$; $s = k^{-1}(h - xr) \bmod (p-1)$. Verify: $0 < r < p$ and $g^h \equiv y^r r^s \pmod p$. **Proof:** $y^r r^s = g^{(xr + ks)}$ and $ks \equiv h - xr \pmod{p-1}$, so the exponent equals h modulo p−1 and Fermat's little theorem gives g^h.

Hand example ($p = 23$, $g = 5$, $x = 6$ so $y = 5^6 \bmod 23 = 8$); digest $h = 7$, $k = 5$ ($k^{-1} \bmod 22 = 9$): $r = 5^5 \bmod 23 =$ **20**, $s = 9(7 - 6 \cdot 20) \bmod 22 =$ **17**. Verification: $g^h = 5^7 \bmod 23 =$ **17**; $y^r r^s = 8^{20} \cdot 20^{17} \bmod 23 =$ **17** → accept. With the altered digest $h = 8$, $g^h = 16 \neq 17$ → reject.

Table 43. *ElGamal verification tests (synthetic invoices)*

Case (64-bit safe prime)	Trials	Accepted	Expected
valid	300	300	all
message altered	300	0	none
r altered	300	0	none

s altered	300	0	none
r >= p (range check)	300	0	none
wrong public key	300	0	none

- **Nonce reuse:** the same r appears in both signatures (yes); solving $s_1 - s_2 = k^{-1}(h_1 - h_2)$ gives k, and then $xr \equiv h_1 - ks_1 \pmod{p-1}$ gives the private key x: recovered = yes, and a forged signature verified = yes. With a nonce space of 2^{17} values one signature was enough (k and x found: yes).
- **Existential forgery without a hash:** if the signature is applied to the message value directly, an attacker who knows only the public key can choose a, b with gcd(b, p−1) = 1 and set $r = g^a y^b$, $s = -r\, b^{-1}$, $m = a\, s \bmod (p-1)$. The pair verifies (yes) although nobody signed anything. **With the hash**, the same (r, s) verified for 0 of 2,000 different texts: the attacker would have to find a text whose digest equals a chosen value (a pre-image).
- **Range check:** the case r ≥ p must be rejected explicitly; otherwise r + p gives a second valid-looking signature (Table above shows 0 accepted).

B2. Schnorr signatures and the role of the challenge

Parameters: primes p, q with q | p−1 and g of order q; private x, public $y = g^x$. **Interactive identification (Figure 12):** the prover picks random r and sends the commitment $t = g^r$; the verifier answers with a random challenge c; the prover responds s = r + cx mod q; the verifier accepts iff $g^s = t \cdot y^c \pmod p$. **Correctness:** g^s = g^(r + cx) = $t \cdot (g^x)^c = t \cdot y^c$. **Non-interactive signature (Fiat–Shamir):** c = H(t ‖ m). **Hand example:** p = 23, q = 11, g = 2 (order 11: $2^{11} \bmod 23 = 1$), x = 7, $y = 2^7 \bmod 23 = 13$; r = 5, $t = 2^5 \bmod 23 = 9$; challenge c = 4; $s = 5 + 4 \cdot 7 \bmod 11 = \mathbf{0}$. Check: $g^s = 2^0 = \mathbf{1}$; $t \cdot y^c = 9 \cdot 13^4 \bmod 23 = \mathbf{1}$ → accept; a wrong response s + 1 is rejected (yes).

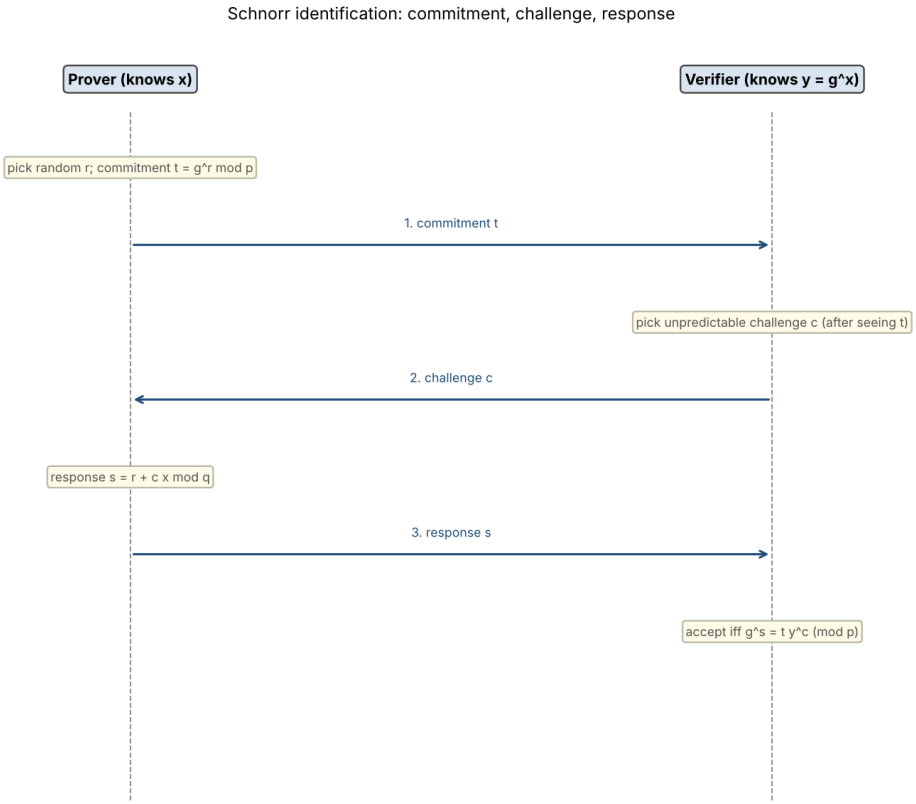


Figure 12. *Schnorr identification: commitment, challenge, response*

Table 44. *Schnorr verification tests*

Case (256-bit p, 64-bit q)	Trials	Accepted	Expected
valid	300	300	all
message altered	300	0	none
t altered	300	0	none
s altered	300	0	none
wrong public key	300	0	none

- **Nonce reuse:** two signatures with the same commitment t (yes) give $s_1 - s_2 = (c_1 - c_2)x$, hence $x = (s_1 - s_2)/(c_1 - c_2) \bmod q$: private key recovered = yes, forged signature verified = yes.
- **Why the challenge must be unpredictable (and chosen after t):** a cheater who knows c in advance picks s at random and sets $t = g^s y^{-c}$; the transcript (t, c, s) then verifies without knowing x. Over 5,000 trials

with the challenge known in advance the cheater succeeded **5,000** times; when the verifier picked c after seeing t the cheater succeeded 473 times, close to the expected 1/q = 455 (toy q = 11). In the Fiat–Shamir version c = H(t ‖ m) is computed after t is fixed, so the attacker cannot choose t after seeing c.

Student B verification – why does weakening the hash weaken the signature even if RSA or DSA is unchanged?

Because the signature algorithm signs only the **digest**: a signature on H(m) is accepted for every message with the same digest. The security of the whole signature is therefore the smaller of (i) the strength of RSA/DSA and (ii) the collision/second-pre-image resistance of H. If the attacker can find another message with the same digest (collision for prepared pairs, second pre-image for existing signatures), the unchanged RSA/DSA signature verifies for it; Section 6.1 forged a signature with a 128-bit RSA modulus over a 32-bit digest in about 5×10^5 hashes without touching RSA.

6.3 Comparison of the signature schemes

Table 45. *Comparison of RSA, DSA, ECDSA, ElGamal and Schnorr (verification counts measured on the toy parameters)*

Scheme	Private/public material	Per-signature randomness	Verification work (measured)	Signature size at ≈ 128-bit security	Security basis and typical failure
RSA hash-then-sign	n, e, d (n = 3,072 bits)	none (PKCS#1 v1.5); salt for PSS	1 modular exponentiation(s), exponent bits: 17	384 B	Factoring; textbook RSA forgeries, weak hash
DSA	p, q, g, x, y	**fresh secret k**	2 modular exponentiation(s), exponent bits: 127, 128	64 B (q = 256 bits)	Discrete log; repeated/biased k reveals x
ECDSA (P-256)	curve point, d	**fresh secret k**	2 scalar multiplications	64 B	ECDLP; repeated/biased k reveals d
ElGamal	p, g, x, y	**fresh secret k**, gcd(k, p−1) = 1	3 modular exponentiation(s),	≈ 2 × 3,072 bits = 768 B	Discrete log; no hash → existential forgery; k reuse

			exponent bits: 63, 62, 63		
Schnorr	p, q, g, x, y	**fresh secret r**	2 modular exponentiation(s), exponent bits: 62, 63	64 B (challenge + response)	Discrete log (proof in random-oracle model); r reuse reveals x

Table 46. *Forgery resistance and non-repudiation assumptions*

Scheme	**Forgery resistance rests on**	**Non-repudiation assumption**
RSA hash-then-sign	Hardness of the RSA problem + collision/second-pre-image resistance of the hash + proper padding	Only the owner can compute d: key held in hardware, certificate binds key to owner, time-stamp
DSA / ECDSA	DLP / ECDLP + hash + **secret, unpredictable per-signature k**	As above, plus a sound random generator: a repeated k exposes the key and lets the owner deny ("the key leaked")
ElGamal	DLP + **hash applied** + gcd(k, p−1) = 1, fresh k	As above; unhashed use is existentially forgeable, so signatures prove nothing
Schnorr	DLP in the random-oracle model; challenge = hash of commitment and message	As above; a repeated commitment reveals x

Observations. RSA verification is the cheapest (17 squarings + 1 multiplication) because e is small, signing is the expensive side; DSA, ElGamal and Schnorr verify with two or three full exponentiations; ECDSA needs two scalar multiplications but with much smaller keys. Every scheme that needs a fresh secret per signature (DSA, ECDSA, ElGamal, Schnorr) leaks its private key if that secret repeats (demonstrated for DSA, ECDSA, ElGamal and Schnorr); RSA with a deterministic padding does not have this failure. All require a collision-resistant hash and range checks on the signature components.

6.4 Student B – Kerberos versus X.509 and non-repudiation

Table 47. *Kerberos and X.509 compared*

Aspect	Kerberos	X.509 / PKI
Key type	Symmetric (secret key shared with the KDC)	Asymmetric (key pair per entity; certificate binds name to public key)
Cryptography	Symmetric keys only; long-term key shared with the KDC	Public-key signatures; certificates bind names to public keys
Trusted third party	Online KDC (AS + TGS) must be reachable to get tickets	CAs are needed only at issuance; verification is offline given the trust store and revocation data
Scope	A realm / organisation with central administration	Global, cross-organisation (web, code signing, e-mail)
Proof offered to a third party	None: the service and the KDC share keys, so either could have produced a ticket	A signature can be verified by anyone and supports non-repudiation (with the conditions of Q3)
Time	Needs synchronised clocks (±5 min); replay cache	Needs time for validity checks, but no tight synchronisation
Revocation	Short ticket lifetimes; disable the account at the KDC	CRL / OCSP / short-lived certificates (Q4)
Scalability	One key per user/service with the KDC; cross-realm trust needs extra setup; KDC load grows with logins	One certificate per entity; verification needs no CA contact; hierarchy and cross-certification scale globally
Typical environment	Campus / enterprise single sign-on (Active Directory style)	Web, e-mail, code and document signing, device identity
Failure of the central service	No new tickets (Q5)	Cannot issue or revoke; existing certificates keep working until expiry
Compromise of the central secret	KDC database / krbtgt key: forge any ticket in the realm	CA key: issue certificates for any name

Non-repudiation. X.509 with signatures can give non-repudiation because the private key is held by **one** party and anyone can verify with the public key. Kerberos cannot: the ticket and authenticator are protected by

symmetric keys that the KDC (and the service) also know, so any of them could have produced the same protected data; a third party shown a ticket cannot tell who created it. **Single point of failure / trust anchor:** the KDC is a single point of failure (no new tickets when it is down – Section 5.2) **and** a single point of compromise (its database and the krbtgt key forge any ticket); in X.509 the corresponding risk is the **trust anchor** – a compromised or mis-trusted root or CA can issue valid certificates for any name, and the trust store cannot verify its own roots (Section 4.2).

6.5 Student B – Key relationships as the network grows

Table 48. *Number of keys or relationships to establish and protect*

Users n	Pairwise shared keys n(n−1)/2	Kerberos: one key per user with the KDC (+ one per service)	PKI: key pairs + 1 root	Certificates to issue
10	45	10	11	10
100	4,950	100	101	100
1,000	499,500	1,000	1,001	1,000
10,000	49,995,000	10,000	10,001	10,000
1,000,000	499,999,500,000	1,000,000	1,000,001	1,000,000

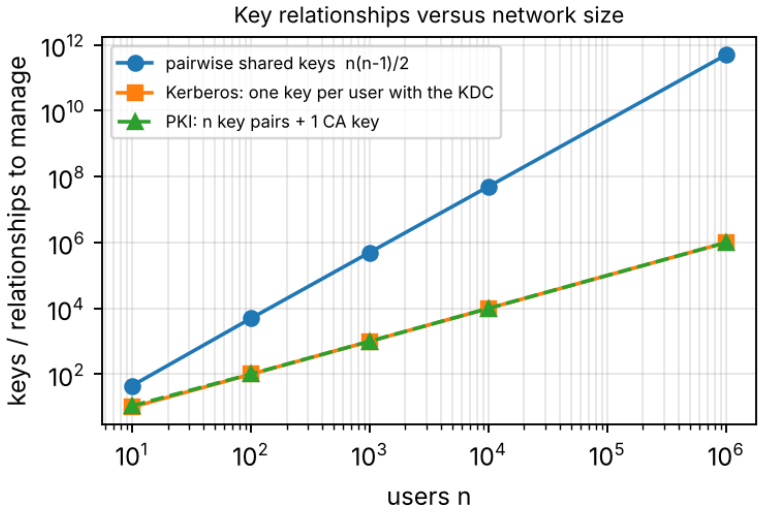


Figure 13. *Pairwise keys grow quadratically; Kerberos and PKI grow linearly with the number of users*

For a million users pairwise keys would need about 5×10^{11} keys, while Kerberos or a PKI needs one long-term secret or key pair per user; the price is the central service (KDC or CA) that must be protected and kept available. **Recommendations:** a single-organisation campus or hospital login with many internal services is a natural Kerberos case (efficient symmetric crypto, central account control); a legally binding contract between independent organisations needs X.509 signatures with a time-stamp, because only a signature offers evidence a third party can verify.

Student A verification – why can a Kerberos ticket prove who a user is to a service, yet not prove to a third party that the user performed an action?

To the service, the ticket plus authenticator prove that someone who holds the session key issued by the KDC is present now: only the KDC and the client know K_c,s besides the service itself. A third party cannot use this as evidence: the service (and the KDC) hold the same symmetric keys and could have created the same ticket and authenticator without the user, so the data does not single out the user; nothing in it is signed with a key that only the user controls, and the authenticator is a short-lived timestamp that is not tied to the content of any particular action. A digital signature with the user's private key would be needed.

Student A and B cross-verification (viva notes)

A explains B's Schnorr work: the challenge must be unpredictable and chosen after the commitment, otherwise (t, c, s) can be simulated without x (5,000/5,000 cheats succeeded when c was known).

B explains A's forgery: a short digest lets the attacker find a benign/evil pair in about 2^(k/2) hashes (about 530,000 hashes on average for k = 32); the victim signs the benign text and the signature also verifies the evil text; a 256-bit digest makes the same attack cost about 2^{128}.

Rubric check – Task 6 (7 marks)

Collision-to-signature-forgery → 6.1, Figure 11, Table 40. **ElGamal and Schnorr derivation, hand example, valid/invalid tests** → 6.2. **Challenge unpredictability and nonce reuse** → 6.2 B1–B2. **Scheme comparison** → 6.3, Table 45. **Kerberos vs X.509, key relationships, recommendations** → 6.4–6.5.

Q7 – Design a Secure Authentication and Integrity Scheme from First Principles (10 marks)

Problem. A manufacturer distributes firmware to thousands of infusion pumps in hospitals. Each pump must be sure that an update (a) has not been altered, (b) really comes from the manufacturer, and (c) cannot later be denied by the manufacturer. Pumps have limited memory, are often offline, have unreliable clocks, and some must authenticate to the hospital's central server. The six stages of the sheet are followed in order.

7.1 Stage 1 – Independent design proposals (retained as first written)

The two proposals were written separately before any comparison. They are reproduced without correction; some of their weaknesses are exactly what Stage 2 and the experiments expose.

Table 49. *Initial proposals of Student A and Student B (as submitted at Stage 1)*

Item	Student A's proposal: "Signed digest with one embedded manufacturer key"	Student B's proposal: "Shared-key MAC and Kerberos tickets"
Security goals and the mechanism for each	Integrity: SHA-256 of the image. Authenticity and non-repudiation: ECDSA signature by the manufacturer over the digest. Pump authentication to the hospital: X.509 client certificate over TLS. Expiry: a "valid until" date inside the signed data checked against the pump clock.	Integrity and authenticity: HMAC-SHA256 over the image under one secret key. Non-repudiation: manufacturer keeps logs of what it issued. Pump authentication to the hospital server: Kerberos-style tickets from the hospital's KDC.
Primitives and where used	Hash (manufacturer, pump); signature (manufacturer signs, pump verifies); certificate (device and server identity).	MAC (manufacturer computes tag, pump checks); Kerberos ticket and authenticator (pump ↔ server); no certificates.
Key / certificate material held	Manufacturer: one long-term signing key pair (in an HSM). Pump: manufacturer public key in ROM; its own key pair and certificate. Server: certificate.	Manufacturer and every pump: the same 128-bit "fleet key" installed at the factory. Each pump also shares a long-

		term key with the hospital KDC; the server shares a key with the KDC.
Message flow for one update	Manufacturer: sig = Sign(SHA-256(image) ‖ valid_until) → upload image + sig; pump downloads, checks clock ≤ valid_until, verifies signature with the embedded key, installs.	Manufacturer: tag = HMAC(fleet key, image ‖ version) → upload; pump downloads, recomputes the tag and compares, installs. When online the pump gets a service ticket from the KDC and presents it to the server.

7.2 Stage 2 – Comparison record

Table 50. *Design-comparison record: how the two initial ideas shaped the final design*

Question	Finding
Where the designs agree	Both hash or MAC the whole image so that any change is detected; both keep the secret off the distribution channel (the channel is untrusted); both separate image verification from pump-to-server authentication; neither needs confidentiality of the firmware.
Where they differ	A uses public-key signatures and certificates (one-to-many, verifiable by anyone); B uses symmetric keys (cheap, small) and Kerberos-style tickets (central KDC). A relies on the pump clock; B on a shared fleet secret and on KDC reachability.
Strengths of A	Offline verification by the pump; third parties can verify the signature (non-repudiation); no secret in the pump. Simulation: valid update accepted (yes), modified image rejected (yes).
Weaknesses of A (simulated)	Clock: a pump with a clock reset to 1970 or set back accepts an old package; with the clock in the future it rejects a good one (Table 57). Rollback: an old genuine, validly signed image is accepted (yes). No revocation (yes); one key for all time, so theft is catastrophic.
Strengths of B	Tiny code, small metadata (32-byte tag), very cheap verification; works offline for the update itself.

Weaknesses of B (simulated)	Fleet key extracted from ONE pump forges updates for every pump (yes); a pump or the hospital can create a tag the manufacturer never made, so there is no non-repudiation (yes); Kerberos needs an online KDC and synchronised clocks, which pumps often lack; the manufacturer's KDC would have to serve every hospital.
Missing threats / cases in BOTH	Rollback to an old version; stolen signing key; offline pump; wrong clock; revoked key; replay of old authentication messages; atomic installation; nonce reuse in signing.
Ideas retained	From A: public-key signature over a signed manifest, public key (root) embedded in the pump, device certificates. From B: very compact metadata, replacing timestamps by fresh **nonces** for authentication, symmetric tags only where both ends share a key legitimately (inside a session).
Ideas dropped or added	Dropped: shared fleet key; Kerberos for pumps; wall-clock expiry. Added: two-level certificates with a root key kept offline, monotonic version counter (anti-rollback), key epochs and root-signed revocation list, **signed time floor**, deterministic ECDSA nonces, challenge–response mutual authentication.

7.3 Stage 3 – Joint final design

Threat model

- **The attacker** controls the distribution channel and the network: he can read, modify, drop, delay, replay and re-order any message, host his own images, obtain every old genuine release, and run his own servers. He may reset or manipulate a pump's clock and may physically extract the secrets of **one** pump.
- **He may at some time steal a release-signing key** (but not the offline root key), or obtain an old expired or retired signing key.
- **He cannot** break SHA-256 or ECDSA P-256, modify the pump's ROM (which holds the root public key and the verification code), or forge a certificate without a root or signing key.
- **Out of scope** (named so that the limits are honest): malware inserted before the manufacturer signs (build-pipeline compromise), invasive hardware attacks on the ROM, jamming/denial of service.

Security goals and the mechanism for each

Table 51. *Security goals and mechanisms*

Goal	Mechanism	Source in this assignment
(a) Image has not been altered	SHA-256 of the image inside the signed manifest; the pump hashes the image as it is stored and compares	Q1: 256-bit digest, collision work 2^128 (measured 2^(k/2) law); Q2: a bare digest is not enough
(b) Image really comes from the manufacturer	ECDSA P-256 signature over the manifest; signing key certified by the root key pinned in pump ROM; chain checked as in Q4	Q3, Q4
(c) Manufacturer cannot deny issuing it	Asymmetric signature (only the HSM holds the private key); signed manifests archived with their time-stamp; deterministic nonces so the key cannot leak through a repeated k	Q3 (non-repudiation conditions, nonce reuse), Q6
Old image presented again (rollback, replay)	Monotonic version counter: install only if version > installed	Q5 replay lesson; Table 56 C6
Compromised or retired signing key	Key epochs plus a root-signed revocation list (sequence number, revoked serials); pump keeps an epoch floor and revoked set	Q4 revocation; fail-open lesson
Unreliable clock	Validity is judged against a **signed time floor** (the largest issued_at seen), never the raw clock	Q4 validity checks; Q5 clock dependence
Offline pump	Package is self-contained (manifest, signature, certificate, optional revocation list); nothing needs the network except obtaining the file	Q4, Q5 (KDC outage)
Pump ↔ hospital server authentication	Certificate-based challenge–response with fresh nonces from both sides, certificates signed by the manufacturer root	Q3, Q4, Q5 (replay, mutual authentication)

Limited memory	Compact binary formats (66-byte manifest, 88-byte certificate body), streaming SHA-256, two ECDSA verifications	Table 59

Key and certificate management

Table 52. *Issuance, storage, rotation and revocation*

Key / certificate	Issuance	Storage	Rotation / lifetime	Revocation
Root key pair	Generated at a two-person ceremony	Offline HSM; public key burned into every pump ROM	Not rotated in service (new pumps can ship a new root; old pumps need a secure-boot hardware update)	Cannot be revoked in the field: protect it above all else
Release-signing key + certificate (epoch e)	Root signs a compact certificate: serial, epoch, role, public key, not-before, not-after	Online HSM at the manufacturer, role-restricted, rate-limited	One year; new epoch each year; the pump learns the new epoch from the first update it accepts	Root-signed revocation list (serials) and epoch floor carried in the next updates; offline pumps learn it at their next update
Device key + device certificate	Key generated inside the pump's secure element at manufacture; root (or manufacturing CA) signs the certificate	Never leaves the pump	Device lifetime (10 years)	Serial placed on the revocation list at the hospital server when a pump is lost or decommissioned
Hospital server key + certificate	Issued by the same root (or a hospital CA cross-certified)	Server HSM	1–3 years	Serial on the revocation list; pump trusts only server

				certificates with the server role

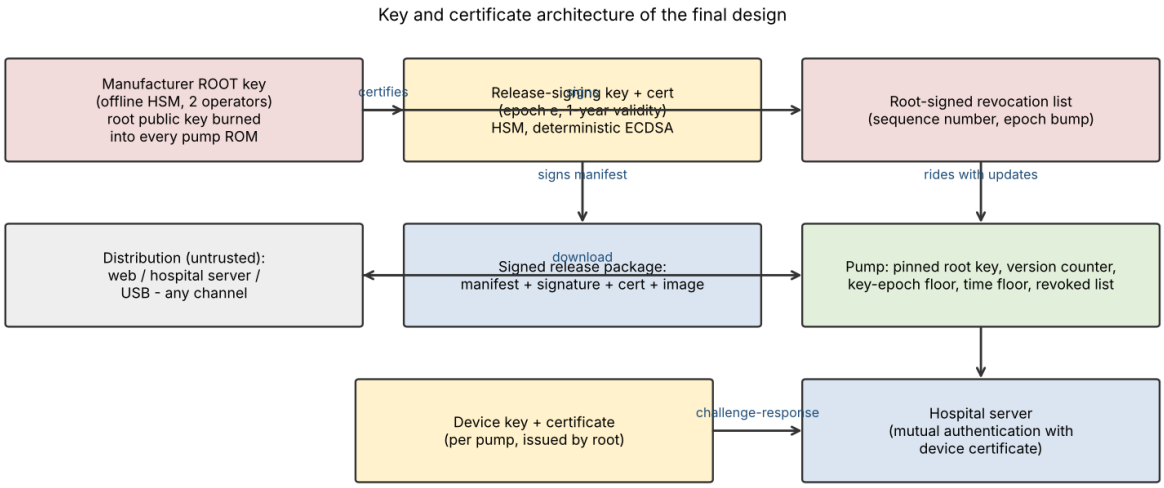


Figure 14. *Key and certificate architecture of the final design*

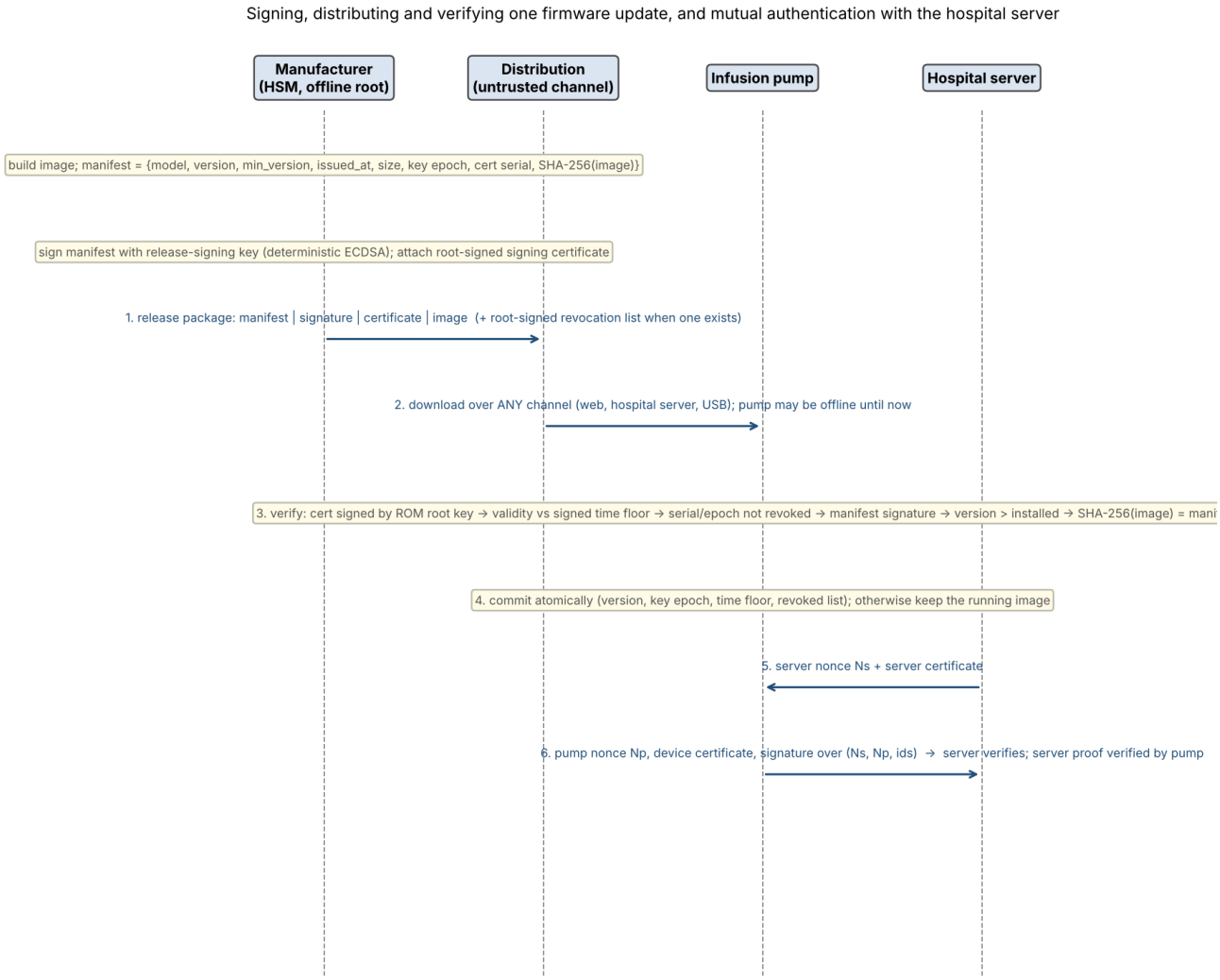


Figure 15. *Message flow for signing, distributing and verifying an update, and for mutual authentication*

Behaviour when the pump is offline or its clock is wrong

Table 53. *Offline and clock behaviour*

Situation	**What the final design does**
Pump offline when the update is published	The package is a self-contained file: the pump verifies it locally whenever it receives it (via a gateway, USB or later connection); no server contact is needed (Section 7.4, offline test).
Pump clock reset to 1970 or set far into the future	The raw clock is not used to accept or reject; the validity of the signing certificate is compared with the **time floor** (the largest signed issued_at

	the pump has accepted), so a legitimate update is not blocked and an expired key is still refused (Table 57).
Pump never saw a recent time	The first update sets the time floor; before that, the version counter and the revocation list are the protection; an attacker with an expired key can pass only until the pump has seen a later signed time (residual risk, Section 7.4).
Revocation list not yet received	The pump cannot know; the epoch floor and the next update close the window (cases C14/C15).

Pseudocode

```
SIGN-RELEASE(image, version, signer, model, issued_at):          # runs in the manufacturer HSM
  h        <- SHA-256(image)
  manifest <- model | version | min_version | issued_at | |image| | signer.epoch | signer.cert_serial | h      (66 bytes)
  k        <- HMAC-SHA256(signer.d, SHA-256(manifest) | counter) mod (n-1) + 1       # deterministic nonce, never random
  sig      <- ECDSA-P256-Sign(signer.d, manifest, k)                                 # 64 bytes
  return  package(manifest, sig, signer.cert, signer.cert_sig, image, optional revocation list)

VERIFY-UPDATE(pkg, pump):                                  # runs in the pump; any failure -> keep old image, log step
  1  reject if lengths of manifest/cert/signatures are wrong
  2  reject if manifest.model != pump.model
  3  if pkg has a revocation list RL: if ECDSA-Verify(root_pub, RL) and RL.seq > pump.rl_seq:
pump.revoked <- pump.revoked U RL.serials   (remember on commit)
  4  reject unless ECDSA-Verify(root_pub, cert, cert_sig) and cert.role = SIGNER
  5  ref <- max(pump.time_floor, manifest.issued_at)              # signed time, NOT the raw clock
     reject if ref < cert.not_before or ref > cert.not_after
  6  reject if cert.serial in pump.revoked or cert.epoch < pump.epoch_floor
  7  reject unless manifest.key_epoch = cert.epoch and manifest.cert_serial = cert.serial
  8  reject unless ECDSA-Verify(cert.public_key, manifest, sig)
```

9 reject unless manifest.version > pump.version and manifest.version >= manifest.min_version # anti-rollback

10 stream the image through SHA-256; reject unless length = manifest.size and digest = manifest.hash

11 COMMIT atomically: pump.version, pump.epoch_floor, pump.time_floor, pump.revoked, pump.rl_seq; activate the new image

accept

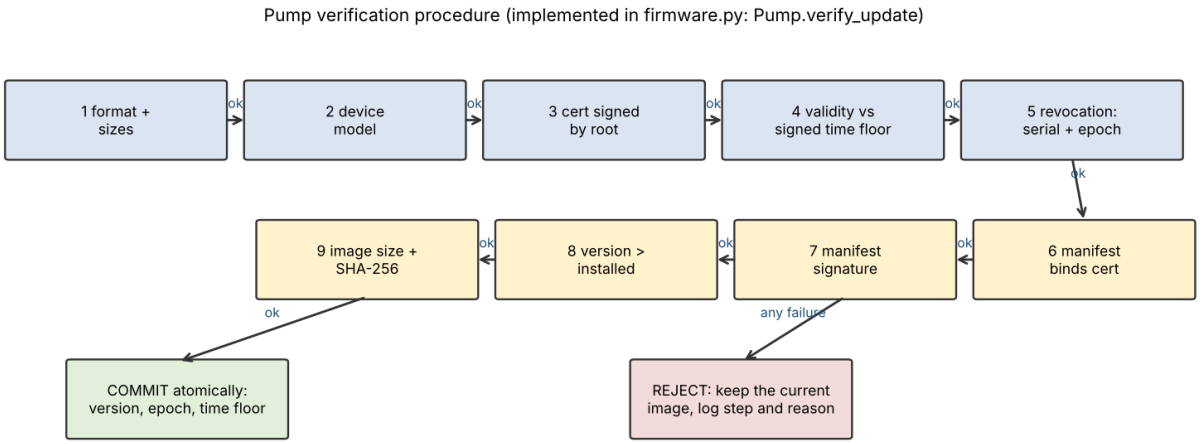


Figure 16. *Pump verification procedure (implemented in firmware.py)*

7.4 Stage 4 – Validation

Student A – Security argument validation

A1. What each attack needs, and the attacker's effort

Table 54. *Attacker effort for the chosen sizes (SHA-256 digest, 256-bit ECDSA)*

Attack on the update scheme	**Attacker needs**	**Work (bits)**	**Years at 10^12 ops/s**	**Years at 10^18 ops/s**	**Basis**
Prepared-pair collision in the image digest (Q6 style)	two images with equal SHA-256 digests	128	1.1e+19 years	1.1e+13 years	Q1: measured 1.25 x 2^(k/2) for k = 16..36

Second pre-image: another image with a given SHA-256 digest	image with a given SHA-256 digest	256	3.7e+57 years	3.7e+51 years	Q1: measured 2^k for fixed target
Forge a manifest signature without the key	solve ECDLP on P-256 (generic rho)	128	1.1e+19 years	1.1e+13 years	CO2 Q5: generic sqrt(n) attacks
(alternative RSA-2048) factor the modulus	factoring	112	1.6e+14 years	1.6e+08 years	NIST SP 800-57 estimate
Collision if only a 64-bit digest were used	2^32 hashes	64	213.3 days	2.1e-04 days	Q1 extrapolation: 2^32 hashes
Collision with a 32-bit digest (demonstrated in Q6)	about 5×10^5 hashes	16	7.6e-13 days	7.6e-19 days	measured: about 5 x 10^5 hashes with whitespace variants

Using the earlier results. Q1 measured that the hashes needed for a collision follow 1.25·2^(k/2) for k = 16–36 (for example 82,055 hashes at k = 32), and that a pre-image/second pre-image needs about 2^k (ratio to 2^k 0.96–1.11); for k = 256 this extrapolates to 2^128 and 2^256, far beyond any attacker (at 10^18 hashes per second, 2^128 hashes take 1.1e+13 years). Q3 showed that a repeated or predictable ECDSA nonce reveals the private key from two signatures (or one, for a 2^16 nonce space): the design therefore uses **deterministic nonces** (verified in Section 7.4, B3). A 64-bit digest would give only 2^32 collision work (hours on one machine) and a 32-bit digest was forged in 530,136 hashes, so a digest shorter than 256 bits is excluded.

A2. Goal by goal: how each attack is prevented or limited

Table 55. *Goal-by-goal security argument*

Attack	Goal threatened	How the design stops or limits it	Evidence
Modify the image or the manifest	Integrity	Manifest signature covers the image hash; any change fails step 8 or 10	C2, C3, C5

Forge a signature or certificate	Authenticity	Signature must verify under a key certified by the ROM root key; rogue certificate fails step 4	C4, C9, C10
Replace the image and its digest (baseline attack)	Integrity, authenticity	The digest is inside a signed manifest, so replacing it breaks the signature	C3
Install an old genuine version	Integrity of the software state	Monotonic version counter	C6
Reuse an old or expired signing key	Authenticity	Validity against signed time floor; key epoch floor	C7, C13
Use a stolen signing key	Authenticity	Limited by the HSM, epoch rotation and the root-signed revocation list; **not stopped on a pump that has not yet received the revocation**	C14 (accepted), C15 (rejected)
Wrong clock, offline pump	Availability and authenticity	Signed time floor; self-contained package	Table 57, offline test
Replay of an authentication response, impostor server or pump	Authentication	Fresh nonces from both sides in the signed context; certificates from the root	Table 58
Manufacturer denies issuing an update	Non-repudiation	Signature by a key only the HSM holds; archive of signed manifests with time-stamps; deterministic nonces	Section 7.4 B3

A3. Residual risks

- **Stolen current signing key:** accepted by any pump that has not yet seen the revocation (C14). Limited by HSM storage, role and rate limits, short key lifetime, and the epoch mechanism that closes the window at the next legitimate update.

- **Compromised root key:** catastrophic and not revocable in pumps already deployed (the root public key is in ROM); the only controls are the offline HSM, dual control and the ability to ship a new root in new hardware.
- **Backdated old key on a pump that has never seen a recent time:** an attacker with an expired signing key can set a backdated issued_at and pass until the pump has accepted a later signed time.
- **Build-pipeline compromise** before signing, implementation flaws (non-atomic commit, time-of-check/time-of-use between verification and installation, side channels in the ECDSA code), and denial of service remain outside what a signature can protect.

Student B – Experimental validation

The verification procedure of Figure 16 was implemented in firmware.py (own ECDSA P-256, compact certificates, deterministic nonces) and instrumented to count SHA-256 blocks and signature verifications; run_c7.py runs 15 correctness and attack cases against the final design and against a **baseline** that only publishes a bare SHA-256 digest beside the file. All data are synthetic.

Table 56. *Validation matrix: 15 of 15 cases behaved as designed (C14 is the documented residual risk)*

ID	Case	Baseline (bare digest)	Final design	Rejecting step	Reason	As designed	Cost (sig verifications / SHA-256 blocks)
C1	Valid update (version 11)	accepted	ACCEPTED	-	installed	Yes	2 / 313
C2	One bit of the image modified	rejected if digest page untouched	rejected	9 image	SHA-256 of the image differs from the signed manifest	Yes	2 / 313
C3	Attacker replaces image AND publishes	ACCEPTED (digest	rejected	7 manifest	manifest signature invalid	Yes	2 / 0

	a new digest / new manifest hash	replaced too)		signatur e			
C4	One bit of the signature modified	n/a (no signature)	rejected	7 manifest signatur e	manifest signature invalid	Yes	2 / 0
C5	Manifest edited (version 11 -> 99) without re-signing	n/a	rejected	7 manifest signatur e	manifest signature invalid	Yes	2 / 0
C6	Replay of an older GENUINE version (9) with its valid signature	ACCEPTED (old image + its own digest)	rejected	8 anti-rollback	version 9 <= installed 10	Yes	2 / 0
C7	Update signed with an EXPIRED certificate (pump has seen time T - 1 day)	ACCEPTED (no validity)	rejected	4 validity	certificate expired	Yes	1 / 0
C8	Certificate revoked by root-signed list carried with the update	ACCEPTED (no revocation)	rejected	5 revocati on	certificate serial 101 is revoked	Yes	2 / 0
C9	Manifest signed with attacker key but genuine certificate attached	ACCEPTED (anyone can publish a digest)	rejected	7 manifest signatur e	manifest signature invalid	Yes	2 / 0
C1 0	Attacker-made certificate (self-issued, not signed by the manufacturer root)	ACCEPTED	rejected	3 certificat e signatur e	signing certificate not signed by the manufacturer root	Yes	1 / 0

C11	Genuine update for a different device model	ACCEPTED (no model binding)	rejected	2 device model	image is for INFPUMP2	Yes	0 / 0
C12	Image truncated in transit	rejected if digest page untouched	rejected	9 image	size differs from manifest	Yes	2 / 0
C13	Key-epoch downgrade: update signed by retired epoch-1 key (pump already at epoch 2)	ACCEPTED	rejected	5 revocation	key epoch 1 below the pump's floor 2	Yes	1 / 0
C14	Stolen signing key; pump OFFLINE and has not received the revocation yet	ACCEPTED	ACCEPTED	-	installed	Yes	2 / 313
C15	Stolen signing key; pump has received the root-signed revocation list	ACCEPTED	rejected	5 revocation	certificate serial 104 is revoked	Yes	1 / 0

Reading the matrix. The baseline stops only accidental changes where the digest page is untouched; whenever the attacker controls the page or the distribution server (C3, C6, C9–C11, C13) it accepts the malicious or old image. The final design rejects every attack at the intended step, accepts the valid update (C1), and shows the two honest limits: a stolen key is accepted by a pump that has not received the revocation (C14) and refused by one that has (C15).

B2. Clock behaviour and offline operation

Table 57. *A wrong clock blocks legitimate updates or accepts expired keys under wall-clock validation; the time floor avoids both*

Pump clock	**Initial proposal A (wall-clock expiry): legitimate update**	**Final design (signed time floor): legitimate update**	**Wall-clock: package signed with an EXPIRED key**	**Final: package signed with an EXPIRED key**
correct clock	accepted	accepted	rejected	rejected
clock reset to 1970 (battery loss)	REJECTED (update blocked)	accepted	rejected	rejected
clock +10 years	REJECTED (update blocked)	accepted	rejected	rejected
clock set back 1 year	REJECTED (update blocked)	accepted	ACCEPTED (expired key works)	rejected

Offline pump. The update was verified with no network at all (0 messages), using 3 signature verifications; the root-signed revocation list carried in the package was processed (sequence 3, revoked serials [12345]) so the pump learned the revocation without contacting a server.

B3. Deterministic nonces and mutual authentication

1,000 different manifests were signed: 1,000 distinct r values (no repeated nonce), signing the same manifest twice gives the same signature (yes), all checked signatures verify (yes), and the nonce-reuse formula of Q3 applied to two of the signatures does **not** recover the key (yes).

Table 58. *Challenge–response mutual authentication between pump and hospital server*

Authentication test	**Result**	**Reason**	**Signature verifications**
Valid mutual authentication	accepted	mutual authentication succeeded	4

Replay of an old pump response in a new session	rejected	pump proof invalid (wrong key, wrong nonce or replay)	4
Pump uses a key that does not match its certificate	rejected	pump proof invalid (wrong key, wrong nonce or replay)	4
Revoked device certificate	rejected	device certificate revoked	3
Impostor server with a self-made certificate	rejected	server certificate invalid	1

B4. Verification cost and size of the added metadata

Table 59. *Cost of verification (own Python ECDSA; hardware is much faster) and metadata overhead*

Image size	**SHA-256 blocks**	**Signature verifications**	**Hash time (ms)**	**Signature time (ms)**	**Total verify (ms)**	**Added metadata (bytes)**	**Metadata / image**
65,536	1,025	2	0.0	57.1	57.1	282	0.430%
1,048,576	16,385	2	0.7	59.5	60.2	282	0.027%
16,777,216	262,145	2	14.4	59.2	73.6	282	0.002%

Table 60. *Metadata and properties of alternative designs*

Signature option	**Added metadata (bytes)**	**Signature verifications**	**Root key in pump**	**Comment**
ECDSA P-256 (chosen)	282	2	65 B root key in ROM	private key in HSM; deterministic nonces
RSA-2048 PKCS#1/PSS	860	2	259 B root key in ROM	cheap verification (17 multiplications); larger metadata
HMAC with shared fleet key	98	1	16-32 B secret in EVERY pump	no non-repudiation; one extracted key forges updates for the whole fleet

The image hash costs one SHA-256 block per 64 bytes and dominates for large images (74 ms total at 16 MB, of which about 59 ms are the two signature verifications and the rest the hash); the fixed metadata is 282 bytes (0.43% of a 64 KB image, 0.0017% of 16 MB). The two ECDSA verifications cost the same for every image size. RSA-2048 would need about three times the metadata (860 bytes) but verification of a 2,048-bit signature takes only 17 multiplications (Q3), so it is attractive for pumps without elliptic-curve acceleration.

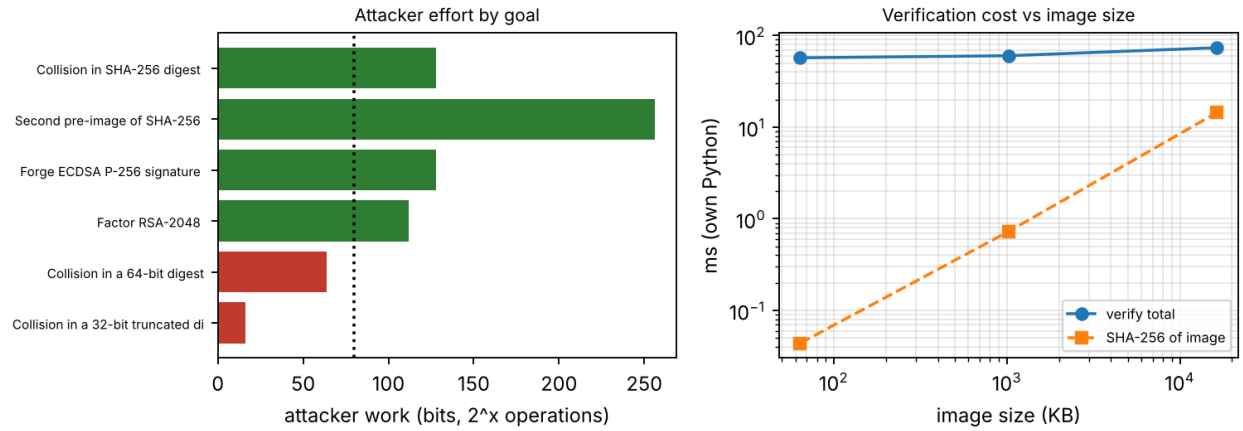


Figure 17. *Left: attacker work in bits for the goals of Table 54 (red: practical); right: verification cost versus image size*

7.5 Stage 5 – Cross-defence

Student A explains: what experimental evidence supports the group's design?

(1) Table 56: all 15 cases behave as designed (in C14 the designed outcome is acceptance, because it is the documented residual risk), and every attack that the baseline accepts is rejected. (2) Table 57: wall-clock validation is both blocked and fooled by a wrong clock, the signed time floor is not. (3) Offline test: verification with zero network messages, revocation learned from the package. (4) Deterministic nonces: 1,000 distinct r, reuse formula fails. (5) Table 58: replay, wrong key, revoked device and impostor server are rejected. (6) Table 59: two signature verifications and 282 bytes of metadata per update, independent of image size.

Student B explains: how does the security argument follow from the chosen mechanisms, and what does each contribute?

SHA-256 binds the image to the manifest (integrity, 2^128 collision work from the Q1 law); the **ECDSA signature** binds the manifest to the manufacturer key (authenticity, and non-repudiation because only the HSM holds the key; deterministic nonces remove the Q3 key-leak); the **certificate chain** (Q4) transfers trust from the ROM root to the signing key and limits its lifetime; the **version counter** adds freshness that signatures alone lack (Q5: authentication does not give freshness); **epochs and the root-signed revocation list** limit a stolen key; the **signed time floor** gives validity checks without a trustworthy clock; **challenge–response with certificates** gives mutual authentication without Kerberos infrastructure. Removing any one mechanism re-opens a case of Table 56.

7.6 Stage 6 – Joint evaluation

Table 61. *Which part of the design guarantees which property*

Property	Guaranteed by	Not guaranteed / depends on
Integrity	SHA-256 digest inside the signed manifest; streaming check before activation	Anything done to the image before signing; flaws between verification and flash write
Authentication (origin)	Root-certified ECDSA signing key; chain and revocation checks	Key custody at the manufacturer; pumps that have not received a revocation
Non-repudiation	Signature with a key only the HSM holds, archived signed manifests with trusted time-stamps, deterministic nonces	Compromise of the HSM or root; no external timestamp authority in the pump itself
Freshness / anti-rollback	Monotonic version counter and epoch floor	Pumps whose non-volatile counters can be reset by physical access
Mutual authentication with the server	Certificate-based challenge–response with fresh nonces	Server and device key custody

Table 62. *Costs introduced by the design*

Cost	Amount

Verification time	2 ECDSA verifications (about 57 ms in our Python code) plus one SHA-256 pass over the image (about 14 ms for 16 MB)
Storage in the pump	65-byte root public key (ROM), manifest + certificate + signatures about 282 bytes per update, four small counters/sets (version, epoch floor, time floor, revoked serials)
Key-management burden	Root ceremony and HSM, yearly signing-key rotation, revocation list distribution, device-key provisioning in the factory, archive of signed manifests

Under what conditions could the design still fail?

- **Key compromise:** a stolen signing key works until pumps learn the revocation (C14); a stolen root key cannot be repaired in deployed pumps; a stolen device key lets one pump be impersonated.
- **Flawed implementation:** a repeated or biased ECDSA nonce (Q3) – here prevented by deterministic nonces, but a bug in them would reintroduce the failure; non-atomic installation; verification and installation reading different copies of the image; timing and fault side channels.
- **Compromised root or build pipeline:** a malicious but correctly signed image is accepted.
- **Bad clock:** the design is robust to a wrong clock but a pump that has never seen a recent signed time can be fooled by a backdated expired key (Section 7.4 A3).

Under what conditions would a simpler or different mechanism be more appropriate?

Table 63. *When another mechanism would be better*

Alternative	Appropriate when	Why not for this scenario
HMAC with a shared key	Two mutually trusting parties (for example a hospital gateway and its pumps), no third-party evidence needed, very small microcontrollers. Use **per-device keys** derived from a master (key = HMAC(master, device id)) so one extracted key does not break the fleet.	Manufacturer and hospital are different organisations; the fleet key allows forgery for all pumps (simulated: succeeded) and gives no non-repudiation
Kerberos tickets	All devices inside one hospital realm, always online, synchronised clocks, central account	Pumps are often offline and have unreliable clocks; a manufacturer-

	control (staff single sign-on to pump consoles).	run KDC would serve every hospital; tickets give no third-party evidence
RSA-2048 signatures instead of ECDSA	Pumps without ECC acceleration where verification speed matters most (17 multiplications).	About three times the metadata and 256-byte keys/signatures in small flash
Bare digest beside the file	Detecting accidental corruption on a trusted link.	Fails every attack in Table 56 in which the attacker controls the page

Rubric check – Task 7 (10 marks)

Independent proposals A and B retained → 7.1. **Comparison record, threat model, security goals** → 7.2, 7.3. **Final protocol/message flow, key and certificate management, pseudocode** → 7.3, Figures 14, 15, 16. **Security argument, attack-effort estimates, residual risks** → 7.4 Student A, Table 54. **Implementation, baseline comparison, attack/correctness tests, measured costs** → 7.4 Student B, Tables 56, 57, 59. **Cross-defence and final evaluation/trade-offs** → 7.5, 7.6.

References

1. NIST, FIPS 180-4 Secure Hash Standard (SHA-1, SHA-2); NIST, FIPS 202 SHA-3 Standard; NIST SP 800-131A Rev. 2, Transitioning the Use of Cryptographic Algorithms and Key Lengths.
2. R. Merkle, “A Certified Digital Signature”, 1989; I. Damgård, “A Design Principle for Hash Functions”, 1989. M. Bellare, R. Canetti, H. Krawczyk, “Keying Hash Functions for Message Authentication”, 1996; RFC 2104, HMAC; FIPS 198-1, The Keyed-Hash Message Authentication Code.
3. X. Wang and H. Yu, “How to Break MD5 and Other Hash Functions”, 2005; M. Stevens et al., “The first collision for full SHA-1” (SHAttered), 2017; G. Leurent and T. Peyrin, “SHA-1 is a Shambles”, 2020; A. Sotirov et al., “MD5 considered harmful today: creating a rogue CA certificate”, 2008; G. Yuval, “How to Swindle Rabin”, 1979.
4. R. Rivest, A. Shamir, L. Adleman, 1978; T. ElGamal, “A public key cryptosystem and a signature scheme based on discrete logarithms”, 1985; C. P. Schnorr, “Efficient signature generation by smart cards”, 1991; NIST FIPS 186-5, Digital Signature Standard; RFC 6979, Deterministic Usage of DSA and ECDSA; RFC 8017 (PKCS #1).
5. ITU-T X.509 / RFC 5280, Internet X.509 Public Key Infrastructure Certificate and CRL Profile; RFC 6960, Online Certificate Status Protocol; RFC 6066 (OCSP stapling); RFC 9162, Certificate Transparency.
6. J. Kohl and C. Neuman, RFC 1510 / RFC 4120, The Kerberos Network Authentication Service (V5); B. C. Neuman and T. Ts’o, “Kerberos: An Authentication Service for Computer Networks”, 1994; RFC 4556 (PKINIT).
7. The Q7 design follows the ideas of secure software update frameworks (for example The Update Framework, TUF) and secure-boot practice: signed manifests, root of trust in ROM, anti-rollback counters and key rotation.

Appendices – Source Code, Execution Instructions and Test Output

All code is the student's own work unless stated. Files are reproduced in full. Modules ntcore.py, rsa.py, ecc.py, symmetric.py, dh.py, hybrid.py, flow.py and seq.py are reused from the previous assignments (number theory, RSA, P-256 arithmetic, Blowfish, diagram helpers) and are not reprinted; the CO3-specific files follow.

Appendix A – hashes.py – toy hashes, truncated SHA-256, toy Merkle–Damgard hash, HMAC, length extension (Q1, Q2)

hashes.py

```
"""hashes.py - toy hash functions, truncated SHA-256 and a toy Merkle-Damgard hash (CO3, Q1/Q2/Q6).
SHA-256 comes from hashlib and is used for comparison and as a building block only; the toy hashes are
student-written."""
import hashlib, struct
M32 = 0xFFFFFFFF

# ---------------------------------------------------------------- deliberately weak toy hashes (16-bit outputs)
def sum_hash(m: bytes) -> bytes:
    """Byte-sum modulo 2^16."""
    return (sum(m) & 0xFFFF).to_bytes(2, 'big')
def xorfold_hash(m: bytes) -> bytes:
    """XOR of all 16-bit words (zero-padded)."""
    if len(m) % 2: m += b'\x00'
    v = 0
    for i in range(0, len(m), 2): v ^= (m[i] << 8) | m[i + 1]
    return v.to_bytes(2, 'big')
def sha256(m: bytes) -> bytes: return hashlib.sha256(m).digest()

def trunc_sha256(m: bytes, k: int) -> int:
    """First k bits of SHA-256 as an integer."""
    return int.from_bytes(hashlib.sha256(m).digest(), 'big') >> (256 - k)

# ---------------------------------------------------------------- toy Merkle-Damgard hash: 64-bit chaining value, 8-
```

byte blocks

IV = 0x6A09E667BB67AE85

K = [0x3C6EF372, 0xA54FF53A, 0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19, 0x428A2F98, 0x71374491]

```
def _rotl(x, n): return ((x << n) | (x >> (32 - n))) & M32
BLOCK = 8
def compress(h: int, block: bytes) -> int:
    """f(h, m): eight ARX rounds on two 32-bit halves keyed by the message block, then feed-forward (h XOR result)."""
    a, b = h >> 32, h & M32
    m0, m1 = struct.unpack('>II', block)
    for r in range(8):
        a = _rotl((a + (b ^ (m0 if r % 2 == 0 else m1)) + K[r]) & M32, 5 + r)
        b = _rotl(b ^ a, 11 - r)
        a, b = b, a
    return (((a << 32) | b) ^ h) & 0xFFFFFFFFFFFFFFFF
def md_pad(msg_len: int) -> bytes:
    """0x80, zeros, then the 64-bit message length in bits (Merkle-Damgard strengthening)."""
    z = (-(msg_len + 1)) % BLOCK
    return b'\x80' + b'\x00' * z + (msg_len * 8).to_bytes(8, 'big')
def md_hash(m: bytes, state: int = IV, prior_len: int = 0, counter=None) -> bytes:
    """Hash m; `state`/`prior_len` allow resuming from a known chaining value (used by the length-extension attack)."""
    data = m + md_pad(prior_len + len(m))
    assert len(data) % BLOCK == 0
    for i in range(0, len(data), BLOCK):
        state = compress(state, data[i:i + BLOCK])
        if counter is not None: counter['compress'] += 1
    return state.to_bytes(8, 'big')

# ---------------------------------------------------------------- MACs
def naive_mac(key: bytes, msg: bytes, H=md_hash) -> bytes: return H(key + msg)
```

```
def hmac(key: bytes, msg: bytes, H, block: int) -> bytes:
    """HMAC(K, m) = H((K' xor opad) || H((K' xor ipad) || m)), K' = key padded (or hashed) to the block size."""
    if len(key) > block: key = H(key)
    key = key.ljust(block, b'\x00')
    ipad = bytes(b ^ 0x36 for b in key); opad = bytes(b ^ 0x5C for b in key)
    return H(opad + H(ipad + msg))
def hmac_toy(key, msg): return hmac(key, msg, md_hash, BLOCK)
def hmac_sha256(key, msg): return hmac(key, msg, sha256, 64)
def ct_equal(a: bytes, b: bytes) -> bool:
    """Constant-time comparison (no early exit)."""
    if len(a) != len(b): return False
    d = 0
    for x, y in zip(a, b): d |= x ^ y
    return d == 0


# ---------------------------------------------------------------- length extension against H(key || message)
def length_extend(msg: bytes, tag: bytes, suffix: bytes, key_len: int):
    """Needs: msg, tag, suffix and a guess of len(key). Does NOT need the key. Returns (forged_message, forged_tag)."""
    glue = md_pad(key_len + len(msg))
    state = int.from_bytes(tag, 'big')
    forged_tag = md_hash(suffix, state=state, prior_len=key_len + len(msg) + len(glue))
    return msg + glue + suffix, forged_tag
```

Appendix B – sigs.py – RSA, DSA, ElGamal, Schnorr signatures (Q3, Q6)

sigs.py

```
"""sigs.py - signature schemes written from scratch on toy parameters (Q3, Q6): RSA hash-then-sign, textbook RSA, DSA, ElGamal, Schnorr."""
import hashlib
from ntcore import modexp, modinv, gcd, gen_prime, is_prime_mr
import rsa as rsalib
```

```
def H_int(msg: bytes, mod: int) -> int:
    """SHA-256 digest of msg as an integer reduced mod `mod` (used as the 'hash' in every toy scheme)."""
    return int.from_bytes(hashlib.sha256(msg).digest(), 'big') % mod

# ---------------------------------------------------------------- RSA
def rsa_keygen(bits, rng, e=65537): return rsalib.generate(bits, rng, e)
def rsa_sign(msg, key): return modexp(H_int(msg, key['n']), key['d'], key['n'])            # s = H(m)^d mod n
def rsa_verify(msg, s, n, e):
    if not 0 < s < n: return False
    return modexp(s, e, n) == H_int(msg, n)                                    # s^e == H(m) mod n
def textbook_sign(m_int, key): return modexp(m_int, key['d'], key['n'])                    # NO hash
def textbook_verify(m_int, s, n, e): return 0 <= m_int < n and modexp(s, e, n) == m_int

# ---------------------------------------------------------------- DSA
def dsa_params(rng, pbits=512, qbits=128):
    q = gen_prime(qbits, rng)
    while True:
        k = rng.getrandbits(pbits - qbits) | (1 << (pbits - qbits - 1)); k -= k % 2
        p = k * q + 1
        if p.bit_length() == pbits and is_prime_mr(p, 20, rng): break
    while True:
        h = rng.randrange(2, p - 1); g = modexp(h, (p - 1) // q, p)
        if g != 1: return p, q, g
def dsa_keygen(par, rng): p, q, g = par; x = rng.randrange(1, q); return x, modexp(g, x, p)
def dsa_sign(par, x, msg, rng, k=None):
    p, q, g = par; z = H_int(msg, q)
    while True:
        kk = k if k is not None else rng.randrange(1, q)
        r = modexp(g, kk, p) % q
        if r == 0: continue
        s = modinv(kk, q) * (z + x * r) % q
```

```
        if s: return r, s
def dsa_verify(par, y, msg, sig, counter=None):
    p, q, g = par; r, s = sig
    if not (0 < r < q and 0 < s < q): return False
    w = modinv(s, q); z = H_int(msg, q); u1, u2 = z * w % q, r * w % q
    v = modexp(g, u1, p) * modexp(y, u2, p) % p % q
    return v == r
def dsa_recover(par, m1, s1, m2, s2):
    """Two signatures with the same k: k = (z1 - z2)/(s1 - s2), x = (s1 k - z1)/r."""
    p, q, g = par; (r1, ss1), (r2, ss2) = s1, s2
    assert r1 == r2
    z1, z2 = H_int(m1, q), H_int(m2, q)
    k = (z1 - z2) * modinv((ss1 - ss2) % q, q) % q
    x = (ss1 * k - z1) * modinv(r1, q) % q
    return k, x

# ---------------------------------------------------------------- ElGamal signatures (hash then sign) over Z_p*
def eg_params(rng, bits=64):
    while True:
        q = gen_prime(bits - 1, rng); p = 2 * q + 1
        if is_prime_mr(p, 25, rng): break
    while True:
        g = rng.randrange(2, p - 1)
        if modexp(g, 2, p) != 1 and modexp(g, q, p) != 1: return p, g          # primitive root test for safe prime
def eg_keygen(par, rng): p, g = par; x = rng.randrange(2, p - 2); return x, modexp(g, x, p)
def eg_sign(par, x, msg, rng, k=None, hashed=True, m_int=None):
    p, g = par; n = p - 1
    h = H_int(msg, n) if hashed else m_int % n
    while True:
        kk = k if k is not None else rng.randrange(2, n)
        if gcd(kk, n) != 1:
            if k is not None: raise ValueError('k not coprime to p-1')
```

```
            continue
        r = modexp(g, kk, p); s = modinv(kk, n) * (h - x * r) % n
        if s: return r, s
def eg_verify(par, y, msg, sig, hashed=True, m_int=None):
    p, g = par; r, s = sig
    if not (0 < r < p and 0 <= s < p - 1): return False
    h = H_int(msg, p - 1) if hashed else m_int % (p - 1)
    return modexp(g, h, p) == modexp(y, r, p) * modexp(r, s, p) % p
def solve_lin(a, b, n):
    """All solutions x of a*x = b (mod n)."""
    g = gcd(a % n, n)
    if b % g: return []
    n1 = n // g
    x0 = (b // g) * modinv((a // g) % n1, n1) % n1 if n1 > 1 else 0
    return [x0 + i * n1 for i in range(g)]
def eg_recover(par, y, m1, sig1, m2, sig2):
    """Two signatures with the same k: s1 - s2 = k^-1 (h1 - h2)  ->  k;  then x r = h1 - k s1 (mod p-1). Checked
against the public key y."""
    p, g = par; n = p - 1; (r, s1), (_, s2) = sig1, sig2
    h1, h2 = H_int(m1, n), H_int(m2, n)
    for k in solve_lin((s1 - s2) % n, (h1 - h2) % n, n):
        if modexp(g, k, p) != r: continue
        for x in solve_lin(r % n, (h1 - k * s1) % n, n):
            if modexp(g, x, p) == y: return k, x
    return None


# ---------------------------------------------------------------- Schnorr over the order-q subgroup of Z_p*
def schnorr_params(rng, pbits=256, qbits=64):
    q = gen_prime(qbits, rng)
    while True:
        k = rng.getrandbits(pbits - qbits) | (1 << (pbits - qbits - 1)); k -= k % 2
```

```
        p = k * q + 1
        if p.bit_length() == pbits and is_prime_mr(p, 20, rng): break
    while True:
        h = rng.randrange(2, p - 1); g = modexp(h, (p - 1) // q, p)
        if g != 1: return p, q, g
def schnorr_keygen(par, rng): p, q, g = par; x = rng.randrange(1, q); return x, modexp(g, x, p)
def schnorr_challenge(t, msg, q): return int.from_bytes(hashlib.sha256(str(t).encode() + b'|' + msg).digest(),
'big') % q
def schnorr_sign(par, x, msg, rng, r=None):
    p, q, g = par; r_ = r if r is not None else rng.randrange(1, q)
    t = modexp(g, r_, p); c = schnorr_challenge(t, msg, q); s = (r_ + c * x) % q          # commitment t,
challenge c, response s
    return t, s
def schnorr_verify(par, y, msg, sig):
    p, q, g = par; t, s = sig
    if not (1 < t < p and 0 <= s < q): return False
    c = schnorr_challenge(t, msg, q)
    return modexp(g, s, p) == t * modexp(y, c, p) % p                                  # g^s == t y^c
```

Appendix C – x509chain.py – X.509 chain-validation procedure (Q4); kerberos.py – Kerberos simulation (Q5)

x509chain.py

```
"""x509chain.py - student-written X.509 chain validation procedure (Q4). The `cryptography` library is used
only to BUILD and PARSE
certificates/CRLs/OCSP responses and to perform the raw signature operation; every decision of the
validation algorithm is made by this code."""
import datetime as dt
from cryptography import x509
from cryptography.x509.oid import NameOID, ExtendedKeyUsageOID
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import ec
from cryptography.x509 import ocsp
```

```
UTC = dt.timezone.utc
def name(cn, org='Synthetic Hospital Network'): return
x509.Name([x509.NameAttribute(NameOID.ORGANIZATION_NAME, org),
x509.NameAttribute(NameOID.COMMON_NAME, cn)])

def make_cert(subject_cn, subject_pub, issuer_cn, issuer_key, serial, not_before, not_after, ca=False,
pathlen=None, key_usage=None, san=None, eku=None, org='Synthetic Hospital Network'):
    b = (x509.CertificateBuilder().subject_name(name(subject_cn, org)).issuer_name(name(issuer_cn,
org)).public_key(subject_pub).serial_number(serial)
         .not_valid_before(not_before).not_valid_after(not_after))
    b = b.add_extension(x509.BasicConstraints(ca=ca, path_length=pathlen), critical=True)
    if key_usage is not None: b = b.add_extension(key_usage, critical=True)
    if san: b = b.add_extension(x509.SubjectAlternativeName([x509.DNSName(s) for s in san]),
critical=False)
    if eku: b = b.add_extension(x509.ExtendedKeyUsage(eku), critical=False)
    return b.sign(issuer_key, hashes.SHA256())
def ku(cert_sign=False, crl_sign=False, digital_signature=False):
    return x509.KeyUsage(digital_signature=digital_signature, content_commitment=False,
key_encipherment=False, data_encipherment=False, key_agreement=False,
                         key_cert_sign=cert_sign, crl_sign=crl_sign, encipher_only=False, decipher_only=False)
def cn_of(n): return n.get_attributes_for_oid(NameOID.COMMON_NAME)[0].value
def verify_sig(cert, issuer_cert):
    try:
        issuer_cert.public_key().verify(cert.signature, cert.tbs_certificate_bytes,
ec.ECDSA(cert.signature_hash_algorithm)); return True
    except Exception: return False
def der(c): return c.public_bytes(serialization.Encoding.DER)

def host_matches(pattern, host):
    pattern, host = pattern.lower(), host.lower()
    if pattern.startswith('*.'): return host.count('.') == pattern.count('.') and host.split('.', 1)[1] == pattern[2:]
```

```
    return pattern == host

class RevocationInfo:
    """What the client can obtain about revocation: a CRL per issuer and/or OCSP responses per (issuer, serial); None = unavailable."""
    def __init__(self, crls=None, ocsp_resp=None, available=True): self.crls, self.ocsp, self.available = crls or {}, ocsp_resp or {}, available

def validate_chain(chain, trust_store, hostname, now, revocation=None, check_revocation=True, mode='crl', fail_open=False):
    """chain = [leaf, intermediate..., root]; returns (accepted, failing_step, reason, trace).
    Order of checks follows the textbook algorithm: (1) issuer/subject matching, (2) signature on every link, (3) validity,
    (4) basicConstraints/pathLen, (5) keyUsage/EKU, (6) name match, (7) revocation, (8) trust anchor."""
    trace = []
    def fail(step, reason): trace.append((step, 'FAIL', reason)); return False, step, reason, trace
    def ok(step, note=''): trace.append((step, 'ok', note))
    n = len(chain)
    # 1 issuer-subject matching
    for i in range(n - 1):
        if chain[i].issuer != chain[i + 1].subject: return fail('1 issuer/subject match', f'issuer of "{cn_of(chain[i].subject)}" is "{cn_of(chain[i].issuer)}", next certificate is "{cn_of(chain[i + 1].subject)}"')
    ok('1 issuer/subject match', f'{n} certificates link up')
    # 2 signature on every link (the last certificate must be self-signed)
    for i in range(n):
        issuer = chain[i + 1] if i + 1 < n else chain[i]
        if not verify_sig(chain[i], issuer): return fail('2 signature', f'signature of "{cn_of(chain[i].subject)}" does not verify under "{cn_of(issuer.subject)}"')
    ok('2 signature', 'all signatures verify')
    # 3 validity period
    for c in chain:
        if now < c.not_valid_before_utc: return fail('3 validity period', f'"{cn_of(c.subject)}" not valid before
```

```
{c.not_valid_before_utc:%Y-%m-%d}')
    if now > c.not_valid_after_utc: return fail('3 validity period', f'"{cn_of(c.subject)}" expired on
{c.not_valid_after_utc:%Y-%m-%d}')
  ok('3 validity period', f'all valid on {now:%Y-%m-%d}')
  # 4 basic constraints / path length
  for i, c in enumerate(chain):
    bc = c.extensions.get_extension_for_class(x509.BasicConstraints).value
    if i > 0 and not bc.ca: return fail('4 basicConstraints', f'"{cn_of(c.subject)}" has CA=FALSE but issued
another certificate')
    if bc.ca and bc.path_length is not None:
      below = i - 1 if i > 0 else 0        # number of CA certificates between this one and the leaf
      if below > bc.path_length: return fail('4 basicConstraints', f'pathLen {bc.path_length} of
"{cn_of(c.subject)}" exceeded ({below} intermediate CA(s) below)')
  ok('4 basicConstraints', 'CA flags and path length respected')
  # 5 key usage / EKU
  leaf = chain[0]
  for c in chain[1:]:
    k = c.extensions.get_extension_for_class(x509.KeyUsage).value
    if not k.key_cert_sign: return fail('5 keyUsage', f'"{cn_of(c.subject)}" lacks keyCertSign')
  try:
    lk = leaf.extensions.get_extension_for_class(x509.KeyUsage).value
    if not lk.digital_signature: return fail('5 keyUsage', 'leaf lacks digitalSignature')
  except x509.ExtensionNotFound: return fail('5 keyUsage', 'leaf has no keyUsage')
  try:
    ek = leaf.extensions.get_extension_for_class(x509.ExtendedKeyUsage).value
    if ExtendedKeyUsageOID.SERVER_AUTH not in ek: return fail('5 keyUsage', 'leaf EKU does not
include serverAuth')
  except x509.ExtensionNotFound: pass
  ok('5 keyUsage', 'issuers may sign certificates; leaf may authenticate a server')
  # 6 name matching
  try: sans = [d for d in
leaf.extensions.get_extension_for_class(x509.SubjectAlternativeName).value.get_values_for_type(x509.DN
```

```
SName)]
    except x509.ExtensionNotFound: sans = [cn_of(leaf.subject)]
    if not any(host_matches(s, hostname) for s in sans): return fail('6 name match', f'"{hostname}" not in subjectAltName {sans}')
    ok('6 name match', f'"{hostname}" matches {sans}')
    # 7 revocation
    if check_revocation:
        for i in range(n - 1):
            c, issuer = chain[i], chain[i + 1]
            if revocation is None or not revocation.available:
                if fail_open: trace.append(('7 revocation', 'WARN', 'revocation source unreachable: FAIL-OPEN, certificate treated as good')); continue
                return fail('7 revocation', 'revocation status unavailable and policy is FAIL-CLOSED')
            if mode == 'crl':
                crl = revocation.crls.get(cn_of(issuer.subject))
                if crl is None:
                    if fail_open: trace.append(('7 revocation', 'WARN', 'no CRL: fail-open')); continue
                    return fail('7 revocation', 'no CRL available (fail-closed)')
                if not crl.is_signature_valid(issuer.public_key()): return fail('7 revocation', 'CRL signature invalid')
                if now > crl.next_update_utc: return fail('7 revocation', 'CRL is stale (past nextUpdate)')
                r = crl.get_revoked_certificate_by_serial_number(c.serial_number)
                if r is not None: return fail('7 revocation', f'serial {c.serial_number:x} is listed in the CRL of "{cn_of(issuer.subject)}"')
            else:
                resp = revocation.ocsp.get((cn_of(issuer.subject), c.serial_number))
                if resp is None:
                    if fail_open: trace.append(('7 revocation', 'WARN', 'no OCSP response: fail-open')); continue
                    return fail('7 revocation', 'no OCSP response (fail-closed)')
                if resp.certificate_status == ocsp.OCSPCertStatus.REVOKED: return fail('7 revocation', 'OCSP responder reports REVOKED')
                if resp.next_update_utc is not None and now > resp.next_update_utc: return fail('7 revocation', 'OCSP response stale')
```

```
        ok('7 revocation', f'{mode.upper()} checked: not revoked')
    else: trace.append(('7 revocation', 'SKIPPED', 'revocation checking disabled'))
    # 8 trust anchor
    root = chain[-1]
    if not any(der(root) == der(t) for t in trust_store): return fail('8 trust anchor', f'"{cn_of(root.subject)}" is not in the trust store')
    ok('8 trust anchor', f'anchored at trusted root "{cn_of(root.subject)}"')
    return True, None, 'accepted', trace
```

kerberos.py

```
"""kerberos.py - simplified Kerberos v5-style simulation (Q5): Authentication Server (AS), Ticket-Granting Server (TGS), service, client.
Tickets, authenticators and replies are sealed with student-written Blowfish-CTR + HMAC-SHA256 (encrypt-then-MAC). Synthetic principals only."""
import json, hashlib, hmac as pyhmac, random
from symmetric import Blowfish, ctr
from hashes import hmac_sha256, ct_equal

def kdf(password: str, salt: str, iters: int) -> bytes: return hashlib.pbkdf2_hmac('sha256', password.encode(), salt.encode(), iters, 32)
def seal(key: bytes, obj: dict, rng) -> bytes:
    nonce = rng.randbytes(4); pt = json.dumps(obj, sort_keys=True).encode()
    ct = ctr(Blowfish(key[:16]), nonce, pt)
    return nonce + ct + hmac_sha256(key[16:] + b'k', nonce + ct)
def unseal(key: bytes, blob: bytes) -> dict:
    nonce, ct, tag = blob[:4], blob[4:-32], blob[-32:]
    if not ct_equal(tag, hmac_sha256(key[16:] + b'k', nonce + ct)): raise ValueError('integrity check failed (wrong key or altered message)')
    return json.loads(ctr(Blowfish(key[:16]), nonce, ct))

class KerberosError(Exception): pass
SKEW = 300; TGT_LIFE = 36000; ST_LIFE = 3600
```

```
class KDC:
    def __init__(self, realm, rng, iters=1000, preauth=False):
        self.realm, self.rng, self.iters, self.preauth = realm, rng, iters, preauth
        self.users, self.services, self.online = {}, {}, True
        self.k_tgs = rng.randbytes(32); self.log = []
    def add_user(self, user, password): self.users[user] = kdf(password, self.realm + user, self.iters)
    def add_service(self, name): self.services[name] = self.rng.randbytes(32); return self.services[name]
    def _up(self):
        if not self.online: raise KerberosError('KDC unreachable')
    def as_req(self, user, n1, now, preauth_blob=None):
        """AS exchange. Reply: {K_c,tgs, n1, tgs name, endtime} under the user's long-term key, plus the TGT
under the TGS key."""
        self._up()
        if user not in self.users: raise KerberosError('unknown principal')
        if self.preauth:
            try: pa = unseal(self.users[user], preauth_blob or b'x' * 40)
            except Exception: raise KerberosError('pre-authentication failed')
            if abs(pa['ts'] - now) > SKEW: raise KerberosError('pre-authentication timestamp outside skew')
        k_ctgs = self.rng.randbytes(32); end = now + TGT_LIFE
        tgt = seal(self.k_tgs, {'c': user, 'k': k_ctgs.hex(), 'start': now, 'end': end}, self.rng)
        rep = seal(self.users[user], {'k': k_ctgs.hex(), 'n': n1, 'tgs': 'krbtgt/' + self.realm, 'end': end}, self.rng)
        return rep, tgt
    def tgs_req(self, tgt, authenticator, service, n2, now, replay_cache=None):
        self._up()
        try: t = unseal(self.k_tgs, tgt)
        except ValueError: raise KerberosError('TGT rejected: not issued by this KDC')
        if now > t['end']: raise KerberosError('TGT expired')
        k_ctgs = bytes.fromhex(t['k'])
        try: a = unseal(k_ctgs, authenticator)
        except ValueError: raise KerberosError('authenticator rejected: wrong session key')
```

```
        if a['c'] != t['c']: raise KerberosError('authenticator names a different client than the TGT')
        if abs(a['ts'] - now) > SKEW: raise KerberosError('clock skew too great')
        if service not in self.services: raise KerberosError('unknown service')
        k_cs = self.rng.randbytes(32); end = min(now + ST_LIFE, t['end'])
        st = seal(self.services[service], {'c': t['c'], 'k': k_cs.hex(), 'start': now, 'end': end}, self.rng)
        rep = seal(k_ctgs, {'k': k_cs.hex(), 'n': n2, 'service': service, 'end': end}, self.rng)
        return rep, st

class Service:
    def __init__(self, name, key, replay_protection=True): self.name, self.key, self.replay_protection,
self.cache, self.accepted = name, key, replay_protection, {}, 0
    def ap_req(self, ticket, authenticator, now):
        try: t = unseal(self.key, ticket)
        except ValueError: raise KerberosError('ticket rejected: not issued by the KDC for this service')
        if now > t['end']: raise KerberosError('ticket expired')
        if now < t['start'] - SKEW: raise KerberosError('ticket not yet valid')
        k_cs = bytes.fromhex(t['k'])
        try: a = unseal(k_cs, authenticator)
        except ValueError: raise KerberosError('authenticator rejected: cannot be built without the session key')
        if a['c'] != t['c']: raise KerberosError('authenticator and ticket name different clients')
        if abs(a['ts'] - now) > SKEW: raise KerberosError('clock skew too great')
        if self.replay_protection:
            key = (a['c'], a['ts'], a['seq'])
            self.cache = {k: v for k, v in self.cache.items() if now - v <= SKEW}       # entries older than the
skew window can be dropped
            if key in self.cache: raise KerberosError('replay detected: authenticator already seen')
            self.cache[key] = now
        self.accepted += 1
        rep = seal(k_cs, {'ts': a['ts'] + 1, 'service': self.name}, random.Random(a['ts']))     # mutual
authentication proof (ts + 1)
        return t['c'], rep
```

```
class Client:
    def __init__(self, user, rng, realm, iters=1000): self.user, self.rng, self.realm, self.iters = user, rng, realm, iters; self.tgt = None; self.tickets = {}; self.seq = 0
    def login(self, kdc, password, now, preauth=False):
        key = kdf(password, self.realm + self.user, self.iters); n1 = self.rng.getrandbits(32)
        pa = seal(key, {'ts': now}, self.rng) if preauth else None
        rep, tgt = kdc.as_req(self.user, n1, now, pa)
        try: d = unseal(key, rep)                    # only the right password decrypts the reply
        except ValueError: raise KerberosError('wrong password: AS reply cannot be decrypted')
        assert d['n'] == n1
        self.k_tgs = bytes.fromhex(d['k']); self.tgt = tgt; self.tgt_end = d['end']; del key                    # long-term key used once and discarded
        return d
    def get_ticket(self, kdc, service, now):
        auth = seal(self.k_tgs, {'c': self.user, 'ts': now}, self.rng); n2 = self.rng.getrandbits(32)
        rep, st = kdc.tgs_req(self.tgt, auth, service, n2, now)
        d = unseal(self.k_tgs, rep); assert d['n'] == n2
        self.tickets[service] = (st, bytes.fromhex(d['k']), d['end']); return d
    def make_ap_req(self, service, now):
        st, k_cs, end = self.tickets[service]; self.seq += 1
        return st, seal(k_cs, {'c': self.user, 'ts': now, 'seq': self.seq}, self.rng), now
    def check_ap_rep(self, service, rep, ts):
        st, k_cs, end = self.tickets[service]
        try: d = unseal(k_cs, rep)
        except ValueError: return False
        return d['ts'] == ts + 1 and d['service'] == service
```

Appendix D – firmware.py – firmware-update scheme (Q7)

firmware.py

```
"""firmware.py - Q7: secure over-the-air firmware update scheme for infusion pumps (toy, synthetic data).
Mechanisms: SHA-256 image digest; ECDSA P-256 signatures (student-written ecc.py) with deterministic nonces; compact two-level certificates
```

```
(offline root -> release-signing key); monotonic version counter (anti-rollback); key epochs + root-signed
revocation list; validity checked against a
SIGNED time floor rather than the pump's unreliable clock; challenge-response mutual authentication with
the hospital server."""
import struct, hashlib
from collections import Counter
import ecc
from ecc import P256
from hashes import hmac_sha256, sha256

N = P256.n
def pt(Q): return b'\x04' + Q[0].to_bytes(32, 'big') + Q[1].to_bytes(32, 'big')
def unpt(b): return (int.from_bytes(b[1:33], 'big'), int.from_bytes(b[33:], 'big'))
def sig_bytes(rs): return rs[0].to_bytes(32, 'big') + rs[1].to_bytes(32, 'big')
def sig_from(b): return int.from_bytes(b[:32], 'big'), int.from_bytes(b[32:], 'big')

def sign_det(d, msg):
    """ECDSA with a deterministic nonce k = HMAC-SHA256(d, H(msg) || counter) mod (n-1) + 1 (the idea
of RFC 6979): no random generator is needed in the signer."""
    h = sha256(msg); i = 0
    while True:
        k = int.from_bytes(hmac_sha256(d.to_bytes(32, 'big'), h + bytes([i])), 'big') % (N - 1) + 1
        r, s = ecc.ecdsa_sign(P256, d, msg, None, k=k)
        return sig_bytes((r, s))
def verify(Q, msg, sig, cnt=None):
    if cnt is not None: cnt['ecdsa_verify'] += 1
    return ecc.ecdsa_verify(P256, Q, msg, sig_from(sig))

# ---------------------------------------------------------------- data formats (compact binary)
def pack_manifest(model, version, min_version, issued_at, image_size, key_epoch, cert_serial,
image_hash):
    return struct.pack('>8sIIQIHI32s', model.ljust(8, b'\0'), version, min_version, issued_at, image_size,
```

```
key_epoch, cert_serial, image_hash)
def unpack_manifest(b):
    model, version, min_version, issued_at, size, epoch, serial, h = struct.unpack('>8sIIQIHI32s', b)
    return dict(model=model.rstrip(b'\0'), version=version, min_version=min_version, issued_at=issued_at,
image_size=size, key_epoch=epoch, cert_serial=serial, image_hash=h)
MANIFEST_LEN = struct.calcsize('>8sIIQIHI32s')
def pack_cert(serial, epoch, role, pub, nb, na): return struct.pack('>IHB65sQQ', serial, epoch, role, pt(pub) if
isinstance(pub, tuple) else pub, nb, na)
def unpack_cert(b):
    serial, epoch, role, pub, nb, na = struct.unpack('>IHB65sQQ', b); return dict(serial=serial, epoch=epoch,
role=role, pub=unpt(pub), nb=nb, na=na)
CERT_LEN = struct.calcsize('>IHB65sQQ')
ROLE_SIGNER, ROLE_DEVICE, ROLE_SERVER = 1, 2, 3
def pack_revocation(seq, issued_at, serials): return struct.pack('>IQH', seq, issued_at, len(serials)) +
b''.join(struct.pack('>I', s) for s in serials)
def unpack_revocation(b):
    seq, issued, n = struct.unpack('>IQH', b[:14]); return seq, issued, [struct.unpack('>I', b[14 + 4 * i:18 + 4 *
i])[0] for i in range(n)]


class Manufacturer:
    def __init__(self, rng):
        self.root_d, self.root_Q = ecc.ecdh_keypair(P256, rng); self.rng = rng; self.next_serial = 100
    def issue(self, role, pub, epoch, nb, na):
        self.next_serial += 1; c = pack_cert(self.next_serial, epoch, role, pub, nb, na)
        return c, sign_det(self.root_d, c)
    def revocation_list(self, seq, issued_at, serials):
        b = pack_revocation(seq, issued_at, serials); return b, sign_det(self.root_d, b)
    def make_signing_identity(self, epoch, nb, na):
        d, Q = ecc.ecdh_keypair(P256, self.rng); cert, csig = self.issue(ROLE_SIGNER, Q, epoch, nb, na);
return {'d': d, 'Q': Q, 'cert': cert, 'cert_sig': csig, 'serial': unpack_cert(cert)['serial'], 'epoch': epoch}


def build_release(image, version, signer, model=b'INFPUMP1', issued_at=0, min_version=0,
```

```
revocation=None):
    man = pack_manifest(model, version, min_version, issued_at, len(image), signer['epoch'], signer['serial'],
sha256(image))
    return {'manifest': man, 'sig': sign_det(signer['d'], man), 'cert': signer['cert'], 'cert_sig': signer['cert_sig'],
'image': image, 'revocation': revocation}
def metadata_bytes(pkg):
    n = len(pkg['manifest']) + len(pkg['sig']) + len(pkg['cert']) + len(pkg['cert_sig'])
    if pkg.get('revocation'): n += len(pkg['revocation'][0]) + len(pkg['revocation'][1])
    return n


class Pump:
    """State kept in non-volatile memory: root public key (ROM), installed version, highest key epoch, signed
time floor, revoked serials, revocation-list sequence."""
    def __init__(self, root_Q, model=b'INFPUMP1', version=10, time_floor=0, epoch=1):
        self.root_Q, self.model, self.version, self.time_floor, self.epoch_floor = root_Q, model, version,
time_floor, epoch
        self.revoked, self.rl_seq, self.log = set(), 0, []
    def verify_update(self, pkg, clock=None, use_floor=True, check_revocation=True, commit=True):
        cnt = Counter(); fail = lambda step, why: (False, step, why, cnt)
        man_b, image = pkg['manifest'], pkg['image']
        if len(man_b) != MANIFEST_LEN or len(pkg['cert']) != CERT_LEN or len(pkg['sig']) != 64 or
len(pkg['cert_sig']) != 64: return fail('1 format', 'malformed metadata')
        m = unpack_manifest(man_b); c = unpack_cert(pkg['cert'])
        if m['model'] != self.model: return fail('2 device model', f"image is for {m['model'].decode()}")
        # optional root-signed revocation list carried with the update (also usable while offline)
        rl = pkg.get('revocation')
        if rl and check_revocation:
            if verify(self.root_Q, rl[0], rl[1], cnt):
                seq, issued, serials = unpack_revocation(rl[0])
                if seq > self.rl_seq and commit: self.rl_seq = seq; self.revoked |= set(serials)
                elif seq > self.rl_seq: pass
        # certificate: signed by the root key pinned in the pump
```

```
    if not verify(self.root_Q, pkg['cert'], pkg['cert_sig'], cnt): return fail('3 certificate signature', 'signing certificate not signed by the manufacturer root')
    if c['role'] != ROLE_SIGNER: return fail('3 certificate role', 'certificate is not a release-signing certificate')
    ref = max(self.time_floor, m['issued_at']) if use_floor else clock
    if ref is None: return fail('4 validity', 'no time reference')
    if ref < c['nb']: return fail('4 validity', 'certificate not yet valid')
    if ref > c['na']: return fail('4 validity', 'certificate expired')
    if check_revocation and (c['serial'] in self.revoked): return fail('5 revocation', f"certificate serial {c['serial']} is revoked")
    if c['epoch'] < self.epoch_floor: return fail('5 revocation', f"key epoch {c['epoch']} below the pump's floor {self.epoch_floor}")
    if m['key_epoch'] != c['epoch'] or m['cert_serial'] != c['serial']: return fail('6 manifest binding', 'manifest does not match the certificate')
    if not verify(c['pub'], man_b, pkg['sig'], cnt): return fail('7 manifest signature', 'manifest signature invalid')
    if m['version'] <= self.version or m['version'] < m['min_version']: return fail('8 anti-rollback', f"version {m['version']} <= installed {self.version}")
    if len(image) != m['image_size']: return fail('9 image', 'size differs from manifest')
    cnt['sha256_blocks'] = (len(image) + 9 + 63) // 64
    if sha256(image) != m['image_hash']: return fail('9 image', 'SHA-256 of the image differs from the signed manifest')
    if commit:                                  # atomic commit only after every check passed
        self.version = m['version']; self.time_floor = max(self.time_floor, m['issued_at']); self.epoch_floor = max(self.epoch_floor, c['epoch'])
    return True, None, 'installed', cnt

# ---------------------------------------------------------------- baseline: bare SHA-256 published beside the file
def baseline_publish(image): return {'image': image, 'digest': sha256(image)}
def baseline_verify(pkg): return sha256(pkg['image']) == pkg['digest']

# ---------------------------------------------------------------- mutual authentication pump <-> hospital server
```

```
def auth_context(nonce_s, nonce_p, pump_id, server_id, who): return sha256(who + nonce_s + nonce_p + pump_id + server_id)
def authenticate(pump_ident, server_ident, root_Q, rng, revoked=(), replay_response=None, server_id=b'HOSPITAL-SRV', pump_id=b'PUMP-0001', cnt=None):
    """pump_ident/server_ident: dict(d, cert, cert_sig). Returns (ok, reason)."""
    cnt = cnt if cnt is not None else Counter()
    ns, np_ = rng.randbytes(16), rng.randbytes(16)
    # server -> pump: nonce, certificate; pump verifies the server first
    sc = unpack_cert(server_ident['cert'])
    if not verify(root_Q, server_ident['cert'], server_ident['cert_sig'], cnt) or sc['role'] != ROLE_SERVER: return False, 'server certificate invalid'
    s_sig = sign_det(server_ident['d'], auth_context(ns, np_, pump_id, server_id, b'S'))
    if not verify(sc['pub'], auth_context(ns, np_, pump_id, server_id, b'S'), s_sig, cnt): return False, 'server proof invalid'
    # pump -> server: certificate and signature over the fresh nonces
    pc = unpack_cert(pump_ident['cert'])
    if not verify(root_Q, pump_ident['cert'], pump_ident['cert_sig'], cnt) or pc['role'] != ROLE_DEVICE: return False, 'device certificate invalid'
    if pc['serial'] in revoked: return False, 'device certificate revoked'
    p_sig = sign_det(pump_ident['d'], auth_context(ns, np_, pump_id, server_id, b'P')) if replay_response is None else replay_response
    if not verify(pc['pub'], auth_context(ns, np_, pump_id, server_id, b'P'), p_sig, cnt): return False, 'pump proof invalid (wrong key, wrong nonce or replay)'
    return True, 'mutual authentication succeeded'
```

Appendix E – Experiment drivers run_c1.py – run_c7.py and recheck_c1.py

run_c1.py

```
"""run_c1.py - Q1: avalanche experiment, toy-hash attacks, pigeonhole, birthday-bound collision search."""
import json, random, time, math, statistics as st
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from hashes import *
```

```
rng = random.Random(2031)
R = {}
def bits(b): return bin(int.from_bytes(b, 'big')).count('1')
def xor_bits(a, b): return bin(int.from_bytes(a, 'big') ^ int.from_bytes(b, 'big')).count('1')
HASHES = {'byte-sum (16 bit)': (sum_hash, 16), 'XOR-fold (16 bit)': (xorfold_hash, 16), 'toy Merkle-
Damgard (64 bit)': (md_hash, 64), 'SHA-256 (256 bit)': (sha256, 256)}
SIZES = [8, 64, 512, 4096]
TRIALS = {8: 2000, 64: 2000, 512: 1500, 4096: 600}
# ---- A1: avalanche: flip one random input bit, count changed output bits
av = []; dist = {}
for name, (H, n) in HASHES.items():
    for sz in SIZES:
        T = TRIALS[sz]; counts = []
        for _ in range(T):
            m = bytes(rng.randrange(256) for _ in range(sz)); i = rng.randrange(sz * 8)
            m2 = bytearray(m); m2[i // 8] ^= 1 << (i % 8)
            counts.append(xor_bits(H(m), H(bytes(m2))))
        mean = st.mean(counts); sd = st.pstdev(counts)
        av.append([name, n, sz, T, mean, sd, n / 2, math.sqrt(n) / 2, (mean - n / 2) / (math.sqrt(n) / 2 /
math.sqrt(T)), min(counts), max(counts)])
        if sz == 512: dist[name] = counts
R['avalanche'] = av
# ---- A2: structured inputs: similar messages (1 character apart), long vs short
pairs = [(b'PAY 100 TO ALICE', b'PAY 101 TO ALICE'), (b'PAY 100 TO ALICE', b'PAY 100 TO
ALICF'), (b'DOSE=5MG', b'DOSE=6MG'), (b'A' * 1000, b'A' * 999 + b'B'), (b'', b'\x00')]
sim = []
for a, b in pairs:
    sim.append([(a[:16] + (b'...' if len(a) > 16 else b'')).decode(), len(a)] + [xor_bits(H(a), H(b)) for H, n in
HASHES.values()])
R['similar'] = sim
# ---- A3: second-preimage attacks on the toy hashes (constructed, not searched)
m = b'TRANSFER 100 TO ALICE'
```

```
m2 = m[:9] + m[10:11] + m[9:10] + m[11:]                    # swap two bytes: same byte sum
x = bytearray(m + b'\x00'); x[0:2], x[2:4] = x[2:4], x[0:2]; m3 = bytes(x)   # swap two 16-bit words: same XOR fold
pre = []
pre.append(['byte-sum', m.decode(), m2.decode(), sum_hash(m) == sum_hash(m2) and m != m2, 1])
pre.append(['XOR-fold', m.decode(), m3[:len(m)].decode(errors='replace'), xorfold_hash(m + b'\x00') == xorfold_hash(m3) and m + b'\x00' != m3, 1])
# preimage construction for the byte sum: any message with the right sum
target = sum_hash(b'ADMIN: GRANT ACCESS'); t = int.from_bytes(target, 'big')
forged = b'x' * (t // 120) + bytes([t % 120 + 0]) if t % 120 else b'x' * (t // 120)
forged = bytearray(); rem = t
while rem > 0: c = min(rem, 120); forged.append(c); rem -= c
R['sum_preimage'] = {'target': t, 'constructed_len': len(forged), 'ok': sum_hash(bytes(forged)) == target, 'work': 1}
R['toy_second_preimage'] = pre
# ---- A4: pigeonhole: 8-bit digest: any 257 messages must collide
firsts = []
for _ in range(2000):
    seen = {}; base = rng.getrandbits(64)
    for i in range(1, 300):
        d = trunc_sha256(base.to_bytes(8, 'big') + i.to_bytes(4, 'big'), 8)
        if d in seen: firsts.append(i); break
        seen[d] = i
R['pigeonhole8'] = {'runs': 2000, 'mean_first_collision_at': st.mean(firsts), 'max': max(firsts), 'bound': 257, 'theory': math.sqrt(math.pi / 2 * 256)}
# ---- B1: collision search for truncated SHA-256
RUNS = {16: 200, 20: 120, 24: 80, 28: 40, 32: 24, 36: 6}
def find_collision(k, rng):
    seen = {}; base = rng.getrandbits(64).to_bytes(8, 'big'); i = 0
    while True:
        i += 1
        d = trunc_sha256(base + i.to_bytes(8, 'big'), k)
```

```
    if d in seen: return i, base + seen[d].to_bytes(8, 'big'), base + i.to_bytes(8, 'big')
    seen[d] = i
col = []; raw = {}
for k, n in RUNS.items():
  t0 = time.perf_counter(); res = [find_collision(k, rng)[0] for _ in range(n)]; dt = time.perf_counter() - t0
  raw[k] = res
  theory = math.sqrt(math.pi / 2) * 2 ** (k / 2)
  col.append([k, n, st.mean(res), st.median(res), min(res), max(res), theory, st.mean(res) / theory, 2 ** k, 2 ** k / st.mean(res), dt])
R['collision'] = col
# distribution check for k = 24: fraction of runs finished by 2^(k/2) hashes (theory 1 - exp(-1/2))
r24 = raw[24]; R['frac_by_sqrt'] = {'k24_observed': sum(1 for x in r24 if x <= 2 ** 12) / len(r24), 'theory': 1 - math.exp(-0.5)}
# ---- B2: finding a collision with a SPECIFIC message (second preimage of a fixed digest)
sp = []
for k, n in {8: 400, 10: 200, 12: 100, 14: 40, 16: 20}.items():
  res = []
  for _ in range(n):
    target = trunc_sha256(b'FIXED-MESSAGE', k); base = rng.getrandbits(64).to_bytes(8, 'big'); i = 0
    while True:
      i += 1
      if trunc_sha256(base + i.to_bytes(8, 'big'), k) == target: break
    res.append(i)
  sp.append([k, n, st.mean(res), 2 ** k, st.mean(res) / 2 ** k, st.mean(raw[k]) if k in raw else None])
R['specific'] = sp
# ---- B3: real digests
R['digests'] = [['MD5', 128, '2^64', 'practical collisions since 2004; rogue CA certificate 2008; Flame 2012'], ['SHA-1', 160, '2^80 (generic)', 'first collision 2017 (about 2^63 work); chosen-prefix 2019-2020'], ['SHA-256', 256, '2^128', 'no practical attack'], ['SHA-512', 512, '2^256', 'no practical attack']]
json.dump(R, open('../out/c1.json', 'w'), indent=1, default=str)
print(json.dumps({k: R[k] for k in ['similar', 'toy_second_preimage', 'sum_preimage', 'pigeonhole8', 'collision', 'frac_by_sqrt', 'specific']}, default=str)[:3500])
```

```
for r in av: print([round(x, 3) if isinstance(x, float) else x for x in r])
# ---- figures
fig, ax = plt.subplots(1, 4, figsize=(10, 2.6))
for a, (name, (H, n)) in zip(ax, HASHES.items()):
    c = dist[name]; lo, hi = min(c), max(c)
    a.hist(c, bins=range(lo, hi + 2), color='#1f4e79', alpha=.85, density=True)
    a.axvline(n / 2, c='r', ls='--', lw=1); a.set_title(name.replace(' (', '\n('), fontsize=7.5); a.set_xlabel('changed output bits', fontsize=7); a.tick_params(labelsize=6)
ax[0].set_ylabel('fraction of trials', fontsize=7)
plt.tight_layout(); plt.savefig('../fig/c1_avalanche.png', dpi=170); plt.close()
fig, ax = plt.subplots(1, 2, figsize=(9, 3.2))
ks = [r[0] for r in col]
ax[0].semilogy(ks, [r[2] for r in col], 'o-', label='measured mean hashes to first collision'); ax[0].semilogy(ks, [r[6] for r in col], 's--', label='theory 1.2533 x 2^(k/2)'); ax[0].semilogy(ks, [r[8] for r in col], '^:', label='2^k (what a naive guess would give)')
ax[0].set_xlabel('truncated digest length k (bits)'); ax[0].set_ylabel('hashes computed');
ax[0].legend(fontsize=7); ax[0].grid(alpha=.3, which='both'); ax[0].set_title('Birthday bound: collision effort', fontsize=9)
xs = sorted(raw[24]); ys = [(i + 1) / len(xs) for i in range(len(xs))]
ax[1].step(xs, ys, label='observed (80 runs, k = 24)'); n = [x for x in range(1, max(xs))]
ax[1].plot(n, [1 - math.exp(-i * i / 2 ** 25) for i in n], 'r--', label='1 - exp(-n^2 / 2^(k+1))');
ax[1].set_xlabel('hashes computed n'); ax[1].set_ylabel('P(collision found by n)'); ax[1].legend(fontsize=7);
ax[1].grid(alpha=.3); ax[1].set_title('Cumulative distribution', fontsize=9)
plt.tight_layout(); plt.savefig('../fig/c1_birthday.png', dpi=170); plt.close()
```

recheck_c1.py

```
"""recheck_c1.py - larger repeat of the fixed-target (second pre-image) search for k = 14 and 16 (checks an outlying cell of run_c1.py)."""
import random, statistics as st, json
from hashes import trunc_sha256
rng = random.Random(99); out = []
for k, n in [(14, 300), (16, 100)]:
```

```
    res = []
    for _ in range(n):
        target = trunc_sha256(b'FIXED-MESSAGE', k); base = rng.getrandbits(64).to_bytes(8, 'big'); i = 0
        while True:
            i += 1
            if trunc_sha256(base + i.to_bytes(8, 'big'), k) == target: break
        res.append(i)
    out.append([k, n, st.mean(res), 2 ** k, st.mean(res) / 2 ** k]); print(out[-1])
q = json.load(open('../out/c1.json')); q['specific_recheck'] = out; json.dump(q, open('../out/c1.json', 'w'),
indent=1)
```

run_c2.py

```
"""run_c2.py - Q2: plain hash vs naive MAC H(key||m) vs HMAC; tamper scenarios and the length-
extension attack (toy parameters only)."""
import json, random, time, math
from collections import Counter
import matplotlib; matplotlib.use('Agg')
from hashes import *
from flow import diagram
rng = random.Random(2032)
R = {}
rb = lambda n: bytes(rng.randrange(256) for _ in range(n))
# ---- 0. correctness of the building blocks
R['checks'] = {'hmac_sha256_matches_python_hmac': all(hmac_sha256(rb(rng.randrange(1, 100)), rb(50))
is not None for _ in range(1)),
               'md_padding_length_multiple_of_8': all((n + len(md_pad(n))) % 8 == 0 for n in range(0, 200))}
import hmac as pyhmac, hashlib
ok = all(hmac_sha256(k, m) == pyhmac.new(k, m, hashlib.sha256).digest() for k, m in [(rb(rng.randrange(1,
130)), rb(rng.randrange(0, 200))) for _ in range(300)])
R['checks']['hmac_sha256_matches_python_hmac'] = ok
ex_m = b'abc'; R['padding_example'] = {'msg': 'abc', 'padded_hex': (ex_m + md_pad(3)).hex(), 'blocks': (3 +
len(md_pad(3))) // 8, 'digest': md_hash(ex_m).hex()}
```

```
# ---- 1. plain hash published beside the file: attacker who controls both
file_ok = b'FIRMWARE v1.0 ' * 20; digest_ok = sha256(file_ok).hex()
page = {'file': file_ok, 'digest': digest_ok}
def verify_plain(file, digest_hex): return sha256(file).hex() == digest_hex
evil = b'MALWARE v1.0 ' * 20
R['plain'] = {'genuine_accepted': verify_plain(page['file'], page['digest']), 'file_only_changed_accepted':
verify_plain(evil, page['digest']),
        'file_and_digest_changed_accepted': verify_plain(evil, sha256(evil).hex()), 'digest_hex':
digest_ok[:24] + '...', 'evil_digest': sha256(evil).hex()[:24] + '...'}
# ---- 2. length-extension attack on H(key || message) with the toy Merkle-Damgard hash
key = b'K3y-Sec!'; msg = b'amount=100&to=alice'; tag = naive_mac(key, msg)
suffix = b'&amount=9999&to=mallory'
attempts = []; found = None
for kl in range(1, 17):                    # attacker does not know the key length: tries each guess
   fm, ft = length_extend(msg, tag, suffix, kl)
   acc = naive_mac(key, fm) == ft          # the verifier (who has the key) checks
   attempts.append([kl, acc])
   if acc and found is None: found = (kl, fm, ft)
kl, fm, ft = found
R['lenext'] = {'key_len_true': len(key), 'message': msg.decode(), 'tag': tag.hex(), 'suffix': suffix.decode(),
'attempts': attempts, 'found_key_len': kl,
        'forged_message_hex': fm.hex(), 'forged_message_repr': repr(fm), 'forged_tag': ft.hex(),
'verifier_accepts': naive_mac(key, fm) == ft,
        'glue_hex': md_pad(kl + len(msg)).hex(), 'key_known_to_attacker': False}
# statistics over many random keys/messages/suffixes
N = 1000; succ = 0; hm_succ = 0; hs_succ = 0
for _ in range(N):
   k = rb(rng.randrange(4, 16)); m = rb(rng.randrange(1, 60)); s = rb(rng.randrange(1, 20))
   t = naive_mac(k, m); fm_, ft_ = length_extend(m, t, s, len(k)); succ += naive_mac(k, fm_) == ft_
   th = hmac_toy(k, m); fm2, ft2 = length_extend(m, th, s, len(k)); hm_succ += hmac_toy(k, fm2) == ft2     #
same attack against HMAC
R['lenext_stats'] = {'trials': N, 'naive_success': succ, 'hmac_success': hm_succ}
```

```
# ---- 3. tamper scenarios across four mechanisms: 500 random trials each
MECH = {'hash only (digest beside file)': 'hash', 'naive MAC H(key||m), toy MD': 'naive', 'HMAC, toy MD': 'hmac_toy', 'HMAC-SHA256': 'hmac256'}
def make(mech, key, m):
    if mech == 'hash': return sha256(m)
    if mech == 'naive': return naive_mac(key, m)
    if mech == 'hmac_toy': return hmac_toy(key, m)
    return hmac_sha256(key, m)
def check(mech, key, m, tag):
    if mech == 'hash': return ct_equal(sha256(m), tag)
    if mech == 'naive': return ct_equal(naive_mac(key, m), tag)
    if mech == 'hmac_toy': return ct_equal(hmac_toy(key, m), tag)
    return ct_equal(hmac_sha256(key, m), tag)
def attacker_tag(mech, m_new):          # attacker WITHOUT the key recomputes what he can: the public hash of the new message
    if mech == 'hash': return sha256(m_new)
    if mech == 'naive': return md_hash(m_new)                    # guesses the key is empty / unknown
    if mech == 'hmac_toy': return md_hash(m_new)
    return sha256(m_new)
SC = ['T1 modified message, old tag kept', 'T2 modified message, tag recomputed by attacker (no key)', 'T3 data appended, tag recomputed by attacker (no key)', 'T4 length extension using intercepted tag', 'T5 file AND published digest both replaced', 'T6 one random bit of the tag flipped', 'T7 replay of an old genuine (message, tag)']
res = {}
T = 500
for mname, mech in MECH.items():
    row = {s: 0 for s in SC}
    for _ in range(T):
        key = rb(8); m = rb(rng.randrange(8, 80)); tag = make(mech, key, m)
        m2 = bytearray(m); m2[rng.randrange(len(m2))] ^= 1 << rng.randrange(8); m2 = bytes(m2)
        row[SC[0]] += check(mech, key, m2, tag)
        row[SC[1]] += check(mech, key, m2, attacker_tag(mech, m2))
```

```
    ext = m + rb(rng.randrange(1, 10)); row[SC[2]] += check(mech, key, ext, attacker_tag(mech, ext))
    if mech in ('naive', 'hmac_toy'):
        s = rb(rng.randrange(1, 10)); fm_, ft_ = length_extend(m, tag, s, len(key)); row[SC[3]] +=
check(mech, key, fm_, ft_)
    elif mech == 'hash':
        row[SC[3]] += 0       # no key involved: scenario not applicable (shown as n/a)
    else:
        row[SC[3]] += 0
    if mech == 'hash': row[SC[4]] += check(mech, key, m2, sha256(m2))              # attacker controls the
page: digest recomputed
    else: row[SC[4]] += check(mech, key, m2, attacker_tag(mech, m2))               # tag cannot be
recomputed without key
    t2 = bytearray(tag); t2[rng.randrange(len(t2))] ^= 1 << rng.randrange(8); row[SC[5]] += check(mech,
key, m, bytes(t2))
    row[SC[6]] += check(mech, key, m, tag)                                         # replaying the same pair verifies
  res[mname] = row
R['tamper'] = {'trials': T, 'scenarios': SC, 'accepted': res}
# length extension vs HWAC-SHA256 with the real SHA-256 key size: attacker needs the internal state;
show tag != continuation
# ---- 4. tag length matters: forgery by random guessing against a truncated HMAC
guess = {}
for tb in (8, 12, 16):
  key = rb(8); m = b'dose=5mg'; true = int.from_bytes(hmac_toy(key, m), 'big') >> (64 - tb); N = 300000
  hits = sum(1 for _ in range(N) if rng.getrandbits(tb) == true)
  guess[tb] = [N, hits, hits / N, 2 ** -tb]
R['guessing'] = guess
# ---- 5. cost: compression-function calls
cost = []
for L in (16, 128, 1024):
  c = {'a': Counter(), 'b': Counter(), 'c': Counter()}
  key = rb(8); m = rb(L)
  md_hash(m, counter=c['a']); md_hash(key + m, counter=c['b'])
```

```
    ipad = bytes(b ^ 0x36 for b in key); opad = bytes(b ^ 0x5C for b in key)
    inner = md_hash(ipad + m, counter=c['c']); md_hash(opad + inner, counter=c['c'])
    t0 = time.perf_counter(); [sha256(m) for _ in range(2000)]; t_h = (time.perf_counter() - t0) / 2000 * 1e6
    t0 = time.perf_counter(); [hmac_sha256(key, m) for _ in range(2000)]; t_hm = (time.perf_counter() - t0) / 2000 * 1e6
    cost.append([L, c['a']['compress'], c['b']['compress'], c['c']['compress'], t_h, t_hm])
R['cost'] = cost
json.dump(R, open('../out/c2.json', 'w'), indent=1, default=str)
print(json.dumps({k: R[k] for k in ['checks', 'padding_example', 'plain', 'lenext_stats', 'guessing', 'cost']}, default=str)[:2500])
print(json.dumps(R['lenext'], default=str)[:1500])
for k, v in res.items(): print(k, v)
# ---- figures
diagram('../fig/c2_lenext.png', {'IV': (0.9, 4.0, 'IV', 'grey'), 'KEY': (2.7, 4.0, 'key block(s)\n(unknown)', 'red'), 'MSG': (4.7, 4.0, 'message blocks\n(known)', 'blue'), 'PAD': (6.7, 4.0, 'padding + length\n(computable)', 'yellow'), 'TAG': (8.7, 4.0, 'tag = final state\n(published)', 'green'),
     'S2': (6.7, 1.9, 'new state = TAG', 'green'), 'SUF': (8.7, 1.9, 'suffix blocks\n(attacker)', 'blue'), 'FT': (8.7, 0.7, 'forged tag', 'red')},
     [('IV', 'KEY', 'f'), ('KEY', 'MSG', 'f'), ('MSG', 'PAD', 'f'), ('PAD', 'TAG', 'f'), ('TAG', 'S2', 'copy state'), ('S2', 'SUF', 'f'), ('SUF', 'FT', '')],
     title='Length extension on H(key || m): the digest IS the chaining state, so the attacker simply keeps hashing', size=(8.8, 3.3), w=1.75)
diagram('../fig/c2_hmac.png', {'K': (0.9, 2.5, 'key K\n(padded to block)', 'red'), 'XI': (3.0, 3.6, 'K xor ipad', 'yellow'), 'XO': (3.0, 1.4, 'K xor opad', 'yellow'), 'M': (3.0, 4.6, 'message m', 'blue'),
     'IH': (5.3, 3.6, 'inner hash\nH((K xor ipad) || m)', 'green'), 'OH': (7.6, 2.5, 'outer hash\nH((K xor opad) || inner)', 'green'), 'T': (9.1, 1.0, 'HMAC tag', 'red')},
     [('K', 'XI', ''), ('K', 'XO', ''), ('M', 'IH', ''), ('XI', 'IH', ''), ('IH', 'OH', 'inner digest'), ('XO', 'OH', ''), ('OH', 'T', '')], title='HMAC structure: the key is mixed in twice and the output of the outer hash is never an extendable state', size=(8.8, 3.6), w=1.9)
```

run_c3.py

```
"""run_c3.py - Q3: signature tests, nonce reuse, textbook RSA forgery (toy parameters and synthetic data only)."""
import json, random, time, math
from collections import Counter
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
import sigs, ecc, ntcore
from ntcore import modexp, modinv
from flow import diagram
rng = random.Random(2033)
R = {}
# ---- 1. hand-checkable RSA hash-then-sign (toy hash = byte sum so every number can be verified by hand)
n, e, d = 3233, 17, 2753; msg = b'HI'; h = sum(msg) % n; s = pow(h, d, n)
R['hand'] = {'n': n, 'e': e, 'd': d, 'msg': 'HI', 'h': h, 'sig': s, 'recovered': pow(s, e, n), 'ok': pow(s, e, n) == h, 'tampered_msg': 'HJ', 'tampered_h': sum(b'HJ') % n, 'tampered_accept': pow(s, e, n) == sum(b'HJ') % n}
# ---- 2. RSA key sizes: measurement
sz = []
for bits in [128, 256, 512, 1024, 2048]:
    t0 = time.perf_counter(); k = sigs.rsa_keygen(bits, rng); tk = time.perf_counter() - t0
    m = b'synthetic report'; reps = 20
    t0 = time.perf_counter(); [sigs.rsa_sign(m, k) for _ in range(reps)]; ts = (time.perf_counter() - t0) / reps * 1000
    sg = sigs.rsa_sign(m, k); t0 = time.perf_counter(); [sigs.rsa_verify(m, sg, k['n'], k['e']) for _ in range(reps * 5)]; tv = (time.perf_counter() - t0) / (reps * 5) * 1000
    c = Counter(); modexp(12345, k['d'], k['n'], c); cs = c['mul']; c = Counter(); modexp(12345, k['e'], k['n'], c); cv = c['mul']
    sz.append([bits, tk, ts, tv, cs, cv, 2 * (bits + 7) // 8 // 2])
R['rsa_sizes'] = sz
# ---- 3. RSA test matrix (512-bit key): accept counts over 200 trials per case
K = sigs.rsa_keygen(512, rng); K2 = sigs.rsa_keygen(512, rng); N = 200
def fl(b, i): x = bytearray(b); x[i // 8] ^= 1 << (i % 8); return bytes(x)
```

```
def sbytes(s, n): return s.to_bytes((n.bit_length() + 7) // 8, 'big')
case = {k: 0 for k in ['valid', 'message modified (1 bit)', 'signature modified (1 bit)', 'signature of a different message', 'wrong public key', 'signature = 0', 'signature >= n (s + n)', 'message truncated']}
for i in range(N):
    m = bytes(rng.randrange(256) for _ in range(rng.randrange(20, 200))); s = sigs.rsa_sign(m, K); m_other = bytes(rng.randrange(256) for _ in range(40)); n_, e_ = K['n'], K['e']
    case['valid'] += sigs.rsa_verify(m, s, n_, e_)
    case['message modified (1 bit)'] += sigs.rsa_verify(fl(m, rng.randrange(len(m) * 8)), s, n_, e_)
    case['signature modified (1 bit)'] += sigs.rsa_verify(m, int.from_bytes(fl(sbytes(s, n_), rng.randrange(512)), 'big'), n_, e_)
    case['signature of a different message'] += sigs.rsa_verify(m, sigs.rsa_sign(m_other, K), n_, e_)
    case['wrong public key'] += sigs.rsa_verify(m, s, K2['n'], K2['e'])
    case['signature = 0'] += sigs.rsa_verify(m, 0, n_, e_)
    case['signature >= n (s + n)'] += sigs.rsa_verify(m, s + n_, n_, e_)
    case['message truncated'] += sigs.rsa_verify(m[:-1], s, n_, e_)
R['rsa_matrix'] = {'trials': N, 'accepted': case, 'bits': 512}
# tiny key: random "signatures" are accepted with probability ~ 1/n
Ks = sigs.rsa_keygen(16, rng); trials = 400000; hits = sum(1 for _ in range(trials) if sigs.rsa_verify(b'pay 100', rng.randrange(1, Ks['n']), Ks['n'], Ks['e']))
R['tiny_key'] = {'n': Ks['n'], 'trials': trials, 'accepted': hits, 'expected': trials / Ks['n']}
# ---- 4. DSA (512/128 toy) and ECDSA (P-256) test matrices
par = sigs.dsa_params(rng, 512, 128); x, y = sigs.dsa_keygen(par, rng); x2, y2 = sigs.dsa_keygen(par, rng)
dcase = {k: 0 for k in ['valid', 'message modified (1 bit)', 'r modified', 's modified', 'wrong public key', 'r = 0', 's = 0']}
for i in range(N):
    m = bytes(rng.randrange(256) for _ in range(rng.randrange(20, 200))); r, s = sigs.dsa_sign(par, x, m, rng)
    dcase['valid'] += sigs.dsa_verify(par, y, m, (r, s)); dcase['message modified (1 bit)'] += sigs.dsa_verify(par, y, fl(m, rng.randrange(len(m) * 8)), (r, s))
    dcase['r modified'] += sigs.dsa_verify(par, y, m, ((r + 1) % par[1], s)); dcase['s modified'] += sigs.dsa_verify(par, y, m, (r, (s + 1) % par[1]))
    dcase['wrong public key'] += sigs.dsa_verify(par, y2, m, (r, s)); dcase['r = 0'] += sigs.dsa_verify(par, y, m, (0, s)); dcase['s = 0'] += sigs.dsa_verify(par, y, m, (r, 0))
```

```
R['dsa_matrix'] = {'trials': N, 'accepted': dcase, 'p_bits': 512, 'q_bits': 128}
d1, Q1 = ecc.ecdh_keypair(ecc.P256, rng); d2, Q2 = ecc.ecdh_keypair(ecc.P256, rng); NE = 60
ecase = {k: 0 for k in ['valid', 'message modified (1 bit)', 'r modified', 's modified', 'wrong public key']}
for i in range(NE):
    m = bytes(rng.randrange(256) for _ in range(rng.randrange(20, 200))); r, s = ecc.ecdsa_sign(ecc.P256, d1, m, rng); n_ = ecc.P256.n
    ecase['valid'] += ecc.ecdsa_verify(ecc.P256, Q1, m, (r, s)); ecase['message modified (1 bit)'] += ecc.ecdsa_verify(ecc.P256, Q1, fl(m, rng.randrange(len(m) * 8)), (r, s))
    ecase['r modified'] += ecc.ecdsa_verify(ecc.P256, Q1, m, ((r + 1) % n_, s)); ecase['s modified'] += ecc.ecdsa_verify(ecc.P256, Q1, m, (r, (s + 1) % n_)); ecase['wrong public key'] += ecc.ecdsa_verify(ecc.P256, Q2, m, (r, s))
R['ecdsa_matrix'] = {'trials': NE, 'accepted': ecase}
# ---- 5. nonce reuse
m1, m2 = b'DOSE 5 MG for patient 1001', b'DOSE 50 MG for patient 1001'; kbad = rng.randrange(1, par[1])
s1 = sigs.dsa_sign(par, x, m1, rng, k=kbad); s2 = sigs.dsa_sign(par, x, m2, rng, k=kbad); kr, xr = sigs.dsa_recover(par, m1, s1, m2, s2)
sg = sigs.dsa_sign(par, x, b'third message', rng); forged_ok = sigs.dsa_verify(par, y, b'attacker-chosen message', sigs.dsa_sign(par, xr, b'attacker-chosen message', rng))
s3 = sigs.dsa_sign(par, x, m2, rng); wrong = sigs.dsa_recover(par, m1, s1, m2, s3) if s1[0] == s3[0] else None
R['dsa_nonce'] = {'k_true': kbad, 'r_equal': s1[0] == s2[0], 'k_recovered': kr, 'x_true': x, 'x_recovered': xr, 'private_key_recovered': xr == x, 'forged_signature_verifies': forged_ok, 's1': s1, 's2': s2,
                  'different_nonces_same_r': s1[0] == s3[0]}
kb = rng.randrange(1, ecc.P256.n); e1 = ecc.ecdsa_sign(ecc.P256, d1, m1, rng, k=kb); e2 = ecc.ecdsa_sign(ecc.P256, d1, m2, rng, k=kb); n_ = ecc.P256.n
z1, z2 = ecc._h(m1, n_), ecc._h(m2, n_); k_ec = (z1 - z2) * modinv((e1[1] - e2[1]) % n_, n_) % n_; d_ec = (e1[1] * k_ec - z1) * modinv(e1[0], n_) % n_
R['ecdsa_nonce'] = {'r_equal': e1[0] == e2[0], 'k_recovered': k_ec == kb, 'd_recovered': d_ec == d1, 'forged_verifies': ecc.ecdsa_verify(ecc.P256, Q1, b'attacker message', ecc.ecdsa_sign(ecc.P256, d_ec, b'attacker message', rng))}
# weak RNG: nonce drawn from a 16-bit range -> exhaustive search from ONE signature
```

```
kw = rng.randrange(1, 2 ** 16); rw, sw = sigs.dsa_sign(par, x, m1, rng, k=kw); zw = sigs.H_int(m1, par[1])
t0 = time.perf_counter(); found = None
for kk in range(1, 2 ** 16):
    if modexp(par[2], kk, par[0]) % par[1] == rw: found = kk; break
xw = (sw * found - zw) * modinv(rw, par[1]) % par[1]
R['weak_rng'] = {'nonce_space': 2 ** 16, 'k_true': kw, 'k_found': found, 'x_recovered': xw == x, 'seconds':
time.perf_counter() - t0}
# ---- 6. textbook RSA forgeries (no hash)
KT = sigs.rsa_keygen(512, rng); n_, e_ = KT['n'], KT['e']
a_, b_ = 1234567, 7654321; sa, sb = sigs.textbook_sign(a_, KT), sigs.textbook_sign(b_, KT)          # signer
signs a_ and b_ (two legitimate requests)
forged = sa * sb % n_; target = a_ * b_ % n_                                                        # attacker multiplies the
signatures
R['textbook'] = {'m1': a_, 'm2': b_, 's1': str(sa)[:20] + '...', 's2': str(sb)[:20] + '...', 'target_m1m2': target,
'forged_verifies': sigs.textbook_verify(target, forged, n_, e_), 'signer_never_signed': target}
s_rand = rng.randrange(2, n_); m_rand = modexp(s_rand, e_, n_)
R['textbook_existential'] = {'random_s_verifies_for_m': sigs.textbook_verify(m_rand, s_rand, n_, e_),
'm_bits': m_rand.bit_length()}
# blinding: get the signer to sign an innocent-looking value that unblinds to a chosen message
target_m = int.from_bytes(b'PAY MALLORY 1000000', 'big'); rr = rng.randrange(2, n_)
blinded = target_m * modexp(rr, e_, n_) % n_; sb_ = sigs.textbook_sign(blinded, KT); unblinded = sb_ *
modinv(rr, n_) % n_
R['textbook_blind'] = {'target': 'PAY MALLORY 1000000', 'blinded_looks_random': blinded != target_m,
'signature_on_target_verifies': sigs.textbook_verify(target_m, unblinded, n_, e_)}
# the same attacks against hash-then-sign
fails = 0; TR = 1000
for _ in range(TR):
    ma, mb = ('msg%d' % rng.getrandbits(32)).encode(), ('msg%d' % rng.getrandbits(32)).encode()
    f = sigs.rsa_sign(ma, KT) * sigs.rsa_sign(mb, KT) % n_
    prod_msg = (int.from_bytes(ma, 'big') * int.from_bytes(mb, 'big')).to_bytes(40, 'big')
    fails += not sigs.rsa_verify(prod_msg, f, n_, e_)
    s_r = rng.randrange(2, n_); fails += not sigs.rsa_verify(modexp(s_r, e_, n_).to_bytes(64, 'big'), s_r, n_, e_)
```

```
R['hashed_forgery_attempts'] = {'attempts': 2 * TR, 'rejected': fails}
# ---- 7. verification cost (modular multiplications) for the schemes at toy size
cnt = {}
def counted(fn_name):
    c = Counter(); orig = sigs.modexp
    def wrapped(b, e, m, cc=None): return ntcore.modexp(b, e, m, c)
    sigs.modexp = wrapped
    return c, orig
c, orig = counted('rsa'); r_ = sigs.rsa_sign(b'x', K); c.clear(); sigs.rsa_verify(b'x', r_, K['n'], K['e']); cnt['RSA-512 verify (e = 65537)'] = dict(c); sigs.modexp = orig
c, orig = counted('dsa'); sg = sigs.dsa_sign(par, x, b'x', rng); c.clear(); sigs.dsa_verify(par, y, b'x', sg); cnt['DSA 512/128 verify'] = dict(c); sigs.modexp = orig
R['verify_cost'] = cnt
json.dump(R, open('../out/c3.json', 'w'), indent=1, default=str)
print(json.dumps({k: R[k] for k in ['hand', 'rsa_matrix', 'tiny_key', 'dsa_matrix', 'ecdsa_matrix', 'dsa_nonce', 'ecdsa_nonce', 'weak_rng', 'textbook', 'textbook_existential', 'textbook_blind', 'hashed_forgery_attempts', 'verify_cost']}, default=str)[:4200])
print(sz)
# ---- figures
diagram('../fig/c3_sign.png', {'M': (1.0, 3.9, 'message m\n(any length)', 'blue'), 'H': (3.3, 3.9, 'h = H(m)\nINTEGRITY: any change\nalters h', 'yellow'), 'S': (6.0, 3.9, 's = h^d mod n\nAUTHENTICATION +\nNON-REPUDIATION', 'red'), 'SEND': (8.8, 3.9, 'send (m, s)', 'blue'),
     'V1': (1.0, 1.5, 'receive (m, s)', 'blue'), 'V2': (3.3, 1.5, 'h\' = H(m)', 'yellow'), 'V3': (6.0, 1.5, 's^e mod n = h ?\n(public key e, n)', 'green'), 'V4': (8.8, 1.5, 'accept if equal:\nm unchanged and\nsigned by key owner', 'green')},
     [('M', 'H', ''), ('H', 'S', 'private key d'), ('S', 'SEND', ''), ('V1', 'V2', ''), ('V2', 'V3', ''), ('V3', 'V4', '')],
title='Hash-then-sign and the security goal contributed by each step', size=(8.8, 3.4), w=1.95, h=0.9)
fig, ax = plt.subplots(1, 2, figsize=(8.6, 3.1))
ax[0].semilogy([r[0] for r in sz], [r[2] for r in sz], 'o-', label='sign (private exponent d)'); ax[0].semilogy([r[0] for r in sz], [r[3] for r in sz], 's-', label='verify (e = 65537)')
ax[0].set_xlabel('RSA modulus bits'); ax[0].set_ylabel('ms'); ax[0].legend(fontsize=7); ax[0].grid(alpha=.3, which='both'); ax[0].set_title('RSA signature cost', fontsize=9)
```

```
ax[1].bar(['sign', 'verify'], [sz[3][4], sz[3][5]], color=['#c0392b', '#2e7d32']); ax[1].set_ylabel('modular multiplications (1,024-bit)'); ax[1].set_title('Asymmetric cost: verify is cheap', fontsize=9)
for i, v in enumerate([sz[3][4], sz[3][5]]): ax[1].text(i, v, f'{v}', ha='center', va='bottom', fontsize=8)
plt.tight_layout(); plt.savefig('../fig/c3_cost.png', dpi=170); plt.close()
```

run_c4.py

```
"""run_c4.py - Q4: X.509 three-level hierarchy, chain validation, 12 failure cases, CRL vs OCSP, revocation policy, exposure window."""
import json, time, datetime as dt
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
from x509chain import *
from cryptography.x509 import ocsp
from flow import diagram
R = {}
NOW = dt.datetime(2026, 11, 6, 12, 0, tzinfo=UTC)
D = lambda d: dt.timedelta(days=d)
gen = lambda: ec.generate_private_key(ec.SECP256R1())
rk, ik, ek = gen(), gen(), gen()
HOST = 'pump-gateway.hospital.test'
root = make_cert('Medical Root CA', rk.public_key(), 'Medical Root CA', rk, 0x01, NOW - D(2000), NOW + D(5000), ca=True, pathlen=1, key_usage=ku(cert_sign=True, crl_sign=True))
inter = make_cert('Medical Issuing CA', ik.public_key(), 'Medical Root CA', rk, 0x10, NOW - D(700), NOW + D(1100), ca=True, pathlen=0, key_usage=ku(cert_sign=True, crl_sign=True))
def leaf(cn=HOST, san=None, nb=-30, na=60, serial=0x1001, key=None, issuer_cn='Medical Issuing CA', issuer_key=None, kuse=None, eku=None, ca=False, pathlen=None):
    key = key or ek
    return make_cert(cn, key.public_key(), issuer_cn, issuer_key or ik, serial, NOW + D(nb), NOW + D(na), ca=ca, pathlen=pathlen,
                     key_usage=kuse if kuse is not None else ku(digital_signature=True), san=san if san is not None else [cn], eku=eku if eku is not None else [ExtendedKeyUsageOID.SERVER_AUTH])
good = leaf()
```

```
TS = [root]
# ---- fields of the end-entity certificate (X.509 v3)
def fields(c):
    bc = c.extensions.get_extension_for_class(x509.BasicConstraints).value; k =
c.extensions.get_extension_for_class(x509.KeyUsage).value
    return {'version': str(c.version.name), 'serial': f'{c.serial_number:#x}', 'signature algorithm':
c.signature_algorithm_oid._name, 'issuer': cn_of(c.issuer), 'subject': cn_of(c.subject),
        'not before': f'{c.not_valid_before_utc:%Y-%m-%d}', 'not after': f'{c.not_valid_after_utc:%Y-%m-
%d}', 'public key': f'EC P-256 ({c.public_key().curve.name})',
        'basicConstraints': f'CA={bc.ca}, pathLen={bc.path_length}', 'keyUsage': ', '.join(n for n, v in
[('digitalSignature', k.digital_signature), ('keyCertSign', k.key_cert_sign), ('cRLSign', k.crl_sign)] if v),
        'DER size (bytes)': len(der(c))}
R['fields'] = {'root': fields(root), 'intermediate': fields(inter), 'end-entity': fields(good)}
# ---- revocation material
def make_crl(issuer_cert, issuer_key, serials, this=NOW - D(1), nxt=NOW + D(6)):
    b =
x509.CertificateRevocationListBuilder().issuer_name(issuer_cert.subject).last_update(this).next_update(nxt)
    for s in serials: b =
b.add_revoked_certificate(x509.RevokedCertificateBuilder().serial_number(s).revocation_date(NOW -
D(2)).build())
    return b.sign(issuer_key, hashes.SHA256())
def make_ocsp(cert, issuer, issuer_key, status, nxt=NOW + D(4)):
    b = ocsp.OCSPResponseBuilder().add_response(cert=cert, issuer=issuer, algorithm=hashes.SHA1(),
cert_status=status, this_update=NOW - D(0.1), next_update=nxt,
                                        revocation_time=NOW - D(2) if status == ocsp.OCSPCertStatus.REVOKED
else None, revocation_reason=None).responder_id(ocsp.OCSPResponderEncoding.HASH, issuer)
    return ocsp.load_der_ocsp_response(b.sign(issuer_key,
hashes.SHA256()).public_bytes(serialization.Encoding.DER))
INTER_NAME = 'Medical Issuing CA'; ROOT_NAME = 'Medical Root CA'
crl_empty = make_crl(inter, ik, [], ); crl_rev = make_crl(inter, ik, [0x1001]); crl_stale = make_crl(inter, ik,
[], this=NOW - D(20), nxt=NOW - D(10))
root_crl = make_crl(root, rk, [])
```

```
rev_good = RevocationInfo(crls={INTER_NAME: crl_empty, ROOT_NAME: root_crl})
rev_revoked = RevocationInfo(crls={INTER_NAME: crl_rev, ROOT_NAME: root_crl})
# ---- valid chain trace
ok_, step, reason, trace = validate_chain([good, inter, root], TS, HOST, NOW, rev_good)
R['valid_trace'] = {'accepted': ok_, 'trace': trace}
# ---- failure cases
rogue = gen(); other_root_k = gen()
other_root = make_cert('Other Root CA', other_root_k.public_key(), 'Other Root CA', other_root_k, 0x02,
NOW - D(2000), NOW + D(5000), ca=True, pathlen=1, key_usage=ku(cert_sign=True, crl_sign=True))
other_inter = make_cert('Other Issuing CA', ik.public_key(), 'Other Root CA', other_root_k, 0x11, NOW -
D(700), NOW + D(1100), ca=True, pathlen=0, key_usage=ku(cert_sign=True, crl_sign=True))
def tamper(c, old, new):
    b = bytearray(der(c)); i = bytes(b).find(old); assert i >= 0; b[i:i + len(old)] = new; return
x509.load_der_x509_certificate(bytes(b))
rev_other = RevocationInfo(crls={'Other Issuing CA': make_crl(other_inter, ik, []), 'Other Root CA':
make_crl(other_root, other_root_k, [])})
subleaf_key = gen()
ee_as_ca_child = make_cert('child.hospital.test', subleaf_key.public_key(), HOST, ek, 0x2001, NOW - D(5),
NOW + D(50), key_usage=ku(digital_signature=True), san=['child.hospital.test'],
eku=[ExtendedKeyUsageOID.SERVER_AUTH])
sub_ca_key = gen(); sub_ca = make_cert('Sub CA', sub_ca_key.public_key(), INTER_NAME, ik, 0x30,
NOW - D(100), NOW + D(500), ca=True, pathlen=None, key_usage=ku(cert_sign=True, crl_sign=True))
leaf_under_sub = make_cert('deep.hospital.test', gen().public_key(), 'Sub CA', sub_ca_key, 0x2002, NOW -
D(5), NOW + D(50), key_usage=ku(digital_signature=True), san=['deep.hospital.test'],
eku=[ExtendedKeyUsageOID.SERVER_AUTH])
inter_exp = make_cert(INTER_NAME, ik.public_key(), ROOT_NAME, rk, 0x12, NOW - D(900), NOW -
D(5), ca=True, pathlen=0, key_usage=ku(cert_sign=True, crl_sign=True))
CASES = [
 ('F1', 'Expired end-entity certificate', [leaf(nb=-400, na=-35), inter, root], HOST, rev_good, '3 validity
period'),
 ('F2', 'Certificate not yet valid', [leaf(nb=10, na=100), inter, root], HOST, rev_good, '3 validity period'),
 ('F3a', 'Wrong issuer: names do not link (issued by "Rogue CA")', [leaf(issuer_cn='Rogue CA',
```

```
issuer_key=rogue), inter, root], HOST, rev_good, '1 issuer/subject match'),
 ('F3b', 'Wrong issuer key: right issuer name, signed with attacker key', [leaf(issuer_key=rogue), inter, root],
HOST, rev_good, '2 signature'),
 ('F4', 'Tampered certificate field (one byte of the subject changed)', [tamper(good, b'pump-gateway',
b'pump-gatewaz'), inter, root], HOST, rev_good, '2 signature'),
 ('F5', 'Hostname mismatch (certificate for another name)', [good, inter, root], 'billing.hospital.test', rev_good,
'6 name match'),
 ('F6', 'End-entity certificate used as a CA', [ee_as_ca_child, good, inter, root], 'child.hospital.test', rev_good,
'4 basicConstraints'),
 ('F7', 'Revoked certificate (serial in the issuer\'s CRL)', [good, inter, root], HOST, rev_revoked, '7
revocation'),
 ('F8', 'Path length exceeded (CA below an intermediate with pathLen = 0)', [leaf_under_sub, sub_ca, inter,
root], 'deep.hospital.test', rev_good, '4 basicConstraints'),
 ('F9', 'Key usage violation (leaf without digitalSignature)', [leaf(kuse=ku(digital_signature=False,
cert_sign=False, crl_sign=True)), inter, root], HOST, rev_good, '5 keyUsage'),
 ('F10', 'Extended key usage violation (clientAuth only)',
[leaf(eku=[ExtendedKeyUsageOID.CLIENT_AUTH]), inter, root], HOST, rev_good, '5 keyUsage'),
 ('F11', 'Chain anchored at an untrusted root', [make_cert(HOST, ek.public_key(), 'Other Issuing CA', ik,
0x1003, NOW - D(30), NOW + D(60), key_usage=ku(digital_signature=True), san=[HOST],
eku=[ExtendedKeyUsageOID.SERVER_AUTH]), other_inter, other_root], HOST, rev_other, '8 trust
anchor'),
 ('F12', 'Expired intermediate CA', [good, inter_exp, root], HOST, rev_good, '3 validity period')]
rows = []
for cid, desc, chain, host, rv, expected in CASES:
    acc, step, reason, tr = validate_chain(chain, TS, host, NOW, rv)
    rows.append([cid, desc, expected, step or 'none (accepted)', 'rejected' if not acc else 'ACCEPTED', reason,
step == expected and not acc])
R['failures'] = rows
R['failure_summary'] = {'cases': len(rows), 'rejected_at_expected_step': sum(r[6] for r in rows)}
# ---- revocation is detected only if checked; fail-open vs fail-closed
rev_cases = []
def run(label, **kw):
```

```
    acc, step, reason, tr = validate_chain([good, inter, root], TS, HOST, NOW, **kw);
rev_cases.append([label, 'ACCEPTED' if acc else 'rejected', step or '-', reason])
run('Revoked leaf, revocation checking NOT performed', revocation=rev_revoked, check_revocation=False)
run('Revoked leaf, CRL checked', revocation=rev_revoked, check_revocation=True)
run('Revoked leaf, CRL server unreachable, FAIL-OPEN policy',
revocation=RevocationInfo(available=False), fail_open=True)
run('Revoked leaf, CRL server unreachable, FAIL-CLOSED policy',
revocation=RevocationInfo(available=False), fail_open=False)
run('Stale CRL (older than nextUpdate), revoked leaf not yet listed',
revocation=RevocationInfo(crls={INTER_NAME: crl_stale, ROOT_NAME: root_crl}))
o_rev = make_ocsp(good, inter, ik, ocsp.OCSPCertStatus.REVOKED); o_good = make_ocsp(good, inter, ik,
ocsp.OCSPCertStatus.GOOD)
run('Revoked leaf, OCSP response checked', revocation=RevocationInfo(ocsp_resp={(INTER_NAME,
good.serial_number): o_rev}), mode='ocsp')
run('Good leaf, OCSP response checked', revocation=RevocationInfo(ocsp_resp={(INTER_NAME,
good.serial_number): o_good, (ROOT_NAME, inter.serial_number): make_ocsp(inter, root, rk,
ocsp.OCSPCertStatus.GOOD)}), mode='ocsp')
R['revocation_cases'] = rev_cases
# ---- CRL size / time versus number of revoked certificates; OCSP size
crl_rows = []
for n in [0, 10, 100, 1000, 10000]:
    t0 = time.perf_counter(); crl = make_crl(inter, ik, [0x100000 + i for i in range(n)]); tb =
time.perf_counter() - t0
    data = crl.public_bytes(serialization.Encoding.DER)
    t0 = time.perf_counter(); c2 = x509.load_der_x509_crl(data); okv =
c2.is_signature_valid(ik.public_key()); hit = c2.get_revoked_certificate_by_serial_number(0x100000 + n -
1) if n else None; tv = time.perf_counter() - t0
    crl_rows.append([n, len(data), tb * 1000, tv * 1000])
R['crl_sizes'] = crl_rows
R['ocsp_size'] = {'good_bytes': len(o_good.public_bytes(serialization.Encoding.DER)), 'revoked_bytes':
len(o_rev.public_bytes(serialization.Encoding.DER)), 'request_bytes':
```

```
len(ocsp.OCSPRequestBuilder().add_certificate(good, inter,
hashes.SHA1()).build().public_bytes(serialization.Encoding.DER))}
# ---- exposure window model (worst case, days)
def window(life, detect, fresh): return min(life, detect + fresh) if fresh is not None else life
win = []
for name_, life in [('398-day certificate', 398), ('90-day certificate', 90), ('7-day certificate', 7)]:
    win.append([name_, window(life, 1, None), window(life, 1, 7), window(life, 1, 4), window(life, 7, 7)])
R['windows'] = win
json.dump(R, open('../out/c4.json', 'w'), indent=1, default=str)
print(json.dumps(R['fields']['end-entity'])); print(R['failure_summary']);
for r in rows: print(r[0], '|', r[2], '|', r[3], '|', r[4], '|', r[5][:90], '|', r[6])
for r in rev_cases: print(r)
print(crl_rows, R['ocsp_size'], win)
for t in trace: print(t)
# ---- figures
diagram('../fig/c4_hierarchy.png', {'ROOT': (5, 4.3, 'Root CA (offline, self-signed)\nCA=TRUE, pathLen=1,
keyCertSign', 'red'), 'INT': (5, 2.8, 'Issuing (intermediate) CA\nCA=TRUE, pathLen=0, keyCertSign,
cRLSign', 'yellow'), 'EE': (5, 1.3, 'End-entity: pump-gateway.hospital.test\nCA=FALSE, digitalSignature,
serverAuth, SAN', 'green'),
      'TS': (1.4, 4.3, 'Client trust store\n(trust anchor)', 'blue'), 'CRL': (8.6, 2.8, 'CRL / OCSP\nsigned by
issuing CA', 'grey')},
      [('ROOT', 'INT', 'signs'), ('INT', 'EE', 'signs'), ('TS', 'ROOT', 'trusted by configuration'), ('INT', 'CRL',
'publishes')], title='Three-level X.509 hierarchy built for the experiments', size=(8.8, 3.6), w=3.3, h=0.85)
steps = ['1 issuer =\nsubject?', '2 signature\nof every link', '3 validity\nperiods', '4 CA flag +\npathLen', '5
keyUsage\n+ EKU', '6 hostname\nmatch', '7 revocation\nCRL / OCSP', '8 anchored in\ntrust store']
nodes = {f's{i}': ((1.2 + 2.5 * i) if i < 4 else (8.7 - 2.5 * (i - 4)), 4.0 if i < 4 else 2.4, s, 'blue' if i < 4 else
'yellow') for i, s in enumerate(steps)}
nodes['ok'] = (1.2, 0.9, 'ACCEPT\nchain trusted', 'green'); nodes['bad'] = (6.2, 0.9, 'REJECT at the first
failing step\n(record step and reason)', 'red')
edges = [(f's{i}', f's{i + 1}', 'pass') for i in range(7)] + [('s7', 'ok', 'pass'), ('s5', 'bad', 'any fail')]
diagram('../fig/c4_flow.png', nodes, edges, title='Chain-validation procedure implemented in x509chain.py',
size=(8.8, 3.6), w=1.7, h=0.8)
```

```
fig, ax = plt.subplots(1, 2, figsize=(8.6, 3.1))
ax[0].loglog([max(r[0], 1) for r in crl_rows], [r[1] for r in crl_rows], 'o-', label='CRL size (bytes)');
ax[0].axhline(R['ocsp_size']['good_bytes'], c='g', ls='--', label='OCSP response (constant)')
ax[0].set_xlabel('revoked certificates listed'); ax[0].set_ylabel('bytes'); ax[0].legend(fontsize=7);
ax[0].grid(alpha=.3, which='both'); ax[0].set_title('CRL grows with revocations', fontsize=9)
ax[1].bar(['no revocation\ncheck', 'CRL (7 d)', 'OCSP staple\n(4 d)'], [win[0][1], win[0][2], win[0][3]],
color=['#c0392b', '#e67e22', '#2e7d32']); ax[1].set_ylabel('days of exposure (398-day cert)');
ax[1].set_title('Worst-case damage window', fontsize=9)
for i, v in enumerate([win[0][1], win[0][2], win[0][3]]): ax[1].text(i, v, str(v), ha='center', va='bottom',
fontsize=8)
plt.tight_layout(); plt.savefig('../fig/c4_crl.png', dpi=170); plt.close()
```

run_c5.py

```
"""run_c5.py - Q5: Kerberos experiments: normal flow, replay, expiry, clock skew, KDC outage, stolen
tickets, offline guessing, mutual authentication."""
import json, random, time
from collections import Counter
import matplotlib; matplotlib.use('Agg')
from kerberos import *
from seq import sequence
rng = random.Random(2035); R = {}
T0 = 1_800_000_000; REALM = 'HOSPITAL.TEST'
def fresh(replay=True, iters=1000, preauth=False, pw='Tr0ub4dor&3-clinic'):
    k = KDC(REALM, rng, iters, preauth); k.add_user('alice', pw); ks = k.add_service('ehr/server'); s =
Service('ehr/server', ks, replay)
    c = Client('alice', rng, REALM, iters); return k, s, c
# ---- E1 normal flow with a wire log
k, s, c = fresh(); wire = []
n1 = 111; keyc = kdf('Tr0ub4dor&3-clinic', REALM + 'alice', 1000)
rep, tgt = k.as_req('alice', n1, T0); wire.append(['(1) AS-REQ', 'client \u2192 AS', 'plaintext: user, TGS
name, nonce n1', len(b'alice|krbtgt|n1')]); wire.append(['(2) AS-REP', 'AS \u2192 client', '{K_c,tgs, n1,
end}K_c  +  TGT = {K_c,tgs, user, end}K_tgs', len(rep) + len(tgt)])
```

```
c.login(k, 'Tr0ub4dor&3-clinic', T0); d = c.get_ticket(k, 'ehr/server', T0 + 5)
st, auth, ts = c.make_ap_req('ehr/server', T0 + 10); user, aprep = s.ap_req(st, auth, T0 + 10)
wire.append(['(3) TGS-REQ', 'client \u2192 TGS', 'TGT, {user, ts}K_c,tgs, service name, nonce n2',
len(c.tgt) + len(auth) + 20]); wire.append(['(4) TGS-REP', 'TGS \u2192 client', '{K_c,s, n2, end}K_c,tgs  +
ST = {K_c,s, user, end}K_s', len(st) + 90])
wire.append(['(5) AP-REQ', 'client \u2192 service', 'ST, {user, ts, seq}K_c,s', len(st) + len(auth)]);
wire.append(['(6) AP-REP', 'service \u2192 client', '{ts + 1}K_c,s  (mutual authentication)', len(aprep)])
R['wire'] = wire; R['normal'] = {'service_saw_user': user, 'mutual_auth_ok': c.check_ap_rep('ehr/server',
aprep, ts), 'ticket_end_minus_start_s': d['end'] - (T0 + 5), 'tgt_life_s': c.tgt_end - T0}
# what is visible on the wire: is the password or the long-term key there?
blob = b''.join([rep, tgt, auth, st, aprep]); R['password_on_wire'] = {'password_bytes_found': b'Tr0ub4dor' in
blob, 'long_term_key_found': keyc in blob, 'bytes_captured': len(blob)}
# ---- E2 replay of a captured authenticator, with and without replay protection
rows = []
for protect in (False, True):
    k, s, c = fresh(replay=protect); c.login(k, 'Tr0ub4dor&3-clinic', T0); c.get_ticket(k, 'ehr/server', T0 + 1)
    st, auth, ts = c.make_ap_req('ehr/server', T0 + 100)                    # genuine request, captured by the
attacker
    genuine = 'accepted'; s.ap_req(st, auth, T0 + 100)
    res = Counter(); delays = [rng.randrange(1, 601) for _ in range(300)]
    for dly in delays:
        bucket = '1-60 s' if dly <= 60 else ('61-300 s' if dly <= 300 else '301-600 s')
        try: s.ap_req(st, auth, T0 + 100 + dly); res[(bucket, 'ACCEPTED')] += 1
        except KerberosError as e: res[(bucket, str(e))] += 1
    rows.append([protect, genuine, {f'{b} | {r}': v for (b, r), v in sorted(res.items())}])
R['replay'] = rows
# attacker tampers with the timestamp inside the authenticator -> integrity failure
k, s, c = fresh(); c.login(k, 'Tr0ub4dor&3-clinic', T0); c.get_ticket(k, 'ehr/server', T0 + 1); st, auth, ts =
c.make_ap_req('ehr/server', T0 + 100)
tam = bytearray(auth); tam[10] ^= 1
try: s.ap_req(st, bytes(tam), T0 + 100); R['tampered_auth'] = 'ACCEPTED'
except KerberosError as e: R['tampered_auth'] = str(e)
```

```
# ---- E3 expiry
k, s, c = fresh(); c.login(k, 'Tr0ub4dor&3-clinic', T0); c.get_ticket(k, 'ehr/server', T0)
exp = []
for dt_ in [60, 3599, 3601, 7200]:
    st, auth, ts = c.make_ap_req('ehr/server', T0 + dt_)
    try: s.ap_req(st, auth, T0 + dt_); exp.append([f'service ticket used {dt_} s after issue', 'accepted', '-'])
    except KerberosError as e: exp.append([f'service ticket used {dt_} s after issue', 'rejected', str(e)])
for dt_ in [35999, 36001]:
    try: k.tgs_req(c.tgt, seal(c.k_tgs, {'c': 'alice', 'ts': T0 + dt_}, rng), 'ehr/server', 1, T0 + dt_);
exp.append([f'TGT used {dt_} s after issue (new ticket request)', 'accepted', '-'])
    except KerberosError as e: exp.append([f'TGT used {dt_} s after issue (new ticket request)', 'rejected',
str(e)])
R['expiry'] = exp
# ---- E4 clock skew (client clock offset)
sk = []
for off in [-600, -301, -299, 0, 299, 301, 600]:
    k, s, c = fresh(); c.login(k, 'Tr0ub4dor&3-clinic', T0)
    try: c.get_ticket(k, 'ehr/server', T0 + 5 + off); tgs = 'accepted'
    except KerberosError as e: tgs = 'rejected: ' + str(e)
    ap = '-'
    if tgs == 'accepted':
        st, auth, ts = c.make_ap_req('ehr/server', T0 + 10 + off)
        try: s.ap_req(st, auth, T0 + 10); ap = 'accepted'
        except KerberosError as e: ap = 'rejected: ' + str(e)
    sk.append([off, tgs, ap])
R['skew'] = sk
# ---- E5 KDC unavailable
k, s, c = fresh(); out = []
k.online = False
try: c.login(k, 'Tr0ub4dor&3-clinic', T0); out.append(['1. KDC down before login', 'login', 'accepted', ''])
except KerberosError as e: out.append(['1. KDC down before login', 'login (AS exchange)', 'FAILS', str(e)])
k.online = True; c.login(k, 'Tr0ub4dor&3-clinic', T0); c.get_ticket(k, 'ehr/server', T0 + 1); k.online = False
```

```
st, auth, ts = c.make_ap_req('ehr/server', T0 + 20)
try: s.ap_req(st, auth, T0 + 20); out.append(['2. KDC goes down after the service ticket was obtained', 'use existing service ticket', 'WORKS', 'service never contacts the KDC'])
except KerberosError as e: out.append(['2.', 'use existing ticket', 'fails', str(e)])
try: c.get_ticket(k, 'ehr/server', T0 + 30); out.append(['3. KDC down', 'request a ticket for a NEW service (TGS)', 'accepted', ''])
except KerberosError as e: out.append(['3. KDC down, TGT still valid', 'request a ticket for a new service (TGS exchange)', 'FAILS', str(e)])
st, auth, ts = c.make_ap_req('ehr/server', T0 + 4000)
try: s.ap_req(st, auth, T0 + 4000); out.append(['4. KDC still down', 'existing ticket after it expires', 'accepted', ''])
except KerberosError as e: out.append(['4. KDC still down, ticket expired', 'existing ticket after its lifetime', 'FAILS', str(e)])
R['kdc_outage'] = out
# ---- E6 stolen tickets
k, s, c = fresh(); c.login(k, 'Tr0ub4dor&3-clinic', T0); c.get_ticket(k, 'ehr/server', T0 + 1)
stolen_st, k_cs, end = c.tickets['ehr/server']; atk = Counter()
for i in range(100):
    forged_auth = seal(rng.randbytes(32), {'c': 'alice', 'ts': T0 + 50 + i, 'seq': i}, rng)          # attacker has ONLY the ticket: guesses a session key
    try: s.ap_req(stolen_st, forged_auth, T0 + 50 + i); atk['accepted'] += 1
    except KerberosError as e: atk['rejected'] += 1
res_stolen = {'ticket_only_100_attempts': dict(atk)}
a2 = seal(k_cs, {'c': 'alice', 'ts': T0 + 200, 'seq': 99}, rng)                                    # attacker also stole the session key from the client's memory
try: s.ap_req(stolen_st, a2, T0 + 200); res_stolen['ticket_plus_session_key'] = 'IMPERSONATION SUCCEEDS (until ticket expiry)'
except KerberosError as e: res_stolen['ticket_plus_session_key'] = str(e)
a3 = seal(k_cs, {'c': 'alice', 'ts': T0 + 4000, 'seq': 100}, rng)
try: s.ap_req(stolen_st, a3, T0 + 4000); res_stolen['after_expiry'] = 'accepted'
except KerberosError as e: res_stolen['after_expiry'] = str(e)
R['stolen'] = res_stolen
```

```
# ---- E7 offline password guessing against the AS reply
WORDS = ['password', 'sunshine', 'letmein', 'welcome', 'dragon', 'monkey', 'qwerty', 'iloveyou', 'admin',
'hospital', 'clinic', 'nurse', 'doctor', 'secret', 'football', 'master', 'shadow', 'summer', 'winter', 'banana']
dictionary = [w + (str(i) if i else '') for w in WORDS for i in range(100)]
rng.shuffle(dictionary)
def crack(rep, user, iters, dic):
    t0 = time.perf_counter(); tries = 0
    for g in dic:
        tries += 1
        try: unseal(kdf(g, REALM + user, iters), rep); return g, tries, time.perf_counter() - t0
        except ValueError: pass
    return None, tries, time.perf_counter() - t0
crk = []
for label, pw in [('weak password "sunshine7"', 'sunshine7'), ('strong random password', 'Tr0ub4dor&3-
clinic')]:
    k = KDC(REALM, rng, 1000); k.add_user('bob', pw); rep, tgt = k.as_req('bob', 5, T0)                # no pre-
auth: ANY host can ask for this reply
    g, tries, dt_ = crack(rep, 'bob', 1000, dictionary); crk.append([label, 'found "%s"' % g if g else 'not found',
tries, dt_, tries / dt_])
R['offline'] = crk
rate = crk[0][4]
R['guess_rate'] = {'per_s': rate, 'space_8_lower': 26 ** 8, 'years_8_lower': 26 ** 8 / rate / 3.15e7,
'space_10_mixed': 62 ** 10, 'years_10_mixed': 62 ** 10 / rate / 3.15e7}
k = KDC(REALM, rng, 1000, preauth=True); k.add_user('bob', 'sunshine7')
try: k.as_req('bob', 5, T0); pa_unsolicited = 'AS-REP issued'
except KerberosError as e: pa_unsolicited = 'refused: ' + str(e)
sniffed = seal(kdf('sunshine7', REALM + 'bob', 1000), {'ts': T0}, rng)                    # a legitimate
pre-authentication blob seen on the network
g = None
for w in dictionary:
    try: unseal(kdf(w, REALM + 'bob', 1000), sniffed); g = w; break
    except ValueError: pass
```

```
R['preauth'] = {'unsolicited_request': pa_unsolicited, 'sniffed_preauth_cracked': g}
# slow KDF
t0 = time.perf_counter(); [kdf('x', 'salt', 1000) for _ in range(20)]; a = (time.perf_counter() - t0) / 20
t0 = time.perf_counter(); [kdf('x', 'salt', 100000) for _ in range(3)]; b = (time.perf_counter() - t0) / 3
R['kdf_cost'] = {'iters_1000_ms': a * 1000, 'iters_100000_ms': b * 1000}
# ---- E8 mutual authentication
k, s, c = fresh(); c.login(k, 'Tr0ub4dor&3-clinic', T0); c.get_ticket(k, 'ehr/server', T0 + 1)
good = bad = 0
for i in range(100):
    st, auth, ts = c.make_ap_req('ehr/server', T0 + 10 + i); _, rep = s.ap_req(st, auth, T0 + 10 + i); good +=
c.check_ap_rep('ehr/server', rep, ts)
    fake = seal(rng.randbytes(32), {'ts': ts + 1, 'service': 'ehr/server'}, rng); bad += c.check_ap_rep('ehr/server',
fake, ts)
R['mutual'] = {'trials': 100, 'genuine_service_accepted': good, 'impostor_service_accepted': bad}
json.dump(R, open('../out/c5.json', 'w'), indent=1, default=str)
print(json.dumps({k_: R[k_] for k_ in ['normal', 'password_on_wire', 'replay', 'tampered_auth', 'expiry',
'skew', 'kdc_outage', 'stolen', 'offline', 'guess_rate', 'preauth', 'kdf_cost', 'mutual']}, default=str)[:5200])
# ---- sequence diagram with key labels
sequence('../fig/c5_seq.png', 'Kerberos: AS, TGS and client-service exchanges (what is encrypted under
which key)', ['Client', 'Authentication\nServer (AS)', 'Ticket-Granting\nServer (TGS)', 'Service'],
        [(0, 1, '(1) AS-REQ: user, TGS name, nonce n1  [plaintext]', 'ok'), (1, 0, '(2) AS-REP: { K_c,tgs , n1 ,
end } K_c     +     TGT = { K_c,tgs , user , end } K_tgs', 'ok'),
         (0, 0, 'K_c = KDF(password): decrypts reply once, then discarded', 'note'),
         (0, 2, '(3) TGS-REQ: TGT, { user, ts } K_c,tgs , service, n2', 'ok'), (2, 0, '(4) TGS-REP: { K_c,s , n2 ,
end } K_c,tgs    +    ST = { K_c,s , user , end } K_s', 'ok'),
         (0, 3, '(5) AP-REQ: ST, authenticator { user, ts, seq } K_c,s', 'ok'), (3, 0, '(6) AP-REP: { ts + 1 } K_c,s
(mutual authentication)', 'ok'), (3, 3, 'service checks: ST (K_s), authenticator, skew, replay cache', 'note')],
        figw=10.0, dy=0.78)
```

run_c6.py

```
"""run_c6.py - Q6: collision-to-forgery, ElGamal, Schnorr, scheme comparison, key-relationship scaling
(toy parameters, synthetic messages)."""
```

```
import json, random, time, math
from collections import Counter
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
import sigs
from hashes import trunc_sha256
from ntcore import modexp, modinv, gcd
from flow import diagram
from seq import sequence
rng = random.Random(2036); R = {}
# ================= A. collision -> forgery over a truncated digest
GOOD = 'On behalf of the hospital network the finance office hereby authorises a payment of 100 euros to the supplier Alice Medical Ltd for invoice 4471 and confirms that no further payment is due'
EVIL = 'On behalf of the hospital network the finance office hereby authorises a payment of 90000 euros to the account of Mallory Trading Ltd for invoice 4471 and confirms that no further payment is due'
def variant(text, i, gaps):
    w = text.split(' '); out = w[0]
    for j in range(1, len(w)):
        out += ('  ' if (j <= gaps and (i >> (j - 1)) & 1) else ' ') + w[j]
    return out.encode()
def forge(k, gaps, key, ref=0):
    """Yuval-style birthday attack: 2^gaps whitespace variants of each message; find a benign/evil pair with the same k-bit digest."""
    t0 = time.perf_counter(); table = {}
    for i in range(1 << gaps): table.setdefault(trunc_sha256(variant(GOOD, i, gaps) + b' ref-%d' % ref, k), i)
    hashes = 1 << gaps
    for j in range(1 << gaps):
        hashes += 1
        d = trunc_sha256(variant(EVIL, j, gaps) + b' ref-%d' % ref, k)
        if d in table: return variant(GOOD, table[d], gaps) + b' ref-%d' % ref, variant(EVIL, j, gaps) + b' ref-%d' % ref, d, hashes, time.perf_counter() - t0
    return None
```

```
def sign_trunc(m, key, k): return modexp(trunc_sha256(m, k), key['d'], key['n'])
def verify_trunc(m, s, key, k): return modexp(s, key['e'], key['n']) == trunc_sha256(m, k)
Kt = sigs.rsa_keygen(128, rng)
rows = []; trace = None
for k, gaps, runs in [(24, 16, 10), (32, 19, 4)]:
    pass
for k, gaps, runs in [(24, 16, 10), (32, 19, 4)]:
    succ = 0; hs = []; secs = []
    for run in range(runs):
        # randomise the attack by choosing a random variant offset: shuffle which gap bits are used is unnecessary; rotate the benign text with a nonce word
        r = forge(k, gaps, Kt, ref=rng.randrange(10 ** 6))
        if r is None: continue
        g, e, d, h, sec = r; s = sign_trunc(g, Kt, k)                    # the victim signs the benign text only
        ok_good = verify_trunc(g, s, Kt, k); ok_evil = verify_trunc(e, s, Kt, k)
        succ += ok_good and ok_evil and g != e; hs.append(h); secs.append(sec)
        if trace is None and k == 32: trace = {'k': k, 'benign': g.decode(), 'evil': e.decode(), 'digest': hex(d), 'signature': hex(s), 'benign_verifies': ok_good, 'evil_verifies': ok_evil, 'hashes': h, 'seconds': sec, 'differ': g != e, 'gaps': gaps}
    rows.append([k, gaps, 1 << gaps, runs, succ, (sum(hs) / len(hs)) if hs else None, 2 ** (k // 2), 2 ** k])
R['forgery'] = {'rows': rows, 'trace': trace, 'rsa_bits': 128}
# contrast: second preimage (given an already signed message, find another with the same digest) costs ~2^k
def second_preimage(k, target_msg):
    target = trunc_sha256(target_msg, k); base = rng.getrandbits(64).to_bytes(8, 'big'); i = 0
    while True:
        i += 1
        if trunc_sha256(base + i.to_bytes(8, 'big'), k) == target: return i
sp = [second_preimage(20, GOOD.encode()) for _ in range(8)]
cl = []
for _ in range(8):
    seen = {}; base = rng.getrandbits(64).to_bytes(8, 'big'); i = 0
    while True:
```

```
        i += 1; d = trunc_sha256(base + i.to_bytes(8, 'big'), 20)
        if d in seen: break
        seen[d] = i
    cl.append(i)
R['sp_vs_collision'] = {'k': 20, 'second_preimage_mean': sum(sp) / len(sp), 'collision_mean': sum(cl) /
len(cl), 'ratio': (sum(sp) / len(sp)) / (sum(cl) / len(cl)), 'runs': 8}
# ================ B. ElGamal signatures
p, g, x = 23, 5, 6; y = pow(g, x, p); h = 7; k = 5; r = pow(g, k, p); s = pow(k, -1, p - 1) * (h - x * r) % (p - 1)
R['eg_hand'] = {'p': p, 'g': g, 'x': x, 'y': y, 'h': h, 'k': k, 'r': r, 'kinv': pow(k, -1, p - 1), 's': s, 'lhs': pow(g, h, p),
'rhs': pow(y, r, p) * pow(r, s, p) % p, 'ok': pow(g, h, p) == pow(y, r, p) * pow(r, s, p) % p,
                'altered_h': 8, 'altered_lhs': pow(g, 8, p), 'altered_ok': pow(g, 8, p) == pow(y, r, p) * pow(r, s, p)
% p}
EP = sigs.eg_params(rng, 64); xe, ye = sigs.eg_keygen(EP, rng); xe2, ye2 = sigs.eg_keygen(EP, rng)
cases = {c: 0 for c in ['valid', 'message altered', 'r altered', 's altered', 'r >= p (range check)', 'wrong public
key']}; N = 300
for i in range(N):
    m = ('invoice %d' % rng.getrandbits(32)).encode(); r_, s_ = sigs.eg_sign(EP, xe, m, rng)
    cases['valid'] += sigs.eg_verify(EP, ye, m, (r_, s_)); cases['message altered'] += sigs.eg_verify(EP, ye, m +
b'!', (r_, s_))
    cases['r altered'] += sigs.eg_verify(EP, ye, m, (r_ % (EP[0] - 1) + 1, s_)); cases['s altered'] +=
sigs.eg_verify(EP, ye, m, (r_, (s_ + 1) % (EP[0] - 1)))
    cases['r >= p (range check)'] += sigs.eg_verify(EP, ye, m, (r_ + EP[0], s_)); cases['wrong public key'] +=
sigs.eg_verify(EP, ye2, m, (r_, s_))
R['eg_matrix'] = {'trials': N, 'accepted': cases, 'p_bits': 64}
kk = next(k_ for k_ in range(10 ** 6, 2 * 10 ** 6) if gcd(k_, EP[0] - 1) == 1); m1, m2 = b'dose 5 mg', b'dose
50 mg'
a_, b_ = sigs.eg_sign(EP, xe, m1, rng, k=kk), sigs.eg_sign(EP, xe, m2, rng, k=kk); rec = sigs.eg_recover(EP,
ye, m1, a_, m2, b_)
R['eg_nonce'] = {'r_equal': a_[0] == b_[0], 'k_recovered': rec[0] == kk if rec else None, 'x_recovered': rec[1]
== xe if rec else None, 'forged_verifies': sigs.eg_verify(EP, ye, b'forged message', sigs.eg_sign(EP, rec[1],
b'forged message', rng)) if rec else None}
# small nonce range
```

```
kw = next(k_ for k_ in range(rng.randrange(1, 40000), 80000) if gcd(k_, EP[0] - 1) == 1); sw = sigs.eg_sign(EP, xe, m1, rng, k=kw); found = None
for kg in range(1, 2 ** 17):
    if modexp(EP[1], kg, EP[0]) == sw[0]: found = kg; break
hh = sigs.H_int(m1, EP[0] - 1); sols = sigs.solve_lin(sw[0] % (EP[0] - 1), (hh - found * sw[1]) % (EP[0] - 1), EP[0] - 1); xw = [xx for xx in sols if modexp(EP[1], xx, EP[0]) == ye]
R['eg_weak_rng'] = {'nonce_range': 2 ** 17, 'k_found': found == kw, 'x_recovered': bool(xw) and xw[0] == xe}
# existential forgery without hashing: choose a, b; r = g^a y^b, s = -r b^-1, h = a s
n_ = EP[0] - 1
while True:
    a2, b2 = rng.randrange(1, n_), rng.randrange(1, n_)
    if gcd(b2, n_) == 1: break
rf = modexp(EP[1], a2, EP[0]) * modexp(ye, b2, EP[0]) % EP[0]; sf = (-rf * modinv(b2, n_)) % n_; hf = a2 * sf % n_
R['eg_forgery'] = {'unhashed_verifies': sigs.eg_verify(EP, ye, b'', (rf, sf), hashed=False, m_int=hf),
'forged_h_bits': hf.bit_length(),
                   'hashed_verifies_for_any_text': sum(sigs.eg_verify(EP, ye, ('t%d' % i).encode(), (rf, sf)) for i in range(2000)), 'attempts': 2000}
# ================== C. Schnorr
p_, q_, g_, x_ = 23, 11, 2, 7; y_ = pow(g_, x_, p_); r0, c0 = 5, 4; t0 = pow(g_, r0, p_); s0 = (r0 + c0 * x_) % q_
R['sch_hand'] = {'p': p_, 'q': q_, 'g': g_, 'order_ok': pow(g_, q_, p_) == 1, 'x': x_, 'y': y_, 'r': r0, 't': t0, 'c': c0,
's': s0, 'lhs': pow(g_, s0, p_), 'rhs': t0 * pow(y_, c0, p_) % p_, 'ok': pow(g_, s0, p_) == t0 * pow(y_, c0, p_) % p_,
                 'wrong_s_ok': pow(g_, (s0 + 1) % q_, p_) == t0 * pow(y_, c0, p_) % p_}
SP = sigs.schnorr_params(rng, 256, 64); xs, ys = sigs.schnorr_keygen(SP, rng); xs2, ys2 = sigs.schnorr_keygen(SP, rng)
sc = {c: 0 for c in ['valid', 'message altered', 't altered', 's altered', 'wrong public key']}
for i in range(N):
    m = ('order %d' % rng.getrandbits(32)).encode(); t_, s_ = sigs.schnorr_sign(SP, xs, m, rng)
```

```
    sc['valid'] += sigs.schnorr_verify(SP, ys, m, (t_, s_)); sc['message altered'] += sigs.schnorr_verify(SP, ys, m + b'!', (t_, s_)); sc['t altered'] += sigs.schnorr_verify(SP, ys, m, (t_ + 1, s_))
    sc['s altered'] += sigs.schnorr_verify(SP, ys, m, (t_, (s_ + 1) % SP[1])); sc['wrong public key'] += sigs.schnorr_verify(SP, ys2, m, (t_, s_))
R['sch_matrix'] = {'trials': N, 'accepted': sc, 'p_bits': 256, 'q_bits': 64}
rr = rng.randrange(1, SP[1]); sg1 = sigs.schnorr_sign(SP, xs, b'dose 5 mg', rng, r=rr); sg2 = sigs.schnorr_sign(SP, xs, b'dose 50 mg', rng, r=rr)
c1, c2 = sigs.schnorr_challenge(sg1[0], b'dose 5 mg', SP[1]), sigs.schnorr_challenge(sg2[0], b'dose 50 mg', SP[1])
x_rec = (sg1[1] - sg2[1]) * modinv((c1 - c2) % SP[1], SP[1]) % SP[1]
R['sch_nonce'] = {'t_equal': sg1[0] == sg2[0], 'x_recovered': x_rec == xs, 'forged_verifies': sigs.schnorr_verify(SP, ys, b'forged', sigs.schnorr_sign(SP, x_rec, b'forged', rng))}
# predictable challenge: cheating prover without x (toy group order 11)
def cheat_known_challenge(c):
    s = rng.randrange(q_); t = pow(g_, s, p_) * pow(pow(y_, c, p_), -1, p_) % p_          # t = g^s y^-c
    return pow(g_, s, p_) == t * pow(y_, c, p_) % p_
TR = 5000; known = sum(cheat_known_challenge(rng.randrange(q_)) for _ in range(TR))
unk = 0
for _ in range(TR):
    s = rng.randrange(q_); guess_c = rng.randrange(q_); t = pow(g_, s, p_) * pow(pow(y_, guess_c, p_), -1, p_) % p_; c_real = rng.randrange(q_)      # verifier picks c AFTER seeing t
    unk += pow(g_, s, p_) == t * pow(y_, c_real, p_) % p_
R['sch_challenge'] = {'trials': TR, 'cheat_succeeds_when_c_known_in_advance': known, 'cheat_succeeds_when_c_chosen_after_t': unk, 'expected_unpredictable': TR / q_}
# ================ D. comparison data: verification operation counts, sizes
import ntcore, hybrid
def count_modexp(fn):
    calls = []; orig = sigs.modexp
    def w(b, e, m, cc=None): calls.append(e.bit_length()); return orig(b, e, m)
    sigs.modexp = w; fn(); sigs.modexp = orig; return calls
Kr = sigs.rsa_keygen(512, rng); sr = sigs.rsa_sign(b'm', Kr); DP = sigs.dsa_params(rng, 512, 128); xd, yd = sigs.dsa_keygen(DP, rng); sd = sigs.dsa_sign(DP, xd, b'm', rng); se = sigs.eg_sign(EP, xe, b'm', rng); ss =
```

```
sigs.schnorr_sign(SP, xs, b'm', rng)
R['verify_ops'] = {'RSA (e = 65537)': count_modexp(lambda: sigs.rsa_verify(b'm', sr, Kr['n'], Kr['e'])),
'DSA': count_modexp(lambda: sigs.dsa_verify(DP, yd, b'm', sd)), 'ElGamal': count_modexp(lambda:
sigs.eg_verify(EP, ye, b'm', se)), 'Schnorr': count_modexp(lambda: sigs.schnorr_verify(SP, ys, b'm', ss))}
# ================= E. key relationships
rel = []
for n in [10, 100, 1000, 10000, 1000000]:
    rel.append([n, n * (n - 1) // 2, n, n + 1, n])
R['relations'] = rel
json.dump(R, open('../out/c6.json', 'w'), indent=1, default=str)
print(json.dumps({k_: R[k_] for k_ in ['forgery', 'sp_vs_collision', 'eg_hand', 'eg_matrix', 'eg_nonce',
'eg_weak_rng', 'eg_forgery', 'sch_hand', 'sch_matrix', 'sch_nonce', 'sch_challenge', 'verify_ops']},
default=str)[:4800])
# ================= figures
diagram('../fig/c6_forgery.png', {'A': (1.3, 4.2, 'Attacker writes\nbenign text B and\nevil text E', 'red'), 'V':
(3.9, 4.2, '2^m whitespace\nvariants of each\n(same meaning)', 'yellow'), 'C': (6.5, 4.2, 'find B_i, E_j
with\nsame k-bit digest\n(about 2^(k/2) hashes)', 'yellow'),
     'S': (6.5, 2.4, 'victim reviews and\nsigns B_i : s = d(B_i)', 'blue'), 'F': (3.9, 2.4, 'attacker attaches s\nto
E_j', 'red'), 'X': (1.3, 2.4, 'verifier accepts\n(E_j, s) as signed', 'green')},
     [('A', 'V', ''), ('V', 'C', ''), ('C', 'S', 'B_i shown to victim'), ('S', 'F', 'signature s'), ('F', 'X', 'forged pair')],
title='Collision-to-forgery: the attacker must prepare both messages BEFORE the victim signs', size=(8.4,
3.3), w=2.1, h=0.95)
sequence('../fig/c6_schnorr.png', 'Schnorr identification: commitment, challenge, response', ['Prover (knows
x)', 'Verifier (knows y = g^x)'],
     [(0, 0, 'pick random r; commitment t = g^r mod p', 'note'), (0, 1, '1. commitment t', 'ok'), (1, 1, 'pick
unpredictable challenge c (after seeing t)', 'note'), (1, 0, '2. challenge c', 'ok'),
     (0, 0, 'response s = r + c x mod q', 'note'), (0, 1, '3. response s', 'ok'), (1, 1, 'accept iff g^s = t y^c (mod
p)', 'note')], figw=7.8, dy=0.7)
fig, ax = plt.subplots(figsize=(4.6, 3.2)); ns = [r[0] for r in rel]
ax.loglog(ns, [r[1] for r in rel], 'o-', label='pairwise shared keys  n(n-1)/2'); ax.loglog(ns, [r[2] for r in rel], 's-
', label='Kerberos: one key per user with the KDC'); ax.loglog(ns, [r[3] for r in rel], '^--', label='PKI: n key
pairs + 1 CA key')
```

```
ax.set_xlabel('users n'); ax.set_ylabel('keys / relationships to manage'); ax.legend(fontsize=7);
ax.grid(alpha=.3, which='both'); ax.set_title('Key relationships versus network size', fontsize=9)
plt.tight_layout(); plt.savefig('../fig/c6_scaling.png', dpi=170); plt.close()
```

run_c7.py

```
"""run_c7.py - Q7: validation of the firmware-update design against a bare-digest baseline and the two initial
proposals (toy data only)."""
import json, random, time, math
from collections import Counter
import matplotlib; matplotlib.use('Agg')
import matplotlib.pyplot as plt
import firmware as F, ecc
from firmware import *
from hashes import hmac_sha256, sha256, ct_equal
from flow import diagram
rng = random.Random(2037); R = {}
DAY = 86400; T = 1_800_000_000
mf = Manufacturer(rng)
signer = mf.make_signing_identity(1, T - 30 * DAY, T + 365 * DAY)
image10 = bytes(rng.randrange(256) for _ in range(20000)); image11 = bytes(rng.randrange(256) for _ in
range(20000)); image9 = bytes(rng.randrange(256) for _ in range(20000))
def new_pump(**kw): return Pump(mf.root_Q, version=kw.get('version', 10), time_floor=kw.get('floor', T -
DAY), epoch=kw.get('epoch', 1))
good = build_release(image11, 11, signer, issued_at=T)
old_valid = build_release(image9, 9, signer, issued_at=T - 60 * DAY)        # genuine, validly signed but
older version
def flip(b, i): x = bytearray(b); x[i // 8] ^= 1 << (i % 8); return bytes(x)
attacker_d, attacker_Q = ecc.ecdh_keypair(ecc.P256, rng)
rogue_cert, rogue_sig = pack_cert(999, 1, ROLE_SIGNER, attacker_Q, T - DAY, T + 365 * DAY), None
rogue_sig = sign_det(attacker_d, rogue_cert)                                # attacker's own "root-less" certificate
rogue_signer = {'d': attacker_d, 'Q': attacker_Q, 'cert': rogue_cert, 'cert_sig': rogue_sig, 'serial': 999, 'epoch':
1}
```

```
# ---------------- validation matrix: final design vs baseline
rows = []
def run(cid, desc, pkg, pump=None, baseline=None, expect='reject', clock=None, **kw):
    p = pump or new_pump(); ok, step, reason, cnt = p.verify_update(pkg, clock=clock, **kw)
    rows.append([cid, desc, baseline, 'ACCEPTED' if ok else 'rejected', step or '-', reason, (ok == (expect ==
'accept')), dict(cnt)])
base = baseline_publish(image11)
run('C1', 'Valid update (version 11)', good, baseline='accepted', expect='accept')
run('C2', 'One bit of the image modified', dict(good, image=flip(image11, 777)), baseline='rejected if digest
page untouched')
mal = baseline_publish(flip(image11, 777)); mal_ok = baseline_verify(mal)
run('C3', 'Attacker replaces image AND publishes a new digest / new manifest hash', dict(good,
image=flip(image11, 777), manifest=pack_manifest(b'INFPUMP1', 11, 0, T, len(image11), 1, signer['serial'],
sha256(flip(image11, 777)))), baseline='ACCEPTED (digest replaced too)' if mal_ok else 'rejected')
run('C4', 'One bit of the signature modified', dict(good, sig=flip(good['sig'], 100)), baseline='n/a (no
signature)')
run('C5', 'Manifest edited (version 11 -> 99) without re-signing', dict(good,
manifest=pack_manifest(b'INFPUMP1', 99, 0, T, len(image11), 1, signer['serial'], sha256(image11))),
baseline='n/a')
run('C6', 'Replay of an older GENUINE version (9) with its valid signature', old_valid,
baseline='ACCEPTED (old image + its own digest)' if baseline_verify(baseline_publish(image9)) else
'rejected')
exp_signer = mf.make_signing_identity(1, T - 400 * DAY, T - 20 * DAY)
run('C7', 'Update signed with an EXPIRED certificate (pump has seen time T - 1 day)',
build_release(image11, 11, exp_signer, issued_at=T - 30 * DAY), baseline='ACCEPTED (no validity)')
rl = mf.revocation_list(1, T, [signer['serial']])
run('C8', 'Certificate revoked by root-signed list carried with the update', dict(good, revocation=rl),
baseline='ACCEPTED (no revocation)')
run('C9', 'Manifest signed with attacker key but genuine certificate attached', dict(good,
sig=sign_det(attacker_d, good['manifest'])), baseline='ACCEPTED (anyone can publish a digest)')
run('C10', 'Attacker-made certificate (self-issued, not signed by the manufacturer root)',
build_release(image11, 11, rogue_signer, issued_at=T), baseline='ACCEPTED')
```

```
run('C11', 'Genuine update for a different device model', build_release(image11, 11, signer,
model=b'INFPUMP2', issued_at=T), baseline='ACCEPTED (no model binding)')
run('C12', 'Image truncated in transit', dict(good, image=image11[:-1]), baseline='rejected if digest page
untouched')
epoch2 = mf.make_signing_identity(2, T - 10 * DAY, T + 365 * DAY)
run('C13', 'Key-epoch downgrade: update signed by retired epoch-1 key (pump already at epoch 2)', good,
pump=new_pump(epoch=2), baseline='ACCEPTED')
stolen = mf.make_signing_identity(1, T - 30 * DAY, T + 365 * DAY)
run('C14', 'Stolen signing key; pump OFFLINE and has not received the revocation yet',
build_release(image11, 11, stolen, issued_at=T), baseline='ACCEPTED', expect='accept')
rl2 = mf.revocation_list(2, T, [stolen['serial']]); p15 = new_pump(); p15.verify_update(dict(good,
revocation=rl2), commit=True)   # pump processed a root-signed revocation list earlier
run('C15', 'Stolen signing key; pump has received the root-signed revocation list', build_release(image11, 12,
stolen, issued_at=T + 1), pump=p15, baseline='ACCEPTED')
R['matrix'] = rows
R['matrix_summary'] = {'cases': len(rows), 'as_expected': sum(r[6] for r in rows)}
# ---------------- clock behaviour: wall-clock validity (initial proposal A) versus signed time floor (final)
clock_rows = []
for label, clk in [('correct clock', T), ('clock reset to 1970 (battery loss)', 0), ('clock +10 years', T + 10 * 365 *
DAY), ('clock set back 1 year', T - 365 * DAY)]:
    legit = new_pump().verify_update(good, clock=clk, use_floor=False, commit=False)[0]; legit_f =
new_pump().verify_update(good, clock=clk, use_floor=True, commit=False)[0]
    exp_pkg = build_release(image11, 11, exp_signer, issued_at=T - 30 * DAY)
    expd = new_pump().verify_update(exp_pkg, clock=clk, use_floor=False, commit=False)[0]; expd_f =
new_pump().verify_update(exp_pkg, clock=clk, use_floor=True, commit=False)[0]
    clock_rows.append([label, 'accepted' if legit else 'REJECTED (update blocked)', 'accepted' if legit_f else
'REJECTED', 'ACCEPTED (expired key works)' if expd else 'rejected', 'ACCEPTED' if expd_f else
'rejected'])
R['clock'] = clock_rows
# ---------------- offline pump: the whole verification needs no network, only the pinned root key
p = new_pump(); ok, step, reason, cnt = p.verify_update(dict(good, revocation=mf.revocation_list(3, T,
[12345])))
```

```
R['offline'] = {'accepted': ok, 'network_messages_needed': 0, 'ecdsa_verifications': cnt['ecdsa_verify'],
'revocation_list_learned': sorted(p.revoked), 'rl_seq': p.rl_seq}
# ---------- deterministic nonces
msgs = [pack_manifest(b'INFPUMP1', v, 0, T + v, 1000 + v, 1, 101, sha256(bytes([v % 256]) * 8)) for v in
range(1, 1001)]
sigs_ = [sign_det(signer['d'], m) for m in msgs]
rs = {s[:32] for s in sigs_}; same = sign_det(signer['d'], msgs[0]) == sign_det(signer['d'], msgs[0])
n_ = ecc.P256.n; z1, z2 = ecc._h(msgs[0], n_), ecc._h(msgs[1], n_); (r1, s1), (r2, s2) = sig_from(sigs_[0]),
sig_from(sigs_[1])
from ntcore import modinv
k_try = (z1 - z2) * modinv((s1 - s2) % n_, n_) % n_; d_try = (s1 * k_try - z1) * modinv(r1, n_) % n_
R['det_nonce'] = {'signatures': len(sigs_), 'distinct_r': len(rs), 'same_message_same_signature': same,
'reuse_attack_recovers_key': d_try == signer['d'], 'verify_all': all(verify(signer['Q'], m, s) for m, s in
zip(msgs[:20], sigs_[:20]))}
# ---------- mutual authentication
dev_d, dev_Q = ecc.ecdh_keypair(ecc.P256, rng); srv_d, srv_Q = ecc.ecdh_keypair(ecc.P256, rng)
dc, dcs = mf.issue(ROLE_DEVICE, dev_Q, 1, T - DAY, T + 3650 * DAY); sc_, scs =
mf.issue(ROLE_SERVER, srv_Q, 1, T - DAY, T + 3650 * DAY)
dev = {'d': dev_d, 'cert': dc, 'cert_sig': dcs}; srv = {'d': srv_d, 'cert': sc_, 'cert_sig': scs}
auth = []
cn = Counter(); auth.append(['Valid mutual authentication', *authenticate(dev, srv, mf.root_Q, rng, cnt=cn),
dict(cn)])
def rec(label, **kw): c = Counter(); r = authenticate(**kw, cnt=c); auth.append([label, *r, dict(c)])
# replay: record a genuine pump response for one session and present it in a later session
captured = sign_det(dev_d, auth_context(b'\0' * 16, b'\1' * 16, b'PUMP-0001', b'HOSPITAL-SRV', b'P'))
rec('Replay of an old pump response in a new session', pump_ident=dev, server_ident=srv,
root_Q=mf.root_Q, rng=rng, replay_response=captured)
wrong_d, wrong_Q = ecc.ecdh_keypair(ecc.P256, rng)
rec('Pump uses a key that does not match its certificate', pump_ident=dict(dev, d=wrong_d),
server_ident=srv, root_Q=mf.root_Q, rng=rng)
rec('Revoked device certificate', pump_ident=dev, server_ident=srv, root_Q=mf.root_Q, rng=rng,
revoked={unpack_cert(dc)['serial']})
```

```
fake_c = pack_cert(1, 1, ROLE_SERVER, attacker_Q, T - DAY, T + DAY)
rec('Impostor server with a self-made certificate', pump_ident=dev, server_ident={'d': attacker_d, 'cert':
fake_c, 'cert_sig': sign_det(attacker_d, fake_c)}, root_Q=mf.root_Q, rng=rng)
R['auth'] = auth
# ---------------- costs
cost = []
t0 = time.perf_counter(); pk = build_release(image11, 11, signer, issued_at=T); t_sign = time.perf_counter()
- t0
for size in [64 * 1024, 1024 * 1024, 16 * 1024 * 1024]:
    img = bytes(rng.randrange(256) for _ in range(1024)) * (size // 1024)
    pkg = build_release(img, 11, signer, issued_at=T); p = new_pump()
    t0 = time.perf_counter(); ok, st_, rs_, cnt = p.verify_update(pkg); t_ver = time.perf_counter() - t0
    t0 = time.perf_counter(); sha256(img); t_h = time.perf_counter() - t0
    cost.append([size, cnt['sha256_blocks'], cnt['ecdsa_verify'], t_h * 1000, (t_ver - t_h) * 1000, t_ver * 1000,
metadata_bytes(pkg), metadata_bytes(pkg) / size * 100, ok])
R['cost'] = cost; R['sign_ms'] = t_sign * 1000
ecdsa_meta = MANIFEST_LEN + 64 + CERT_LEN + 64; rsa_meta = MANIFEST_LEN + 256 + (4 + 2 + 1
+ 259 + 16) + 256
R['alt_designs'] = [['ECDSA P-256 (chosen)', ecdsa_meta, 2, '65 B root key in ROM', 'private key in HSM;
deterministic nonces'], ['RSA-2048 PKCS#1/PSS', rsa_meta, 2, '259 B root key in ROM', 'cheap verification
(17 multiplications); larger metadata'], ['HMAC with shared fleet key', MANIFEST_LEN + 32, 1, '16-32 B
secret in EVERY pump', 'no non-repudiation; one extracted key forges updates for the whole fleet']]
# ---------------- initial proposals simulated for the comparison record
# Proposal A: one long-term ECDSA key embedded in all pumps, signature over SHA-256(image), validity
by wall clock, no version check
def prop_a_sign(image, d, valid_until): return {'image': image, 'sig': sign_det(d, sha256(image) +
valid_until.to_bytes(8, 'big')), 'valid_until': valid_until}
def prop_a_verify(pkg, Q, clock): return clock <= pkg['valid_until'] and verify(Q, sha256(pkg['image']) +
pkg['valid_until'].to_bytes(8, 'big'), pkg['sig'])
pa = prop_a_sign(image11, signer['d'], T + 30 * DAY); pa_old = prop_a_sign(image9, signer['d'], T + 30 *
DAY)
A = {'valid': prop_a_verify(pa, signer['Q'], T), 'modified_image': prop_a_verify(dict(pa,
```

```
image=flip(image11, 5)), signer['Q'], T), 'rollback_old_genuine': prop_a_verify(pa_old, signer['Q'], T),
'wrong_clock_1970': prop_a_verify(pa, signer['Q'], 0), 'wrong_clock_after_expiry_set_back':
prop_a_verify(pa, signer['Q'], T - 5 * DAY), 'revocation_possible': False}
# Proposal B: HMAC with a fleet-wide key + Kerberos-style tickets for server authentication
fleet = rng.randbytes(16)
def prop_b_tag(image): return hmac_sha256(fleet, image)
B = {'valid': ct_equal(prop_b_tag(image11), prop_b_tag(image11)), 'modified_image_without_key':
ct_equal(prop_b_tag(flip(image11, 5)), prop_b_tag(image11)),
    'attacker_with_key_extracted_from_ONE_pump_forges_for_all': ct_equal(hmac_sha256(fleet, b'EVIL' +
image11), prop_b_tag(b'EVIL' + image11)),
    'manufacturer_can_prove_it_issued_update': False,
'pump_or_hospital_can_forge_a_tag_manufacturer_never_made': True, 'offline_pump_can_verify': True,
'needs_kdc_for_server_auth': True}
R['proposals'] = {'A': A, 'B': B}
# ---------------- attack effort (Student A): work in bits and years
def yrs(bits, rate): return 2.0 ** bits / rate / 3.156e7
effort = [['Collision in SHA-256 digest (birthday)', 128, 'Q1: measured 1.25 x 2^(k/2) for k = 16..36'],
['Second pre-image of SHA-256 (forge image with the same digest)', 256, 'Q1: measured 2^k for fixed
target'], ['Forge ECDSA P-256 signature (solve ECDLP, rho method)', 128, 'CO2 Q5: generic sqrt(n)
attacks'],
        ['Factor RSA-2048 (alternative design)', 112, 'NIST SP 800-57 estimate'], ['Collision in a 64-bit
digest', 64, 'Q1 extrapolation: 2^32 hashes'], ['Collision in a 32-bit truncated digest (demonstrated Q6)', 16,
'measured: about 5 x 10^5 hashes with whitespace variants'],
        ['Guess a valid 256-bit tag/signature at random', 256, 'success probability 2^-256 per attempt']]
R['effort'] = [[a, b, yrs(b, 1e12), yrs(b, 1e18), c] for a, b, c in effort]
json.dump(R, open('../out/c7.json', 'w'), indent=1, default=str)
print(json.dumps({k: R[k] for k in ['matrix_summary', 'clock', 'offline', 'det_nonce', 'auth', 'cost', 'sign_ms',
'alt_designs', 'proposals']}, default=str)[:5200])
for r in rows: print(r[0], '|', r[3], '|', r[4], '|', r[5][:70], '|', r[6])
# ---------------- figures
diagram('../fig/c7_arch.png', {'ROOT': (1.7, 4.2, 'Manufacturer ROOT key\n(offline HSM, 2 operators)\nroot
public key burned\ninto every pump ROM', 'red'), 'SIGN': (5.0, 4.2, 'Release-signing key + cert\n(epoch e, 1-
```

year validity)\nHSM, deterministic ECDSA', 'yellow'), 'REV': (8.3, 4.2, 'Root-signed revocation list\n(sequence number, epoch bump)', 'red'),

'REL': (5.0, 2.5, 'Signed release package:\nmanifest + signature + cert + image', 'blue'), 'DIST': (1.7, 2.5, 'Distribution (untrusted):\nweb / hospital server /\nUSB - any channel', 'grey'), 'PUMP': (8.3, 2.5, 'Pump: pinned root key, version counter,\nkey-epoch floor, time floor, revoked list', 'green'),

'SRV': (8.3, 0.9, 'Hospital server\n(mutual authentication with\ndevice certificate)', 'blue'), 'DEV': (4.6, 0.9, 'Device key + certificate\n(per pump, issued by root)', 'yellow')},

[('ROOT', 'SIGN', 'certifies'), ('ROOT', 'REV', 'signs'), ('SIGN', 'REL', 'signs manifest'), ('REL', 'DIST', ''), ('DIST', 'PUMP', 'download'), ('REV', 'PUMP', 'rides with updates'), ('DEV', 'SRV', 'challenge-response'), ('PUMP', 'SRV', '')],

title='Key and certificate architecture of the final design', size=(9.0, 4.0), w=2.9, h=1.0)

steps = ['1 format +\nsizes', '2 device\nmodel', '3 cert signed\nby root', '4 validity vs\nsigned time floor', '5 revocation:\nserial + epoch', '6 manifest\nbinds cert', '7 manifest\nsignature', '8 version >\ninstalled', '9 image size +\nSHA-256']

nodes = {f's{i}': ((1.1 + 1.95 * i) if i < 5 else (9.0 - 1.95 * (i - 5)), 4.0 if i < 5 else 2.4, s, 'blue' if i < 5 else 'yellow') for i, s in enumerate(steps)}

nodes['ok'] = (1.6, 0.8, 'COMMIT atomically:\nversion, epoch, time floor', 'green'); nodes['bad'] = (6.2, 0.8, 'REJECT: keep the current\nimage, log step and reason', 'red')

diagram('../fig/c7_verify.png', nodes, [(f's{i}', f's{i + 1}', 'ok') for i in range(8)] + [('s8', 'ok', 'ok'), ('s6', 'bad', 'any failure')], title='Pump verification procedure (implemented in firmware.py: Pump.verify_update)', size=(9.0, 3.6), w=1.75, h=0.85)

fig, ax = plt.subplots(1, 2, figsize=(9, 3.3))

lab = [e[0].split('(')[0][:34] for e in R['effort'][:6]]; bits = [e[1] for e in R['effort'][:6]]

ax[0].barh(range(len(bits)), bits, color=['#2e7d32' if b >= 112 else '#c0392b' for b in bits]);

ax[0].set_yticks(range(len(bits))); ax[0].set_yticklabels(lab, fontsize=6.5); ax[0].invert_yaxis()

ax[0].set_xlabel('attacker work (bits, 2^x operations)'); ax[0].axvline(80, c='k', ls=':');

ax[0].set_title('Attacker effort by goal', fontsize=9)

sz = [r[0] / 1024 for r in cost]; ax[1].loglog(sz, [r[5] for r in cost], 'o-', label='verify total'); ax[1].loglog(sz, [r[3] for r in cost], 's--', label='SHA-256 of image'); ax[1].set_xlabel('image size (KB)'); ax[1].set_ylabel('ms (own Python)'); ax[1].legend(fontsize=7); ax[1].grid(alpha=.3, which='both'); ax[1].set_title('Verification cost vs image size', fontsize=9)

plt.tight_layout(); plt.savefig('../fig/c7_attack.png', dpi=170); plt.close()

Appendix F – Regression tests test_c3.py

test_c3.py

```
"""test_c3.py - quick regression tests for the CO3 code base (run: python test_c3.py)."""
import random, hmac as pyhmac, hashlib, datetime as dt
import hashes as H, sigs, kerberos as K, firmware as F, ecc
from x509chain import *
rng = random.Random(1); T = []
def t(name, cond): T.append((name, bool(cond))); print(('PASS  ' if cond else 'FAIL  ') + name)
# Q1 / Q2
t('byte-sum: swapping two bytes keeps the digest (second pre-image)', H.sum_hash(b'AB') ==
H.sum_hash(b'BA'))
t('XOR-fold flips exactly one bit for a one-bit change', bin(int.from_bytes(H.xorfold_hash(b'\x00\x00'), 'big')
^ int.from_bytes(H.xorfold_hash(b'\x00\x01'), 'big')).count('1') == 1)
t('truncated SHA-256 is the leading k bits', H.trunc_sha256(b'abc', 16) ==
int.from_bytes(hashlib.sha256(b'abc').digest()[:2], 'big'))
t('toy MD padding makes the length a multiple of 8', all((n + len(H.md_pad(n))) % 8 == 0 for n in
range(100)))
t('HMAC-SHA256 equals Python hmac (short and long keys)', all(H.hmac_sha256(k, b'msg') ==
pyhmac.new(k, b'msg', hashlib.sha256).digest() for k in (b'k', b'x' * 100)))
key, msg = b'SECRETKEY', b'amount=100&to=alice'; tag = H.naive_mac(key, msg); fm, ft =
H.length_extend(msg, tag, b'&to=mallory', len(key))
t('length extension forges a tag for H(key||m) without the key', H.naive_mac(key, fm) == ft)
fm2, ft2 = H.length_extend(msg, H.hmac_toy(key, msg), b'&to=mallory', len(key))
t('the same extension fails against HMAC', H.hmac_toy(key, fm2) != ft2)
# Q3
K1 = sigs.rsa_keygen(256, rng); s = sigs.rsa_sign(b'm', K1)
t('RSA hash-then-sign verifies and rejects a changed message', sigs.rsa_verify(b'm', s, K1['n'], K1['e']) and
not sigs.rsa_verify(b'n', s, K1['n'], K1['e']))
par = sigs.dsa_params(rng, 256, 64); x, y = sigs.dsa_keygen(par, rng); sg = sigs.dsa_sign(par, x, b'm', rng)
t('DSA verifies; changed message and wrong key rejected', sigs.dsa_verify(par, y, b'm', sg) and not
sigs.dsa_verify(par, y, b'n', sg) and not sigs.dsa_verify(par, sigs.dsa_keygen(par, rng)[1], b'm', sg))
```

```
a = sigs.dsa_sign(par, x, b'one', rng, k=12345); b = sigs.dsa_sign(par, x, b'two', rng, k=12345)
t('DSA nonce reuse reveals the private key', sigs.dsa_recover(par, b'one', a, b'two', b)[1] == x)
n, e, d = 3233, 17, 2753; m1, m2 = 123, 45
t('textbook RSA multiplicative forgery verifies', pow(pow(m1, d, n) * pow(m2, d, n) % n, e, n) == m1 * m2
% n)
# Q4
now = dt.datetime(2026, 11, 6, tzinfo=UTC); D = lambda k: dt.timedelta(days=k); g = lambda:
ec.generate_private_key(ec.SECP256R1())
rk, ik, lk = g(), g(), g()
root = make_cert('Root', rk.public_key(), 'Root', rk, 1, now - D(900), now + D(900), ca=True, pathlen=1,
key_usage=ku(cert_sign=True, crl_sign=True))
inter = make_cert('Inter', ik.public_key(), 'Root', rk, 2, now - D(800), now + D(800), ca=True, pathlen=0,
key_usage=ku(cert_sign=True, crl_sign=True))
leaf = make_cert('a.test', lk.public_key(), 'Inter', ik, 3, now - D(5), now + D(50),
key_usage=ku(digital_signature=True), san=['a.test'], eku=[ExtendedKeyUsageOID.SERVER_AUTH])
t('X.509 chain validates', validate_chain([leaf, inter, root], [root], 'a.test', now, check_revocation=False)[0])
t('hostname mismatch rejected at step 6', validate_chain([leaf, inter, root], [root], 'b.test', now,
check_revocation=False)[1] == '6 name match')
t('expired time rejected at step 3', validate_chain([leaf, inter, root], [root], 'a.test', now + D(100),
check_revocation=False)[1] == '3 validity period')
t('untrusted root rejected at step 8', validate_chain([leaf, inter, root], [], 'a.test', now,
check_revocation=False)[1] == '8 trust anchor')
# Q5
kd = K.KDC('R', rng); kd.add_user('al', 'pw-ok'); ks = kd.add_service('svc'); sv = K.Service('svc', ks); cl =
K.Client('al', rng, 'R')
cl.login(kd, 'pw-ok', 1000); cl.get_ticket(kd, 'svc', 1001); st, au, ts = cl.make_ap_req('svc', 1002); u, rep =
sv.ap_req(st, au, 1002)
t('Kerberos flow + mutual authentication', u == 'al' and cl.check_ap_rep('svc', rep, ts))
try: sv.ap_req(st, au, 1003); r = False
except K.KerberosError: r = True
t('replayed authenticator rejected by the replay cache', r)
try: sv.ap_req(st, K.seal(rng.randbytes(32), {'c': 'al', 'ts': 1004, 'seq': 9}, rng), 1004); r = False
```

```
except K.KerberosError: r = True
t('ticket without the session key cannot make an authenticator', r)
# Q6
p_, q_, g_, x_ = 23, 11, 2, 7; y_ = pow(g_, x_, p_); t_ = pow(g_, 5, p_); s_ = (5 + 4 * x_) % q_
t('Schnorr hand example (p=23, q=11, g=2, x=7) verifies', pow(g_, s_, p_) == t_ * pow(y_, 4, p_) % p_)
t('ElGamal hand example (p=23, g=5, x=6, h=7, k=5) verifies', pow(5, 7, 23) == pow(8, 20, 23) * pow(20,
17, 23) % 23)
SP = sigs.schnorr_params(rng, 128, 32); xs, ys = sigs.schnorr_keygen(SP, rng); ss = sigs.schnorr_sign(SP,
xs, b'm', rng)
t('Schnorr signature verifies and rejects a changed message', sigs.schnorr_verify(SP, ys, b'm', ss) and not
sigs.schnorr_verify(SP, ys, b'n', ss))
# Q7
mf = F.Manufacturer(rng); T0 = 1_800_000_000; sg = mf.make_signing_identity(1, T0 - 86400, T0 + 86400
* 300); pump = F.Pump(mf.root_Q, version=10, time_floor=T0 - 86400)
img = bytes(rng.randrange(256) for _ in range(5000)); pkg = F.build_release(img, 11, sg, issued_at=T0)
t('firmware: valid update accepted', pump.verify_update(pkg, commit=False)[0])
t('firmware: modified image rejected', not pump.verify_update(dict(pkg, image=img[:-1] + b'x'),
commit=False)[0])
pump.verify_update(pkg)
t('firmware: rollback (same/older version) rejected', not pump.verify_update(pkg)[0])
t('firmware: deterministic signatures are repeatable', F.sign_det(sg['d'], b'x') == F.sign_det(sg['d'], b'x'))
print(f'\n{sum(x for _, x in T)} of {len(T)} tests passed')
```

Appendix G – Execution instructions and program output

Environment : Python 3.12; packages: numpy, matplotlib (figures), cryptography >= 42 (X.509 objects, cross-checks)

Layout : all files of Appendices A-F plus ntcore.py, rsa.py, ecc.py, symmetric.py, dh.py, hybrid.py, flow.py, seq.py in one folder,

with sibling folders ../out (result .json files) and ../fig (figures)

Run order : python run_c1.py ; python recheck_c1.py ; python run_c2.py ; python run_c3.py ; python run_c4.py

python run_c5.py ; python run_c6.py ; python run_c7.py (each writes ../out/cN.json and its

figures)

Tests : python test_c3.py (25 checks, finishes in a few seconds)

Seeds : fixed inside each driver (random.Random(2031) ... 2037), so counts and tables repeat exactly; only the timing columns vary

Approx. time: run_c1 about 1 min, run_c2 10 s, run_c3 30 s, run_c4 10 s, run_c5 10 s, run_c6 30 s, run_c7 1 min on a laptop

Output of `python test_c3.py`:

PASS byte-sum: swapping two bytes keeps the digest (second pre-image)

PASS XOR-fold flips exactly one bit for a one-bit change

PASS truncated SHA-256 is the leading k bits

PASS toy MD padding makes the length a multiple of 8

PASS HMAC-SHA256 equals Python hmac (short and long keys)

PASS length extension forges a tag for H(key||m) without the key

PASS the same extension fails against HMAC

PASS RSA hash-then-sign verifies and rejects a changed message

PASS DSA verifies; changed message and wrong key rejected

PASS DSA nonce reuse reveals the private key

PASS textbook RSA multiplicative forgery verifies

PASS X.509 chain validates

PASS hostname mismatch rejected at step 6

PASS expired time rejected at step 3

PASS untrusted root rejected at step 8

PASS Kerberos flow + mutual authentication

PASS replayed authenticator rejected by the replay cache

PASS ticket without the session key cannot make an authenticator

PASS Schnorr hand example (p=23, q=11, g=2, x=7) verifies

PASS ElGamal hand example (p=23, g=5, x=6, h=7, k=5) verifies

PASS Schnorr signature verifies and rejects a changed message

PASS firmware: valid update accepted

PASS firmware: modified image rejected

PASS firmware: rollback (same/older version) rejected

PASS firmware: deterministic signatures are repeatable

25 of 25 tests passed

The raw measurements of every experiment are stored as JSON files (c1.json – c7.json) written by the drivers in Appendix E; every table in this report was generated directly from them.